

LabelScan — Le cours complet

Backend FastAPI/PostgreSQL et mobile Expo/React Native, expliqués de bout en bout

Préparé pour Brice Fontaine

13 juillet 2026

Table des matières

Partie I — Vue d'ensemble	4
1. Ce que fait LabelScan	4
2. Les concepts fondamentaux (à maîtriser avant le code)	4
3. Le cycle de vie d'un scan, de bout en bout	7
4. La machine à 12 états d'une ingestion	7
5. Comment lire ce cours	7
Partie II — Backend	8
6. Outillage et configuration du projet	8
7. La couche platform/ — l'infrastructure partagée	11
7.1 platform/db/engine.py — la connexion DB	11
7.2 platform/db/audit_context.py — le contexte d'audit transactionnel	11
7.3 platform/observability.py — logs JSON structurés	12
7.4 platform/http/middleware.py — corrélation	13
7.5 platform/http/errors.py — erreurs RFC 9457 (problem+json)	13
7.6 platform/http/jwt.py — le codec JWT	15
7.7 platform/http/security.py — la porte d'authentification	16
7.8 platform/http/deps.py et read_models.py	16
8. L'outbox : platform/outbox/	18
8.1 message.py	18
8.2 backoff.py — les maths de retry, pures	18
8.3 worker.py — le relais transactionnel, ligne par ligne	19
9. Contexte ingestion — le domaine (pur)	22
9.1 domain/status.py	22
9.2 domain/gsl.py — le parseur de codes-barres GS1	22
9.3 domain/extraction.py — le gate anti-fabrication	24
9.4 domain/interim_fields.py — previews déterministes (regex)	27
9.5 domain/reconciliation.py — fusion GS1 ↔ LLM	28
10. Contexte ingestion — la couche application	30
10.1 application/ports.py — les ports	30
10.2 application/submit_ingestion.py — le cas d'usage d'upload	31
10.3 application/extraction_ports.py — les ports providers	31
10.4 application/override_field.py — la correction humaine (HITL)	32
10.5 application/confirm_ingestion.py — finaliser la revue	33
11. Contexte ingestion — adapters (1/2) : stockage et providers	34
11.1 adapters/filesystem_raw_store.py — le store d'objets crash-durable	34
11.2 adapters/sql_ingestion_repository.py — l'écriture durable et idempotente	35
11.3 adapters/google_vision_ocr.py — l'adapter OCR	35
11.4 adapters/claude_llm_provider.py — l'adapter LLM	37
12. Contexte ingestion — adapters (2/2) : le consumer d'extraction, les repos de revue, les routeurs HTTP	40
12.1 adapters/extraction_consumer.py — l'orchestrateur (923 lignes)	40

12.2	adapters/sql_field_override_repository.py — l'override append-only	43
12.3	adapters/sql_confirm_repository.py	44
12.4	adapters/http/router.py — le routeur d'écriture	45
12.5	adapters/http/read_router.py — lectures + long-poll	46
13.	Traceability, HACCP, Identity — les autres contextes	48
13.1	traceability/domain/consistency.py — vérité métier pure	48
13.2	traceability/adapters/registration_consumer.py	48
13.3	haccp/domain/ — règles et machine à états pures	49
13.4	haccp/application/alert_service.py + adapters/sql_alert_repository.py	50
13.5	haccp/adapters/alerting_consumer.py et temperature_recorder.py	50
13.6	Le contexte identity	51
14.	La composition root : app/	53
14.1	app/http_app.py	53
14.2	Les wirings	53
14.3	app/worker_runtime.py — l'entrypoint worker + heartbeat	54
14.4	app/ops_router.py — health et version	54
15.	Le modèle de données et les migrations	56
15.1	0001_schemas_and_roles — schémas et moindre privilège	56
15.2	0002_audit_immutability_foundations — le socle d'intégrité	56
15.3	0003_harden_security_definer — durcissement	57
15.4	Les migrations suivantes, en bref	57
16.	Les tests backend	59
16.1	conftest.py — les fixtures	59
16.2	_fakes.py — les doublures de providers	59
16.3	Les tests comme preuves	60
Partie III —	L'application mobile (Expo / React Native / TypeScript)	61
17.	Fondations : entrée, config, thème, navigation, auth	62
17.1	index.ts et App.tsx	62
17.2	src/config.ts — la leçon de sécurité EXPO_PUBLIC	63
17.3	Le thème (src/theme/)	63
17.4	navigation/RootNavigator.tsx	63
17.5	context/AuthContext.tsx	64
17.6	services/authStorage.ts — le token en lieu sûr	65
17.7	services/auth.ts et l'écran de login	65
18.	Le client API : services/api.ts	67
18.1	ApiError — l'erreur typée côté client	67
18.2	send<T> — le cœur commun	67
18.3	apiRequest et apiUpload	68
18.4	types/api.ts et types/Article.ts	69
19.	Le pipeline mobile : outbox locale, soumission, polling, scan queue	70
19.1	services/outbox.ts — la file durable côté client	70
19.2	services/ingestionSubmit.ts — exécuter un op de capture	72
19.3	services/ingestionPolling.ts — le poll borné et annulable	72
19.4	services/ingestionResult.ts — du poll au résultat de revue	74
19.5	services/scanQueue.ts — la machine à états du workflow (le chef d'orchestre)	74
19.6	services/outboxDrain.ts — le vidage au premier plan	78
19.7	hooks/useScanQueue.ts	79
20.	Les services métier purs du mobile	80
20.1	services/gsl.ts — le miroir client du parseur GS1	80
20.2	services/inputMasks.ts — masques de saisie et validation neutre	80
20.3	fieldOrder.ts, fieldLabels.ts, fieldCompleteness.ts	82
20.4	fieldHistory.ts et allergenSuggestions.ts — l'aide à la saisie, sans fabrication	82

20.5	faoDisplay.ts, articleSearch.ts, articleGrouping.ts, calendar.ts, dates.ts	83
20.6	Le stockage des articles : port + adapters (audit §7.2)	84
21.	Les écrans	86
21.1	CameraScreen.tsx — le viseur à capture enchaînée	86
21.2	ReviewScreen.tsx — la revue/édition (957 lignes)	88
21.3	ArticleListScreen.tsx — l'accueil	90
21.4	ArticleDetailScreen.tsx — la fiche produit	91
21.5	Les composants notables	91
22.	Les tests mobile (Jest)	92
	Partie IV — Synthèse	93
23.	Les 20 tips dev à emporter	94
24.	Checklists d'extension	96
25.	Glossaire	97

Partie I — Vue d'ensemble

1. Ce que fait LabelScan

LabelScan est un système de **traçabilité HACCP pour produits de la mer** (poissonnerie / criée). Le flux métier complet :

1. L'opérateur **photographie une étiquette** de produit avec l'app mobile (Expo / React Native). La caméra lit aussi le **code-barres GS1** nativement.
2. La photo est **envoyée au backend** (FastAPI) qui la stocke durablement, puis répond 202 Accepted — l'opérateur continue de scanner sans attendre.
3. Un **worker asynchrone** prend le relais : OCR (Google Vision) → extraction structurée par **LLM** (Claude Haiku) → **gate anti-fabrication** (le point central du système : aucune valeur non prouvée par le texte OCR n'est stockée) → **réconciliation GS1** (le code-barres, mathématiquement exact, gagne sur le LLM).
4. Le mobile **poll** le statut, affiche les champs au fur et à mesure (GS1 à T+0, préviews regex, puis le run LLM), l'opérateur **valide/corrige les 17 champs** et enregistre l'arrivage.
5. Les extractions validées alimentent la **traçabilité** (lots, fournisseurs, produits) qui alimente le **HACCP** (alertes DLC, température) — le tout par **événements**, jamais par appels directs.
6. Chaque écriture métier est **auditée de façon incontournable** par des triggers PostgreSQL : impossible d'écrire une ligne sans déclarer qui, quoi, pourquoi.

Les deux moitiés du dépôt

Répertoire	Techno	Rôle
server/	Python 3.13, FastAPI, SQLAlchemy 2.0 Core, PostgreSQL 16, Alembic	API HTTP + worker d'extraction + base de données
racine + src/	Expo SDK 54, React Native 0.81, TypeScript	App mobile de capture / revue / consultation

2. Les concepts fondamentaux (à maîtriser avant le code)

2.1 Architecture hexagonale (ports & adapters)

Chaque contexte du backend est découpé en trois couches, avec une **loi de dépendance à sens unique** :

adapters → application → domain
(SQL, HTTP, SDK, I/O) (cas d'usage, ports/Protocol) (règles métier pures, zéro import externe)

- Le **domaine** ne connaît ni SQLAlchemy, ni FastAPI, ni Pydantic, ni httpx. Uniquement stdlib + dataclasses. Résultat : testable en microsecondes, sans DB, sans réseau.
- L'**application** définit les *ports* (interfaces typing.Protocol) et orchestre les cas d'usage. Elle ne sait pas si le stockage est un fichier ou S3.
- Les **adapters** implémentent les ports : dépôt SQL, client Google Vision, routeur HTTP...

Pourquoi ? Le jour où tu remplaces Google Vision par un autre OCR, tu écris un adapter et une ligne de wiring — le domaine, les cas d’usage et les tests métier ne bougent pas. La règle est *appliquée mécaniquement* par `import-linter` (fichier `.importlinter`) : un import interdit fait échouer la CI.

2.2 Bounded contexts (contextes délimités)

Le backend est un **monolithe modulaire** : un seul déploiement, mais six contextes étanches sous `server/src/labelscan/contexts/` :

- ingestion — réception photo, OCR, LLM, gate, overrides humains. Le plus gros.
- traceability — registre produits / fournisseurs / lots.
- haccp — plans de contrôle, alertes, températures.
- identity — utilisateurs, login, scopes.
- audit, compliance — squelettes vides (structure prête, code à venir).

Interdiction d’import croisé entre contextes : ils communiquent uniquement par **événements** (via l’outbox) et par **identifiants**. `traceability` ne fait jamais `from labelscan.contexts.ingestion import ...` ; il consomme l’événement `extraction.completed` et relit la DB.

2.3 Transactional Outbox — le pattern le plus important du projet

Problème classique : « j’écris en base **et** je publie un événement ». Si tu fais les deux séparément, un crash entre les deux perd l’événement (ou publie un événement pour une écriture annulée).

Solution : l’événement est **une ligne dans la table `platform.outbox`, insérée dans la même transaction** que l’écriture métier. Commit → les deux existent. Rollback → aucun des deux.

Producteur (ingestion)	Worker relais	Consommateurs
BEGIN	boucle:	
INSERT ingestion	SELECT ... FOR UPDATE	
INSERT raw_artifact	SKIP LOCKED (claim)	
INSERT outbox(event)	dispatch → handler(s)	
COMMIT	INSERT processed_event	
	UPDATE outbox published	
	COMMIT (tout ou rien)	

Garanties précises (retiens ces termes) :

- **At-least-once** : un événement peut être livré plusieurs fois (crash avant commit → retraitement), jamais zéro fois.
- **Idempotence consommateur** : la table `platform.processed_event` mémorise chaque couple (consumer, event_id) déjà traité — une relivraison est ignorée, donc pas de double effet.
- **DLQ + backoff** : un événement dont le handler lève une exception est retenté avec un délai exponentiel (+ jitter), puis parqué en `dead_letter` après 3 tentatives — un « poison pill » ne bloque plus jamais la file.

2.4 Append-only et immutabilité

Les tables historiques (`audit_log`, `raw_artifact`, `extraction_run`, `extracted_field`, `temperature_log`, `batch...`) sont **append-only** : un trigger PostgreSQL `platform.deny_mutation()` lève une exception sur tout UPDATE/DELETE/TRUNCATE, y compris pour le superuser.

Conséquence structurante : **corriger = ajouter**. Un humain qui corrige un champ ne modifie pas le run d’extraction ; le système écrit un **nouveau** `extraction_run` (copie du précédent avec le champ remplacé, `source='human'`). Le « courant » est une projection de lecture (`max(attempt_no)`), jamais une mutation.

2.5 Audit incontournable (unbypassable audit)

L'audit n'est pas écrit par le code applicatif (qui pourrait l'oublier) mais par un trigger AFTER INSERT en base. Le trigger lit 4 variables **locales à la transaction** :

```
labelscan.actor_id, labelscan.action, labelscan.correlation_id, labelscan.trace_id
```

Si l'une manque → le trigger RAISE → **toute la transaction échoue**. Il est donc physiquement impossible d'insérer une ligne métier sans audit, et l'audit committe atomiquement avec la ligne.

2.6 No-fabrication : la frontière de confiance

Le LLM est traité comme **non fiable par construction**. La règle, appliquée par le *gate* du domaine (evaluate()) :

Une valeur n'est stockée comme valide **que si chaque « evidence » citée par le modèle est une sous-chaîne exacte du texte OCR**. Sinon la valeur est forcée à null, flaggée EVIDENCE_NOT_IN_RAW_OCR, et le run part en revue humaine.

Trois niveaux de confiance dans les données :

1. **GS1 (code-barres)** — exactitude mathématique, confiance 1.0, gagne tout conflit.
2. **LLM validé par le gate** — confiance = min(confiance LLM, confiance OCR), bandé low/medium/high.
3. **Humain** — l'opérateur a validé contre la photo : confiance 1.0, source='human'.

2.7 CQRS léger

Les écritures passent par des cas d'usage + repositories (commande). Les lectures (read_router.py) sont des **query adapters** : SQL direct → DTO Pydantic, zéro logique métier, elles renvoient « exactement ce qui est stocké ». Pas de bus de commandes ni d'event sourcing complet — juste la séparation lecture/écriture là où elle paie.

2.8 Idempotence de bout en bout

Chaque niveau a sa clé :

Niveau	Mécanisme
Upload photo	dédup par hash de contenu sha256(principal:route:sha256(image)) — deux fois la même photo = même ingestion_id, zéro double
Événements Appels providers (OCR/LLM)	processed_event (consumer, event_id) l'existence du raw_artifact = « déjà appelé », on relit au lieu de rappeler
Écritures de revue (override/confirm)	header Idempotency-Key + table request_idempotency (migration 0013)
Mobile	outbox locale AsyncStorage avec clés stables générées une seule fois à l'enqueue

2.9 Correlation & trace

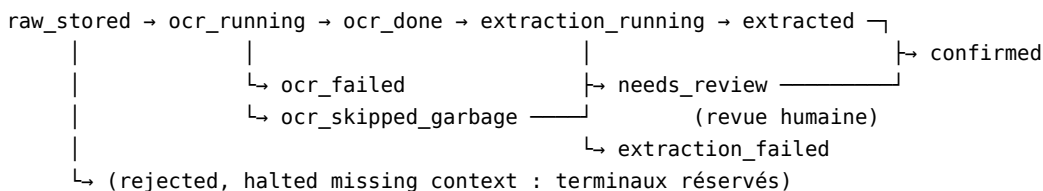
Chaque requête HTTP reçoit (ou génère) un X-Correlation-Id et un trace_id (W3C traceparent). Ils sont propagés **partout** : contexte d'audit, lignes outbox, logs JSON, réponses d'erreur. Tu peux suivre une photo de l'upload jusqu'à l'alerte HACCP avec un seul identifiant.

3. Le cycle de vie d'un scan, de bout en bout

Séquence complète (numéros = sections du cours qui les détaillent) :

1. **Mobile** : shutter → crop/resize → copie durable → op create_ingestion sur l'outbox locale → POST multipart /v1/ingestions (§19-21).
2. **API** : router → SubmitIngestion : hash SHA-256 → RawStore.put (fsync) → transaction unique (audit ctx + idempotency claim + ingestion + raw_artifact + outbox ingestion.raw_stored) → 202 (§10, §12).
3. **Worker** : claim l'événement → ExtractionConsumer : parse GS1 → OCR (dédup par artefact) → gate qualité OCR (skip LLM si illisible) → commit intermédiaire ocr_done + previews regex → LLM (dédup par artefact+modèle) → gate anti-fabrication → éventuelle escalade Opus → réconciliation GS1 → extraction_run + extracted_field + statut + événement extraction.completed (§12).
4. **Chaîne d'événements** : extraction.completed → RegistrationConsumer (traceability) → batch.registered/batch.flagged → AlertingConsumer (HACCP) → alertes (§13).
5. **Mobile** : long-poll GET /v1/ingestions/{id}?wait=25&last_status=... → affichage progressif → édition → save local + PATCH overrides + POST confirm + drain (§19-21).

4. La machine à 12 états d'une ingestion



- extracted : le gate a tout validé. needs_review : au moins un défaut (champ requis manquant, confiance basse, fabrication, incohérence).
- ocr_skipped_garbage : image illisible, le LLM n'a **pas** été appelé (économie de coût), les champs GS1 sont quand même stockés.
- confirmed : un revieweur a finalisé — état terminal de revue.

Le mobile classe ces mêmes états en trois familles (ingestionPolling.ts) : *processing* (continuer à poller), *review_ready* (extracted, needs_review, ocr_skipped_garbage, confirmed), *failed*.

5. Comment lire ce cours

- Chaque chapitre suit un fichier réel du dépôt, code à l'appui, expliqué bloc par bloc et ligne par ligne quand c'est instructif (les blocs répétitifs — styles, listes de paramètres — sont résumés).
- Les encadrés :

Concept — une notion générale (Python, TS, SQL, architecture) expliquée au moment où le code l'utilise.

Tip dev — une pratique à retenir et réutiliser dans tes propres projets.

Piège — une erreur que ce code évite explicitement (souvent apprise à la dure).

Partie II — Backend

6. Outillage et configuration du projet

6.1 server/pyproject.toml

```
[project]
name = "labelscan-server"
requires-python = ">=3.11"
dependencies = [
    "sqlalchemy>=2.0", "alembic>=1.13", "psycopg[binary]>=3.1",
    "fastapi>=0.115", "pydantic>=2.0", "python-multipart>=0.0.9",
    "uvicorn[standard]>=0.30", "httpx>=0.27", "anthropic>=0.40", "pyjwt>=2.8",
]
[project.optional-dependencies]
dev = ["pytest>=8.0", "import-linter>=2.0", "ruff>=0.6"]
```

Ligne par ligne :

- `sqlalchemy>=2.0` — utilisé en mode **Core** (requêtes `text()` paramétrées), pas en ORM. Choix délibéré : le SQL est explicite, les triggers et contraintes DB restent visibles.
- `psycopg[binary]` — driver PostgreSQL v3 (`postgresql+psycopg://`). `[binary]` embarque la libpq compilée : pas de compilation à l'install.
- `python-multipart` — requis par FastAPI pour parser les uploads multipart/form-data (la photo).
- `anthropic` — SDK Claude, importé **paresseusement** uniquement dans l'adapter (§11.4); le paquet fonctionne sans lui hors production.
- `pyjwt` — signature/vérification des JWT HS256.
- Le bloc `dev` est installé par `pip install -e ".[dev]"` : le `-e` (editable) fait pointer le site-packages vers `src/`, donc tes modifications sont prises en compte sans réinstaller.

```
[tool.setuptools]
package-dir = {"" = "src"}
```

Concept — src layout. Le paquet importable `labelscan` vit sous `src/`, pas à la racine. Avantage : impossible d'importer accidentellement le code du répertoire courant non installé — les tests importent le paquet *installé*, comme en prod.

```
[tool.ruff.lint]
select = ["E", "F", "I", "TID"]
[tool.ruff.lint.flake8-tidy-imports]
ban-relative-imports = "all"
```

- `E/F` : erreurs pycodestyle/pyflakes; `I` : tri des imports (remplace `isort`); `TID` : règles d'imports « tidy ».
- `ban-relative-imports = "all"` : tout import est absolu (`from labelscan.contexts...`), jamais `from ..domain import`. Les imports absolus rendent les contrats de frontière (§6.2) vérifiables mécaniquement.

6.2 .importlinter — l'architecture comme contrat exécutable

C'est le fichier le plus original du projet : l'architecture hexagonale n'y est pas une convention mais un **test qui échoue**. Quatre contrats :

```
[importlinter:contract:layering]
type = layers
layers =
    adapters
    application
    domain
containers =
    labelscan.contexts.ingestion
...
```

- type = layers : dans chaque conteneur (contexte), une couche ne peut importer que les couches **en-dessous** d'elle dans la liste. adapters peut importer application et domain; domain ne peut rien importer des deux autres.

```
[importlinter:contract:context-independence]
type = independence
modules = labelscan.contexts.ingestion, labelscan.contexts.traceability, ...
```

- type = independence : aucun des modules listés ne peut en importer un autre. C'est la règle « pas d'import croisé entre contextes ».

```
[importlinter:contract:domain-purity]
type = forbidden
source_modules = labelscan.contexts.*.domain
forbidden_modules =
    labelscan.platform, labelscan.app,
    sqlalchemy, alembic, psycpg, fastapi, httpx, pydantic
allow_indirect_imports = false
```

- type = forbidden : le domaine ne peut importer ni la plateforme, ni les frameworks. allow_indirect_imports = false : même transitif (via un module intermédiaire), c'est interdit.
- Notez include_external_packages = True en tête de fichier : sans lui, les contrats nommant des paquets externes (sqlalchemy...) seraient **silencieusement ignorés** — le commentaire du fichier le rappelle.

```
[importlinter:contract:ingestion-decoupled-from-worker]
source_modules = labelscan.contexts.ingestion
forbidden_modules = labelscan.platform.outbox
```

- Le producteur (ingestion) ne connaît pas le relais : il **écrit une ligne** dans la table outbox (du SQL), il n'importe jamais le worker. Ainsi un producteur ne peut structurellement pas appeler un consommateur.

Tip dev. Dans tout projet en couches, ajoute import-linter dès le premier jour. Une règle d'architecture non exécutée par la CI est une règle déjà morte.

Commande : lint-imports (échoue avec la liste des imports fautifs).

6.3 docker-compose.yml

```
services:
  db:      # postgres:16, healthcheck pg_isready
  server:  # API FastAPI, dépend de db healthy, entrypoint.sh (migrations + uvicorn)
  worker:
    deploy: { replicas: 2 }
    entrypoint: ["python", "-m", "labelscan.app.worker_runtime"]
    healthcheck:
      test: ["CMD", "python", "-m", "labelscan.app.worker_runtime", "--health"]
```

Points remarquables :

- **replicas: 2 pour le worker** — deux workers concurrents sont sûrs parce que le claim outbox utilise FOR UPDATE SKIP LOCKED (§8.3) : chaque ligne n'est prise que par un seul worker à la fois. Pas de container_name sur ce service : les répliques ont besoin de noms uniques.
- **Healthcheck sans HTTP** — le worker n'a pas de serveur web ; sa vivacité est un *fichier heartbeat* rafraîchi à chaque itération de boucle, vérifié par --health (§14.3). Un worker mort devient unhealthy dans docker ps au lieu de laisser silencieusement les photos coincées en raw_stored.
- Volume partagé labelscan-raw-store entre server et workers : le store d'objets filesystem est commun.

6.4 Alembic (migrations)

- alembic upgrade head — applique; alembic revision --autogenerate -m "nom" — génère; alembic downgrade -1 — annule la dernière.
- Ici les migrations sont majoritairement du **SQL brut via op.execute()** (triggers, partitions, grants) — l'autogénération ne sait pas produire ça. Chaque fichier commence par une docstring qui explique le *pourquoi* : lisez-les comme de la documentation d'architecture (§15).

6.5 Commandes du quotidien

```
cd server
pip install -e ".[dev]"          # installer
pytest -v                       # tests (exige un PostgreSQL frais – docker compose up db)
ruff check server/ && ruff format server/
lint-imports                    # vérifier l'architecture
uvicorn labelscan.app.http_app:create_app --factory # lancer l'API en dev
python -m labelscan.app.worker_runtime              # lancer le worker
python -m labelscan.contexts.identity.adapters.cli  # provisionner l'admin
```

Variables d'environnement essentielles :

Variable	Rôle
DATABASE_URL	postgresql+psycpg://... — obligatoire , aucun défaut
LABELSCAN_RAW_STORE_DIR	répertoire du store d'objets — obligatoire
LABELSCAN_JWT_SECRET	secret HS256, ≥ 32 octets — obligatoire pour l'auth
LABELSCAN_OCR_PROVIDER=google + LABELSCAN_GOOGLE_VISION_API_KEY	OCR (aucun défaut : non configuré = erreur claire)
LABELSCAN_LLM_MODEL	défaut claude-haiku-4-5
LABELSCAN_LLM_ESCALATION_ENABLED	escalade Opus, OFF par défaut
LABELSCAN_REVIEW_BELOW, LABELSCAN_BAND_*	seuils de confiance (calibrés, jamais devinés)
LABELSCAN_OCR_QUALITY_GATE_ENABLED, LABELSCAN_OCR_MIN_*	gate qualité OCR
LABELSCAN_ALLOW_HEADER_AUTH	seam d'auth par headers, OFF par défaut

Tip dev — fail loudly. Remarque le motif répété dans tout le projet : **aucune valeur sensible n'a de défaut silencieux**. URL de DB absente → RuntimeError explicite au démarrage, pas un fallback vers localhost. Un process mal configuré doit refuser de démarrer, pas fonctionner « à peu près ».

7. La couche platform/ — l'infrastructure partagée

platform/ contient l'infrastructure transverse : DB, audit, logs, HTTP, outbox. Aucun contexte métier n'y vit; tous s'appuient dessus (sauf le domaine, qui n'a le droit de rien importer).

7.1 platform/db/engine.py — la connexion DB

```
def database_url() -> str:
    url = os.environ.get("DATABASE_URL")
    if not url:
        raise RuntimeError(
            "DATABASE_URL is not set. Expected a postgresql+psycopg://... URL."
        )
    return url

def make_engine(url: str | None = None) -> Engine:
    return create_engine(url or database_url(), pool_pre_ping=True)
```

- `database_url()` : **seul endroit** du dépôt où la chaîne de connexion est résolue (le docstring l'annonce, import-linter le garantit). URL absente → erreur dure.
- `make_engine(url=None)` : le paramètre optionnel permet aux tests d'injecter une URL sans toucher à l'environnement. `url` or `database_url()` — si `url` est `None` (ou vide), on résout l'env.
- `pool_pre_ping=True` : avant de réutiliser une connexion du pool, SQLAlchemy émet un `SELECT 1`; une connexion morte (DB redémarrée, firewall) est remplacée au lieu d'exploser à la première vraie requête.

Concept — SQLAlchemy 2.0 Core. `create_engine` retourne un `Engine` (pool + dialecte). On travaille ensuite avec `engine.begin()` (ouvre connexion + transaction, commit/rollback automatique en sortie de `with`) ou `engine.connect()` (lecture, sans transaction explicite). Les requêtes sont des `text("... :param")` exécutées avec un dict de paramètres — **jamais** de f-string dans du SQL (injection).

7.2 platform/db/audit_context.py — le contexte d'audit transactionnel

Rappel du mécanisme (§2.5) : le trigger DB `platform.audit_on_insert()` lit 4 réglages *transaction-local* et refuse l'insert s'ils manquent. Ce module est le côté Python qui les pose.

```
_SET_LOCAL = text(
    "SELECT "
    "set_config('labelscan.actor_id',      :actor_id,      true), "
    "set_config('labelscan.action',       :action,        true), "
    "set_config('labelscan.correlation_id', :correlation_id, true), "
    "set_config('labelscan.trace_id',     :trace_id,      true)"
)
```

- `set_config(name, value, is_local)` est une fonction PostgreSQL. Le troisième argument `true = local à la transaction` : la valeur disparaît automatiquement au COMMIT/ROLLBACK. Aucune fuite possible vers la transaction suivante d'une connexion poolée — c'est ce détail qui rend le mécanisme sûr avec un pool.
- Les quatre `set_config` dans un seul SELECT = un seul aller-retour réseau.

```
def set_audit_context(conn, *, actor_id, action, correlation_id, trace_id) -> None:
    conn.execute(_SET_LOCAL, {...})
```

- `conn`: Connection — doit être appelé **dans une transaction ouverte, avant** tout insert audité. Les paramètres sont keyword-only (*): impossible d'inverser `actor` et `action` par erreur de position.

@contextmanager

```
def audited_transaction(conn, *, actor_id, action, correlation_id, trace_id):
    with conn.begin():
        set_audit_context(conn, ...)
    yield conn
```

Concept — @contextmanager. Transforme un générateur en context manager : le code avant `yield` s'exécute à l'entrée du `with`, le code après (ici implicite : la sortie du `with conn.begin()`) à la sortie. `conn.begin()` committe si le bloc sort normalement, rollback si une exception traverse — et comme l'audit est écrit par le trigger *dans la même transaction*, un rollback efface ensemble la ligne métier **et** sa ligne d'audit. Cohérence garantie par construction.

7.3 platform/observability.py — logs JSON structurés

Objectif : une ligne JSON par événement sur stdout (style 12-factor), sans dépendance externe.

```
_EXTRA_FIELDS = (
    "correlation_id", "trace_id", "event_type", "consumer", "ingestion_id",
    "run_id", "outcome", "error_code", "status", "path", "error",
    "model", "reason", "cache_creation_input_tokens", "cache_read_input_tokens",
    "ocr_ms", "llm_ms", "attempts", "image_bytes", "interim_field_count",
)
```

- **Allow-list** des champs structurés acceptés via `logger.info("msg", extra={...})`. Tout champ non listé est ignoré : un extra maladroit **ne peut pas** faire fuiter une valeur inattendue (payload, secret) dans les logs. C'est une défense par défaut, pas une convention.

```
class JsonFormatter(logging.Formatter):
    def format(self, record):
        payload = {"ts": ..., "level": record.levelname,
                  "logger": record.name, "message": record.getMessage()}
        for key in _EXTRA_FIELDS:
            value = getattr(record, key, None)
            if value is not None:
                payload[key] = value
        if record.exc_info and record.exc_info[0] is not None:
            payload["exc_type"] = record.exc_info[0].__name__
            payload["exc"] = self.formatException(record.exc_info)
        return json.dumps(payload, ensure_ascii=False, default=str)
```

- logging attache les clés d'extra comme attributs du LogRecord — d'où le `getattr(record, key, None)`.
- `record.exc_info` est peuplé par `logger.exception(...)` : le type et la traceback partent dans le JSON (côté serveur uniquement — le client, lui, ne voit jamais la traceback, §7.5).
- `default=str` : tout objet non sérialisable (UUID, datetime) est converti en chaîne au lieu de faire planter le logging.

```
def configure_logging() -> None:
    global _CONFIGURED
    if _CONFIGURED: return
    handler = logging.StreamHandler(sys.stdout)
    handler.setFormatter(JsonFormatter())
    root = logging.getLogger("labelscan")
    root.handlers[:] = [handler]
    root.setLevel(os.environ.get("LABELSCAN_LOG_LEVEL", "INFO").upper())
    root.propagate = False
    _CONFIGURED = True
```

- **Idempotent** via le drapeau module-level `_CONFIGURED` : appellable depuis chaque entriypoint (API, worker) sans doubler les handlers.
- `root.handlers[:] = [handler]` — remplace la liste **en place** (slice assignment) plutôt que `root.handlers = [...]` : même effet ici, mais évite de casser d'éventuelles références à la liste.
- `propagate = False` : les logs `labelscan.*` ne remontent pas au root logger Python (sinon double affichage si l'hôte a son propre handler).
- `get_logger("ingestion.llm")` → logger `labelscan.ingestion.llm` : un espace de noms par module.

Tip dev. La discipline en trois lignes du docstring vaut pour tout projet : (1) ne jamais logger secrets/payloads, seulement des identifiants et des scalaires ; (2) la réponse d'erreur au client reste aseptisée, le log serveur porte le détail ; (3) allow-lister les champs structurés.

7.4 platform/http/middleware.py — corrélation

```
def _trace_id_from(traceparent: str | None) -> str:
    if traceparent:
        parts = traceparent.split("-")
        if len(parts) >= 2 and parts[1]:
            return parts[1]
    return uuid.uuid4().hex
```

- Le header W3C `traceparent` a la forme `version-traceid-spanid-flags` ; on extrait le 2^e segment (le trace-id global). Absent/malformé → on en génère un.

```
class CorrelationMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request, call_next):
        correlation_id = request.headers.get("X-Correlation-Id") or f"corr_{uuid.uuid4().hex}"
        trace_id = _trace_id_from(request.headers.get("traceparent"))
        request.state.correlation_id = correlation_id
        request.state.trace_id = trace_id
        response = await call_next(request)
        response.headers.setdefault("X-Correlation-Id", correlation_id)
        return response
```

- `request.state` est un espace libre par-requête de Starlette : les routeurs y lisent `correlation_id/trace_id` pour les mettre dans le contexte d'audit.
- `call_next(request)` exécute le reste de la pile (routeur inclus) et rend la réponse ; le middleware écrit ensuite le header de corrélation en écho (`setdefault` : ne l'écrase pas si déjà posé, p.ex. par une réponse d'erreur).

7.5 platform/http/errors.py — erreurs RFC 9457 (problem+json)

Le contrat d'erreur de toute l'API. Trois pièces :

1. **Le catalogue** — `error_code` → (statut HTTP, titre stable, retrieable) :

```

ERROR_CATALOG = {
    "VALIDATION_ERROR": (400, "Malformed request", False),
    "UNAUTHENTICATED": (401, ..., False),
    "FORBIDDEN": (403, ..., False),
    "NOT_FOUND": (404, ..., False),
    "IDEMPOTENCY_KEY_CONFLICT": (409, ..., False),
    "INGESTION_NOT_CONFIRMABLE": (409, ..., False),
    "ALERT_INVALID_TRANSITION": (409, ..., False),
    "FIELD_NOT_EDITABLE": (409, ..., False),
    "PAYLOAD_TOO_LARGE": (413, ..., False),
    "UNSUPPORTED_MEDIA_TYPE": (415, ..., False),
    "RATE_LIMITED": (429, ..., True),
    "INTERNAL_ERROR": (500, ..., True),
    "DEPENDENCY_UNAVAILABLE": (503, ..., True),
}

```

Concept — contrat d'erreur stable. Les clients branchent sur `error_code`, **jamais** sur le statut HTTP ni le texte. Le catalogue est *append-only* : on ajoute des codes, on n'en réutilise jamais un pour un autre sens (même discipline que l'audit). Le booléen `retriable` dit au client si réessayer a un sens — le mobile s'en sert directement (§19.1).

2. `ApiError` — l'exception à lever depuis les adaptors :

```

class ApiError(Exception):
    def __init__(self, error_code, detail=None, errors=None):
        self.error_code = error_code; self.detail = detail; self.errors = errors
        super().__init__(error_code)

```

3. `problem_response` — fabrique la réponse :

```

body = {
    "type": f"https://errors.labelscan/{error_code}",
    "title": title, "status": status, "detail": detail or title,
    "error_code": error_code,
    "correlation_id": correlation_id, "trace_id": trace_id,
    "timestamp": datetime.now(timezone.utc).isoformat(),
    "retriable": retriable,
}
return JSONResponse(..., media_type="application/problem+json", headers=headers)

```

- `type/title/status/detail` = les champs standard RFC 9457; `error_code/correlation_id/trace_id/retriable` = extensions maison. Le `media_type` officiel `application/problem+json` permet aux intermédiaires de reconnaître une erreur structurée.

4. `install_error_handlers(app)` installe trois handlers FastAPI :

```

@app.exception_handler(ApiError)
async def _api_error(request, exc):
    status = ERROR_CATALOG.get(exc.error_code, (500, "", False))[0]
    if status >= 500:
        _log.error("api_error", extra={...})
    return problem_response(request, exc.error_code, ...)

```

- Les 4xx sont des erreurs client *attendues* → silence dans les logs; seuls les 5xx sont loggués (signal/bruit).

```

@app.exception_handler(RequestValidationError)
async def _validation(request, exc):
    errors = [{"field": ".".join(str(p) for p in e.get("loc", [])),
               "code": e.get("type", ""), "message": e.get("msg", "")}
              for e in exc.errors()]
    return problem_response(request, "VALIDATION_ERROR", ...)

```

- Convertit les erreurs Pydantic (loc est un tuple de segments, p.ex. ("body", "value")) en liste plate field/code/message — le format d'erreur de FastAPI ne fuit pas dans le contrat public.

```
@app.exception_handler(Exception)
async def _unhandled(request, exc):
    _log.exception("unhandled_exception", extra={...})
    return problem_response(request, "INTERNAL_ERROR")
```

- Le filet final : la traceback part **dans les logs**, le client reçoit un INTERNAL_ERROR générique. Jamais de fuite d'implémentation (exception SQL, chemin de fichier...) vers l'extérieur — exigence du CLAUDE.md du projet.

7.6 platform/http/jwt.py — le codec JWT

Séparation stricte : ce module **signe et vérifie l'enveloppe**, il ne décide de rien (qui peut se connecter, quels scopes = contexte identity).

```
_ALGORITHM = "HS256"
_MIN_SECRET_BYTES = 32 # plancher RFC 7518 §3.2 pour HS256

def _secret() -> str:
    secret = os.environ.get("LABELSCAN_JWT_SECRET")
    if not secret: raise RuntimeError(...)
    if len(secret.encode("utf-8")) < _MIN_SECRET_BYTES: raise RuntimeError(...)
    return secret
```

- HS256 = HMAC symétrique : le même secret signe et vérifie — suffisant ici car un seul service fait les deux. Le plancher de 32 octets est celui de la RFC (en dessous, un brute-force du secret devient réaliste).

```
def encode(claims, *, ttl_seconds=None) -> str:
    now = datetime.now(timezone.utc)
    ttl = ttl_seconds if ttl_seconds is not None else default_ttl_seconds()
    payload = {**claims,
               "iat": int(now.timestamp()),
               "exp": int((now + timedelta(seconds=ttl)).timestamp())}
    return jwt.encode(payload, _secret(), algorithm=_ALGORITHM)

def decode(token) -> dict:
    try:
        return jwt.decode(token, _secret(), algorithms=[_ALGORITHM])
    except jwt.PyJWTError as exc:
        raise TokenError(str(exc)) from exc
```

- iat/exp (issued-at / expiry) sont posés ici — l'appelant fournit uniquement les claims d'identité. TTL par défaut 12 h, configurable par LABELSCAN_JWT_TTL_SECONDS (validé : entier positif).
- decode passe algorithms=[_ALGORITHM] **explicitement** : c'est la parade à l'attaque classique « alg=None » / confusion d'algorithme. Toute erreur PyJWT (signature, expiration, malformé) est repliée en une seule TokenError maison — les appelants n'ont pas à connaître PyJWT.

Concept — chaînage d'exceptions. raise TokenError(...) from exc conserve la cause dans __cause__ (visible dans la traceback serveur) tout en présentant un type propre à l'appelant. À l'inverse, raise ... from None (utilisé dans l'adapter Google Vision, §11.3) **supprime** la cause — utile quand la cause pourrait contenir un secret.

7.7 platform/http/security.py — la porte d'authentification

```
@dataclass(frozen=True)
class Principal:
    actor_id: str          # uuid — enregistré dans l'audit
    principal: str         # scope d'idempotence (ex. device id)
    scopes: frozenset[str]
```

Concept — @dataclass(frozen=True). Génère `__init__`, `__repr__`, `__eq__` et rend l'instance immuable (assigner un attribut lève `FrozenInstanceError`). Tout le projet l'utilise pour ses value objects : une donnée qui ne peut pas changer est une donnée qu'on peut passer partout sans crainte. `frozenset` de même pour les collections.

Deux chemins d'authentification :

```
def resolve_principal(request: Request) -> Principal:
    authorization = request.headers.get("Authorization")
    if authorization:
        scheme, _, token = authorization.partition(" ")
        if scheme.lower() == "bearer" and token.strip():
            return _principal_from_bearer(token.strip())
        raise ApiError("UNAUTHENTICATED", "unsupported Authorization scheme")
    if _header_auth_enabled():
        return _principal_from_headers(request)
    raise ApiError("UNAUTHENTICATED", "No authenticated principal")
```

- `str.partition(" ")` coupe en (avant, sép, après) — plus sûr que `split` quand le token pourrait contenir des espaces.
- **Chemin 1 (principal)** : `Bearer <JWT> → jwt.decode → claims actor_id, principal, scopes` (liste ou chaîne séparée par espaces — les deux formes sont acceptées).
- **Chemin 2 (legacy)** : headers `X-Actor-Id/X-Principal/X-Scopes` posés par une gateway de confiance. **Désactivé par défaut** (`LABELSCAN_ALLOW_HEADER_AUTH`) : sinon n'importe qui pourrait forger une identité en ajoutant des headers. On ne l'active que derrière une gateway qui les écrase.

```
def require_scope(scope: str):
    def _dep(request: Request) -> Principal:
        principal = resolve_principal(request)
        if scope not in principal.scopes:
            raise ApiError("FORBIDDEN", f"requires scope '{scope}'")
        return principal
    return _dep
```

Concept — dépendance FastAPI paramétrée (closure). `require_scope("ingestion:write")` retourne une fonction `_dep` qui capture `scope`. Dans un routeur : `principal: Principal = Depends(require_scope("ingestion:write"))` — FastAPI appelle `_dep(request)` à chaque requête. 401 = pas authentifié; 403 = authentifié mais `scope` manquant. C'est le motif standard pour des gardes paramétrées.

7.8 platform/http/deps.py et read_models.py

```
_ENGINE: Engine | None = None
def get_engine() -> Engine:
    global _ENGINE
    if _ENGINE is None: _ENGINE = make_engine()
    return _ENGINE
```

- Le provider d'engine des routeurs de lecture, **caché module-level** (un seul pool par process). Les tests le surchargent via `app.dependency_overrides[get_engine] = lambda:`

engine_de_test.

```
class AuditEntry(BaseModel):
    actor_id: str; action: str; occurred_at: str

def audit_entries(conn, subject_id) -> list[AuditEntry]:
    rows = conn.execute(text(
        "SELECT actor_id::text AS actor_id, action, occurred_at::text AS occurred_at "
        "FROM audit.audit_log WHERE subject_id = :s ORDER BY occurred_at"), {"s": subject_id}
    ).mappings().all()
    return [AuditEntry(**r) for r in rows]
```

- Le fragment de lecture partagé par tous les read-routers : l'historique d'audit d'un sujet. Il vit dans `platform` pour que chaque contexte puisse embarquer l'audit **sans importer un autre contexte**.
- `::text` : on caste uuid/timestampz en texte côté SQL — les DTO restent en `str`, pas de sérialisation surprise.
- `.mappings()` : lignes en dict-like (accès par nom) au lieu de tuples; `AuditEntry(**r)` déballe directement.

8. L'outbox : platform/outbox/

Trois fichiers : le message, le calcul de backoff (pur), le worker relais.

8.1 message.py

```
@dataclass(frozen=True)
class OutboxMessage:
    id: str  # id de la ligne outbox – sert aussi d'event_id pour la dédup
    event_type: str
    payload: dict[str, Any]
    correlation_id: str
    trace_id: str
```

- id joue double rôle : clé primaire de la ligne **et** identifiant d'événement pour processed_event. Un seul identifiant = pas de désynchronisation possible.

8.2 backoff.py — les maths de retry, pures

```
DEFAULT_BASE_SECONDS: float = 2.0
DEFAULT_CAP_SECONDS: float = 3600.0

def backoff_delay_seconds(attempt, *, base_seconds=2.0, cap_seconds=3600.0,
                          rng=random.random) -> float:
    if attempt < 1:
        raise ValueError(f"attempt must be >= 1, got {attempt}")
    exponential = base_seconds * (2 ** (attempt - 1))  # 2s, 4s, 8s, ...
    capped = min(exponential, cap_seconds)             # plafonné à 1h
    half = capped / 2.0
    return half + rng() * half                        # "equal jitter"
```

Ligne par ligne :

- attempt est 1-based : tentative 1 → 2 s de base, tentative 2 → 4 s, etc.
- min(..., cap_seconds) : sans plafond, l'exponentielle atteint des jours en quelques tentatives.
- **Equal jitter** : capped/2 + aléa*capped/2. Deux propriétés voulues :
 1. le **jitter** disperse les retries de N workers/événements (anti «thundering herd» : tous ne réessaient pas à la même seconde);
 2. le **plancher garanti** (capped/2) fait que delay(n+1) > delay(n) est toujours vrai — contrairement au «full jitter» (aléa*capped) où le délai peut *rétrécir* entre deux tentatives, ce qui rendrait flaky tout test affirmant «le backoff croît».
- rng: Callable[[], float] = random.random — **l'aléa est injectable**. Un test passe rng=lambda: 0.5 et obtient des valeurs déterministes.

```
def calculate_backoff(attempt, *, now=None, ...) -> datetime:
    reference = now if now is not None else datetime.now(UTC)
```

```
delay = backoff_delay_seconds(attempt, ...)
return reference + timedelta(seconds=delay)
```

- Même principe : `now` injectable pour les tests. Le résultat (un instant UTC absolu) atterrit tel quel dans la colonne `next_retry_at`.

Tip dev — injecter le temps et l'aléa. Chaque fois qu'une fonction dépend de `now()` ou de `random()`, prends-les en paramètre avec un défaut. Coût : zéro. Gain : des tests déterministes sans monkeypatching. Ce projet le fait partout (`today=date.today` dans le consumer HACCP, `now` dans l'outbox mobile...).

Concept — pourquoi ce module est « pur ». Il n'importe que la `stdlib` (docstring : il obéit à la règle de pureté du `CLAUDE.md` même s'il vit dans `platform`). Les maths de scheduling sont ainsi testables sans DB, et le worker (impur) n'est que de la plomberie autour.

8.3 worker.py — le relais transactionnel, ligne par ligne

Les requêtes préparées

```
_CLAIM = text(
    "SELECT id, event_type, payload, correlation_id, trace_id "
    "FROM platform.outbox "
    "WHERE published_at IS NULL "
    "  AND status <> 'dead_letter' "
    "  AND (next_retry_at IS NULL OR next_retry_at <= now()) "
    "  AND event_type = ANY(CAST(:types AS text[])) "
    "ORDER BY created_at "
    "FOR UPDATE SKIP LOCKED "
    "LIMIT 1"
)
```

Décortiquage de la clause la plus importante du backend :

- `published_at IS NULL` — jamais publié (le sentinel historique de « à traiter »).
- `status <> 'dead_letter'` — pas parqué en DLQ.
- `next_retry_at IS NULL OR next_retry_at <= now()` — soit jamais échoué, soit sa fenêtre de backoff est écoulée. C'est **cette** condition qui empêche un événement en échec d'être re-réclamé en boucle chaude.
- `event_type = ANY(CAST(:types AS text[]))` — le worker ne réclame **que** les types pour lesquels il a un handler enregistré ; on passe une liste Python, castée en tableau SQL.
- `ORDER BY created_at` — FIFO.
- `FOR UPDATE SKIP LOCKED` — verrouille la ligne pour cette transaction ; les autres workers **sautent** les lignes verrouillées au lieu d'attendre. C'est ce qui rend replicas: 2 sûr : deux workers ne prendront jamais la même ligne, et aucun n'attend l'autre.
- `LIMIT 1` — une ligne par transaction : l'unité d'atomicité est l'événement.

```
_SEEN = text("SELECT 1 FROM platform.processed_event WHERE consumer = :c AND event_id = :e")
_MARK_PROCESSED = text("INSERT INTO platform.processed_event (consumer, event_id) VALUES (:c, :e)")
_MARK_PUBLISHED = text("UPDATE platform.outbox SET status='published', published_at=now() WHERE
↪ id=:id")
_LOCK_FOR_FAILURE = text(
    "SELECT attempts FROM platform.outbox "
    "WHERE id = :id AND status = 'pending' AND published_at IS NULL FOR UPDATE")
_SCHEDULE_RETRY = text("UPDATE platform.outbox SET attempts=:a, last_error=:e, next_retry_at=:nr
↪ WHERE id=:id")
_DEAD_LETTER = text(
    "UPDATE platform.outbox SET attempts=:a, last_error=:e, status='dead_letter', next_retry_at=NULL
↪ WHERE id=:id")
```

La classe

```
OutboxHandler = Callable[[OutboxMessage, Connection], None]
```

- Signature de tout consommateur : il reçoit le message **et la connexion de la transaction du worker**. Tout effet DB qu'il produit committe donc atomiquement avec la marque `processed_event` et le `published`. Un handler qui écrirait sur sa propre connexion perdrait cette atomicité — d'où la signature imposée.

```
class OutboxWorker:
    def __init__(self, engine, *, max_retries=3, backoff_base_seconds=2.0,
        ↪ backoff_cap_seconds=3600.0):
        self._engine = engine
        self._handlers: dict[str, list[tuple[str, OutboxHandler]]] = {}
        ...
    def register(self, event_type, consumer, handler):
        self._handlers.setdefault(event_type, []).append((consumer, handler))
```

- `_handlers` : `event_type` → [(nom_consommateur, handler), ...]. Plusieurs consommateurs peuvent écouter le même type (chacun avec sa propre dédup). `setdefault` crée la liste au premier enregistrement.
- `max_retries` par défaut lu de `LABELSCAN_OUTBOX_MAX_RETRIES` (3) — réglable en ops sans redéploiement.

```
def run_once(self, max_messages: int = 100) -> int:
    handled = 0
    while handled < max_messages:
        if not self._process_one(): break
        handled += 1
    return handled
```

- Draine jusqu'à `max_messages` lignes « réclamables ». « Handled » = résolue dans cette passe : publiée, **ou** (si le handler a levé) re-planifiée/parquée. Un poison pill compte donc une fois puis est sauté (son `next_retry_at` est dans le futur) — il ne peut ni tourner en boucle chaude ni bloquer les autres.

`_process_one` — le cœur

```
def _process_one(self) -> bool:
    types = list(self._handlers.keys())
    if not types: return False
    claimed_id = None; ctx = {}
    try:
        with self._engine.begin() as conn:                                # (1) transaction atomique
            row = conn.execute(_CLAIM, {"types": types}).mappings().first()
            if row is None: return False                                  # (2) rien à faire
            claimed_id = str(row["id"])
            msg = OutboxMessage(id=claimed_id, ...)
            for consumer, handler in self._handlers.get(msg.event_type, []):
                if conn.execute(_SEEN, {"c": consumer, "e": msg.id}).first():
                    continue                                             # (3) déjà traité → skip
                handler(msg, conn)                                       # (4) effet métier
                conn.execute(_MARK_PROCESSED, {"c": consumer, "e": msg.id}) # (5)
                _log.info("event_handled", extra={"consumer": consumer, **ctx})
                conn.execute(_MARK_PUBLISHED, {"id": msg.id})           # (6)
            return True                                                  # (7) commit ici
    except Exception as exc:
        if claimed_id is None:
            raise                                                         # (8) échec au claim → remonter
        self._record_failure(claimed_id, exc, ctx)                       # (9) poison pill
```

```
return True
```

- (1) `engine.begin()` : claim + handlers + marques + publication dans **une** transaction. Crash n'importe où avant (7) → rollback complet → la ligne redevient pending, rien n'est perdu, rien n'est à moitié fait.
- (3) **Idempotence** : si ce consommateur a déjà traité cet event (livraison précédente committée), on saute — c'est ce qui transforme l'at-least-once en «exactement un effet».
- (8) Distinction fine : si l'exception survient **avant** d'avoir réclamé une ligne, c'est la DB qui va mal — on remonte l'erreur (la boucle de poll dormira), on ne la masque pas.
- (9) Si un **handler** a levé : la transaction (1) est déjà rollback (aucun effet partiel, pas de marque processed). On enregistre l'échec *hors bande* puis on **continue le batch** — le correctif R-B01 : avant, un poison pill re-levait et bloquait toutes les photos derrière lui.

`_record_failure` — la mémoire du poison pill

```
def _record_failure(self, event_id, exc, ctx) -> None:
    trace = "".join(traceback.format_exception(type(exc), exc, exc.__traceback__))
    with self._engine.begin() as conn:                                     # transaction SÉPARÉE
        locked = conn.execute(_LOCK_FOR_FAILURE, {"id": event_id}).first()
        if locked is None:
            return                                                         # course bénigne → rien
        attempts = int(locked[0]) + 1
        if attempts >= self._max_retries:
            conn.execute(_DEAD_LETTER, {...})
            _log.error("event_dead_lettered", extra={...})
            return
        next_retry = calculate_backoff(attempts, now=datetime.now(UTC), ...)
        conn.execute(_SCHEDULE_RETRY, {...})
        _log.warning("event_retry_scheduled", extra={...})
```

- Pourquoi une transaction **séparée** ? La transaction principale a été rollbackée — il faut bien committer quelque chose pour que l'échec laisse une trace. Sans ce commit, la ligne paraîtrait neuve et serait re-réclamée immédiatement, pour toujours (le docstring : «committing the failure is what gives the poison pill a memory»).
- Le garde `_LOCK_FOR_FAILURE` re-vérifie `status='pending' AND published_at IS NULL` sous verrou : si un autre worker a publié la ligne entre-temps, on ne touche à rien (course bénigne, pas de bump erroné d'une ligne saine).
- La traceback complète part dans `last_error` : c'est l'outil de triage de la DLQ pour l'ops.

Piège évité — le poison pill. Sans DLQ ni backoff, l'état « à traiter » est un booléen implicite (`published_at IS NULL`), et un événement qui ne peut jamais réussir est indiscernable d'un neuf : le worker le re-réclame en boucle, en tête de FIFO, et **toutes** les photos derrière sont bloquées. La migration 0009 ajoute `attempts/next_retry_at/last_error/status` précisément pour donner à l'échec une histoire et un état terminal.

Distinction à retenir. Le worker ne dead-letter que les **exceptions** (bugs, payload malformé). Un échec *métier attendu* (p.ex. «providers épuisés») n'est pas une exception qui traverse : le consommateur l'enregistre proprement (`run_extraction_failed`) et **retourne normalement** — l'événement est publié, pas retenté.

9. Contexte ingestion — le domaine (pur)

Cinq modules, zéro dépendance externe. C'est ici que vivent toutes les décisions.

9.1 domain/status.py

```
class IngestionStatus(StrEnum):
    RAW_STORED = "raw_stored"
    OCR_RUNNING = "ocr_running"
    OCR_DONE = "ocr_done"
    OCR_FAILED = "ocr_failed"
    OCR_SKIPPED_GARBAGE = "ocr_skipped_garbage"
    EXTRACTION_RUNNING = "extraction_running"
    EXTRACTED = "extracted"
    EXTRACTION_FAILED = "extraction_failed"
    NEEDS_REVIEW = "needs_review"
    CONFIRMED = "confirmed"
    REJECTED = "rejected"
    HALTED_MISSING_CONTEXT = "halted_missing_context"
```

Concept — StrEnum (Python 3.11+). Chaque membre EST une str (`IngestionStatus.RAW_STORED == "raw_stored"` est vrai, et il se sérialise en JSON sans effort). On obtient le typage fort d'une enum + la compatibilité chaîne du SQL/JSON. Le miroir exact existe côté mobile (`types/api.ts`) et côté DB (contrainte CHECK) : trois copies du même contrat, chacune vérifiée dans son monde.

9.2 domain/gs1.py — le parseur de codes-barres GS1

Un code GS1-128/DataMatrix encode des **Application Identifiers (AI)** : (01) GTIN, (10) lot, (17) DLC, (310x) poids net... Deux formes d'entrée :

- lisible : "(01)03700161210047(17)251231(10)LOT123";
- brute scanner : les AI à longueur **fixe** s'enchaînent sans séparateur, les AI à longueur **variable** se terminent par le caractère FNC1 (`\x1d`, le Group Separator ASCII).

```
FNC1 = "\x1d"
_FIXED_LEN = {"00": 18, "01": 14, "02": 14, "11": 6, "12": 6,
              "13": 6, "15": 6, "16": 6, "17": 6}
_MEASURE_PREFIXES = ("310", "311", "312", "313", "315", "316", "320", "321")
```

- Sous-ensemble HACCP : SSCC, GTIN, dates (production/emballage/DDM/DLC), famille poids. La famille « mesure » a des AI à 4 chiffres `NNNx` où `x` = la position de la décimale, payload toujours 6 chiffres. `_decode_measure` calcule `int(payload) / 10**decimals` : AI 3102 + payload 001500 → $1500/10^2 = 15$ kg; AI 3103 + 001500 → $1500/10^3 = 1,5$ kg. Décodage purement mécanique, zéro interprétation.

```

@dataclass(frozen=True)
class GslResult:
    elements: dict[str, str]          # tous les AI bruts parsés
    gtin: str | None = None
    lot: str | None = None
    expiry_date: str | None = None    # AI 17, ISO
    best_before: str | None = None    # AI 15, ISO
    production_date: str | None = None
    packaging_date: str | None = None
    net_weight_kg: float | None = None
    warnings: tuple[str, ...] = ()

    @property
    def is_empty(self) -> bool:
        return not self.elements

```

- `elements` garde **tout** ce qui a été lu (même les AI inconnus); les champs nommés sont les projections décodées utiles au métier. `warnings` accumule chaque anomalie **au lieu de lever** — contrat : `parse_gsl` ne plante jamais sur une entrée malformée.

Décodage des dates

```

def _decode_date(yymmdd: str) -> str | None:
    if len(yymmdd) != 6 or not yymmdd.isdigit(): return None
    yy, mm, dd = int(yymmdd[:2]), int(yymmdd[2:4]), int(yymmdd[4:6])
    if mm < 1 or mm > 12: return None
    year = 2000 + yy          # fenêtre 20xx simplifiée
    if dd == 0:
        dd = _last_day(year, mm) # spec GS1 : jour "00" = dernier jour du mois
    try:
        return date(year, mm, dd).isoformat()
    except ValueError:
        return None

```

- La spec GS1 autorise JJ=00 pour «fin du mois» — géré explicitement. La construction `date(...)` sert de validation finale (30 février → `ValueError` → `None`).

Lecture positionnelle

```

def _read_ai(s, i):
    if i+4 <= len(s) and s[i:i+3] in _MEASURE_PREFIXES and s[i:i+4].isdigit():
        return s[i:i+4], 4          # AI mesure : 4 chiffres
    two = s[i:i+2]
    if two.isdigit(): return two, 2 # AI standard : 2 chiffres
    return None, 0

```

```

def _parse_positional(s):
    elements, warnings = {}, []
    i, n = 0, len(s)
    while i < n:
        if s[i] == FNC1: i += 1; continue          # séparateur → avancer
        ai, alen = _read_ai(s, i)
        if ai is None:
            warnings.append(f"unrecognised AI at position {i}: ...")
            break                                  # stop – ne pas risquer une lecture désalignée
        i += alen
        if ai in _FIXED_LEN:
            elements[ai] = s[i:i+_FIXED_LEN[ai]]; i += _FIXED_LEN[ai]

```



```

elif len(ai) == 4 and ai[:3] in _MEASURE_PREFIXES:
    elements[ai] = s[i:i+6]; i += 6
else:
    j = s.find(FNC1, i)
    if j == -1: j = n
    elements[ai] = s[i:j]; i = j
return elements, warnings

```

- Le break sur AI inconnu est un choix de sûreté : continuer à deviner désalignerait toutes les lectures suivantes. Mieux vaut un warning honnête.

La forme parenthésée

```
parts = re.split(r"\((\d{2,4})\)", s)
```

Concept — re.split avec groupe capturant. Quand le motif contient un groupe (), les captures sont **insérées** dans la liste résultat : "(01)ABC(10)X" → ["", "01", "ABC", "10", "X"]. On itère alors par paires (AI, payload) avec `it = iter(parts[1:])` puis `next(it, "")` — une astuce idiomatique pour zipper une liste plate en couples.

parse_gsl — l'assemblage

```

def parse_gsl(raw: str | None) -> GslResult:
    if not raw or not raw.strip(): return GslResult(elements={})
    s = raw.strip()
    elements, warnings = _parse_parenthesised(s) if "(" in s else _parse_positional(s)
    gtin = elements.get("01") or None
    if gtin is not None and (len(gtin) != 14 or not gtin.isdigit()):
        warnings.append(f"GTIN (AI 01) is not 14 digits: {gtin!r}")
    ...
    return GslResult(elements=elements, gtin=gtin, lot=lot,
                      expiry_date=_date("17"), best_before=_date("15"),
                      production_date=_date("11"), packaging_date=_date("13"),
                      net_weight_kg=net_weight_kg, warnings=tuple(warnings))

```

- Un GTIN invalide est **conservé** avec warning (jamais corrigé, jamais jeté silencieusement).
- {gtin!r} dans la f-string : le !r applique repr() — les guillemets et caractères invisibles restent visibles dans le warning.

9.3 domain/extraction.py — le gate anti-fabrication

Le module le plus important du projet. D'abord les value objects :

```

class GateOutcome(StrEnum):
    EXTRACTED = "extracted"; NEEDS_REVIEW = "needs_review"

class ConfidenceBand(StrEnum):
    LOW = "low"; MEDIUM = "medium"; HIGH = "high"

_PRODUCTION_METHODS = frozenset({"wild_caught", "farmed"}) # vocabulaire contrôlé

@dataclass(frozen=True)
class LlmField:
    name: str
    value: str | None
    llm_confidence: float
    evidence: tuple[str, ...] # sous-chaînes exactes que le modèle DIT avoir lues

```

```
validation_status: str          # auto-évaluation du modèle (consultatif seulement)
warnings: tuple[str, ...] = ()
```

```
@dataclass(frozen=True)
```

```
class Span:
    page: int; offset_start: int; offset_end: int
```

```
@dataclass(frozen=True)
```

```
class EvaluatedField:
    name: str; value: str | None
    evidence: tuple[str, ...] | None
    spans: tuple[Span, ...]      # provenance : OÙ dans le texte OCR
    validation_status: str; warnings: tuple[str, ...]
    llm_confidence: float; ocr_confidence: float
    combined_confidence: float; confidence_band: ConfidenceBand
```

- LlmField = ce que le modèle **propose** (avant validation); EvaluatedField = ce que le gate **a jugé**, enrichi de la provenance (spans) et de la confiance combinée. Deux types distincts = impossible de confondre du non-validé et du validé dans une signature.

```
@dataclass(frozen=True)
```

```
class RuleSet:
    version: str
    required_fields: frozenset[str]
```

```
@dataclass(frozen=True)
```

```
class Thresholds:
    review_below: float = 0.70
    high_at: float = 0.90
    medium_at: float = 0.70
```

- Les champs obligatoires sont de la **donnée versionnée** (fournie par Compliance), jamais du code : chaque extraction_run enregistre rule_set_version, donc on sait *sous quelles règles* chaque extraction a été jugée. Idem pour les seuils : ils doivent être **calibrés** sur données réelles (scripts/calibrate_confidence.py), et chaque changement bump la version.

Le gate qualité OCR (économiseur de coût)

```
@dataclass(frozen=True)
```

```
class OcrQualityPolicy:
    min_chars: int = 6
    min_alnum_chars: int = 6
    min_mean_confidence: float = 0.15
```

```
def is_ocr_garbage(full_text, mean_confidence, policy=OcrQualityPolicy()) -> bool:
    text = (full_text or "").strip()
    if len(text) < policy.min_chars: return True
    alnum = sum(1 for ch in text if ch.isalnum())
    if alnum < policy.min_alnum_chars: return True
    if 0.0 < mean_confidence < policy.min_mean_confidence: return True
    return False
```

- But : ne pas payer un appel LLM pour une image manifestement illisible. **Biais volontaire vers False** (continuer) : un faux « skip » ne fait que router vers revue humaine, jamais perdre de données.
- Subtilité de la 3^e condition : $0.0 < \text{mean_confidence}$ — Google Vision renvoie 0.0 quand il **ne fournit pas** de confiance du tout (§11.3). Une lecture lisible mais non scorée ne doit pas être jetée sur un 0.0 : on ne fait confiance au score que s'il existe (> 0).

La vérification de provenance — le cœur du no-fabrication

```
def _spans_for(evidence, ocr_text, page) -> tuple[Span, ...] | None:
    spans = []
    for ev in evidence:
        idx = ocr_text.find(ev)
        if idx < 0:
            return None # UNE evidence introuvable = fabrication
        spans.append(Span(page=page, offset_start=idx, offset_end=idx+len(ev)))
    return tuple(spans)
```

- `str.find` retourne l'index de la première occurrence ou -1. **Chaque** evidence doit être une sous-chaîne *verbatim* du texte OCR; une seule manquante → `None` → la valeur entière est traitée comme inventée. Le span (page, début, fin) est stocké : n'importe quel auditeur peut re-pointer la zone du texte source.

evaluate() — le juge

```
def evaluate(fields, *, ocr_text, ocr_confidence, rule_set,
              thresholds=Thresholds(), page=1) -> GateVerdict:
```

Phase 1 — par champ :

```
for f in fields:
    value, evidence, spans, vstatus = f.value, f.evidence or None, (), f.validation_status
    if value is not None:
        grounded = _spans_for(f.evidence, ocr_text, page) if f.evidence else None
        if grounded is None:
            # le modèle a inventé → null + flag + quarantaine
            value = None; evidence = None; vstatus = "invalid"
            unverifiable.append(f.name)
            security_flags.append("EVIDENCE_NOT_IN_RAW_OCR")
        else:
            spans = grounded
            if f.name == "production_method" and value not in _PRODUCTION_METHODS:
                inconsistent.append(f.name); vstatus = "invalid"
            if vstatus in ("invalid", "unnormalizable") and f.name not in inconsistent:
                inconsistent.append(f.name)
    if value is None:
        evidence = None; spans = () # invariant : evidence ⇔ value
    combined = round(min(f.llm_confidence, ocr_confidence) if value is not None else 0.0, 3)
    band = _band(combined, thresholds)
    evaluated.append(EvaluatedField(...))
```

Points clés :

- Une valeur sans evidence prouvée est **coercée à null** — pas rejetée avec erreur, pas corrigée : null + flag sécurité. Le run entier partira en revue.
- **Invariant de stockage** `ck_evidence_iff_value` (contrainte CHECK en DB) : evidence existe *si et seulement si* value existe. Le gate l'applique déjà en amont — une valeur nulle (ambiguë, manquante, ou fabrication coercée) ne garde jamais d'evidence orpheline ; les candidats en conflit restent dans warnings.
- **Confiance combinée** = `min(llm, ocr)` : le maillon faible domine. Un LLM sûr de lui sur un OCR pourri reste peu fiable. `round(..., 3)` : stabilité d'affichage et d'égalité.
- `production_method` est validé contre le **vocabulaire contrôlé** : toute autre valeur que `wild_caught/farmed` est incohérente (champ à enjeu réglementaire).

Phase 2 — inter-champs et règles :

```
by_name = {e.name: e for e in evaluated}
exp, pkg = by_name.get("expiry_date"), by_name.get("packaging_date")
```

```
if exp and pkg and exp.value and pkg.value and exp.value < pkg.value:
    inconsistent.append("expiry_date")      # une DLC avant l'emballage est absurde
```

- La comparaison `exp.value < pkg.value` est **lexicale sur des chaînes ISO** YYYY-MM-DD — l'ordre lexical coïncide avec l'ordre chronologique. (C'est exactement pourquoi le mobile reconvertisse toujours DD/MM/YYYY → ISO avant d'envoyer, §20.2.)

```
for name in rule_set.required_fields:
    ev = by_name.get(name)
    if ev is None or ev.value is None:
        missing_required.append(name)
    elif ev.combined_confidence < thresholds.review_below:
        low_conf_required.append(name)
```

Phase 3 — verdict :

```
reasons = []
if missing_required: reasons.append("missing_required_field")
if low_conf_required: reasons.append("low_confidence_required_field")
if unverifiable: reasons.append("unverifiable_field")
if inconsistent: reasons.append("inconsistent_value")
if security_flags: reasons.append("security_flag")
outcome = GateOutcome.NEEDS_REVIEW if reasons else GateOutcome.EXTRACTED
```

- La moindre raison → revue humaine. Jamais d'acceptation silencieuse d'un défaut.
- `tuple(dict.fromkeys(inconsistent))` dans le retour : dédoublonne **en conservant l'ordre** (les dicts Python ≥ 3.7 sont ordonnés) — l'astuce standard remplaçant `set()` quand l'ordre compte.

9.4 domain/interim_fields.py — préviews déterministes (regex)

Contexte : entre l'OCR (~2 s) et le LLM (~10 s), l'écran mobile serait vide. Ce module extrait par **regex conservatrices** quelques champs évidents pour affichage anticipé. Règles absolues (docstring) :

1. n'émettre que sur un match **étiqueté explicitement** (« DLC : 12/08/2026 », jamais une date nue);
2. émettre la **même forme canonique** que le LLM (YYYY-MM-DD, 0-4 C, 8.95 EUR) pour que le run final *confirme* la préview au lieu de la faire clignoter;
3. plusieurs candidats distincts pour un champ → **ne rien émettre** (l'ambiguïté appartient au LLM);
4. sortie **non autoritaire** : rien ici n'alimente le gate, la réconciliation ou le HACCP.

Morceaux choisis :

```
def _norm(text):
    t = text.lower().replace("'", "")
    for src, dst in (("à", "a"), ("â", "a"), ("é", "e"), ("è", "e"), ("ê", "e"),
                    ("î", "i"), ("ô", "o"), ("û", "u"), ("ç", "c")):
        t = t.replace(src, dst)
    return t
```

- Normalisation minimaliste (minuscules + accents pliés) pour que « À CONSOMMER JUSQU'AU » matche `a consommer jusqu'au` quelle que soit la typographie OCR.

```
_EXPIRY_KEYS = (r"d\c", r"a consommer jusqu'?\s?au", r"a consommer avant le",
                r"a consommer avant", r"use by", r"exp\?.?")
for m in re.finditer(rf"{key}\s*[:.]\s*(?:le\s+)?.{0,6}}?{_DATE_ALT}", norm_text):
```

- La date doit suivre la clé à ≤ 6 caractères de « bruit » (`{0,6}?`, non-greedy) — une date ailleurs sur l'étiquette n'est **pas** attribuée à la clé. Dans une f-string regex, les accolades

quantificateurs se doublent : $\{\{0,6\}\}$.

```
def _normalize_date_match(m):
    if m.group(1) is not None:      # branche ISO
        return _valid_ymd(int(m.group(1)), int(m.group(2)), int(m.group(3)))
    a, b = int(m.group(4)), int(m.group(5))
    ...
    if a > 12 and b <= 12: return _valid_ymd(year, b, a) # a = jour
    if b > 12 and a <= 12: return _valid_ymd(year, a, b) # b = jour
    return None # les deux ≤ 12 → ordre ambigu → jamais deviner
```

- La même règle de désambiguïsation que le prompt LLM : un composant > 12 est forcément le jour; 04/05/2026 (4 mai ou 5 avril?) est abandonné. Cohérence totale entre les trois extracteurs (regex, LLM, humain).

```
return tuple(
    InterimField(name=name, value=next(iter(values)))
    for name, values in candidates.items()
    if len(values) == 1 # exactement UN candidat, sinon rien
)
```

9.5 domain/reconciliation.py — fusion GS1 ↔ LLM

Invariants (docstring, à connaître par cœur) :

- le gate tourne sur les champs LLM **inchangé**; les champs GS1 le **contournent** (leur ancrage est la symbologie du code-barres, pas le texte OCR);
- **conflit** → **GS1 gagne**, la valeur LLM perdante est consignée en warning (jamais supprimée en silence);
- les invariants de stockage tiennent : valeur non nulle ⇒ provenance + artefact source; nulle ⇒ ni l'un ni l'autre;
- l'**outcome** du run reste la décision du gate — reconcile() ne le recalcule pas (c'est adjusted_outcome qui le fait, séparément).

```
@dataclass(frozen=True)
class ReconciledField:
    field_name: str; value: str | None
    evidence: tuple[str, ...] | None
    provenance: dict[str, Any] | None
    source_raw_artifact_id: str | None
    validation_status: str; warnings: tuple[str, ...]
    llm_confidence: float | None; ocr_confidence: float | None
    combined_confidence: float; confidence_band: str
    source: str # 'llm' | 'gs1' | 'human'
```

- Le **sur-ensemble persistable** : exactement les colonnes de extracted_field. Le worker n'aura qu'à écrire bêtement.

```
def _from_evaluated(ef, *, ocr_artifact_id):
    has_value = ef.value is not None
    provenance = {"raw_artifact_id": ocr_artifact_id,
                  "spans": [...]} if has_value else None
    return ReconciledField(..., source="llm")
```

```
def _gs1_candidates(gs1, *, image_artifact_id):
    # lot → batch_number (AI 10)
    # DLC (AI 17) → expiry_date ; sinon DDM (AI 15) AVEC warning explicite :
    # "DDM (AI 15) utilisée faute de DLC (AI 17) – date de durabilité,
    # pas une limite de consommation"
    # GTIN (AI 01), packaging_date (AI 13), weight (AI 310x, unité kg en provenance)
```

- La distinction DLC/DDM est un point de sécurité alimentaire : une DDM (durabilité minimale) n'est **pas** une limite de consommation ; le fallback est autorisé mais tracé.

```
def reconcile(evaluated, gs1, *, ocr_artifact_id, image_artifact_id):
    by_name = {ef.name: _from_evaluated(ef, ...) for ef in evaluated}
    for gs1_field in _gs1_candidates(gs1, ...):
        existing = by_name.get(gs1_field.field_name)
        if existing is not None and existing.value is not None \
            and existing.value != gs1_field.value:
            conflict = (f"Conflit OCR/LLM↔GS1 sur {gs1_field.field_name} : "
                        f"LLM='{existing.value}' vs GS1='{gs1_field.value}' "
                        f"résolu en faveur du GS1.",)
            gs1_field = ReconciledField(**{**gs1_field.__dict__,
                                           "warnings": gs1_field.warnings + conflict})
        by_name[gs1_field.field_name] = gs1_field
    return list(by_name.values())
```

- `ReconciledField(**{**obj.__dict__, "warnings": ...})` : recopie d'une dataclass **frozen** avec un champ modifié (équivalent artisanal de `dataclasses.replace`).

adjusted_outcome — l'« Option Y »

Après réconciliation, faut-il encore une revue humaine ? Le gate ne connaissait pas le GS1 ; cette fonction **re-dérive** l'outcome sans jamais modifier le gate :

```
_CRITICAL_CONFLICT_FIELDS = frozenset({"batch_number", "expiry_date"})

def adjusted_outcome(verdict, reconciled) -> GateOutcome:
    gs1_ok = {n for n, f in by_name.items() if f.source == "gs1" and f.value is not None}
    gs1_conflict = {n for n, f in by_name.items()
                    if f.source == "gs1" and any(w.startswith("Conflit") for w in f.warnings)}

    # D1 - conflit code-barres ↔ imprimé sur lot ou DLC : TOUJOURS revue
    if gs1_conflict & _CRITICAL_CONFLICT_FIELDS:
        return GateOutcome.NEEDS_REVIEW

    residual = False
    # GS1 efface "manquant"/"confiance basse" sur les champs qu'il fournit
    if set(verdict.missing_required) - gs1_ok: residual = True
    if set(verdict.low_confidence_required) - gs1_ok: residual = True
    # D2 - la fabrication BLOQUE toujours, GS1 ou pas
    if verdict.unverifiable or verdict.security_flags: residual = True
    # D3 - incohérences restantes + re-check chronologique sur valeurs GS1
    ...
    return GateOutcome.NEEDS_REVIEW if residual else GateOutcome.EXTRACTED
```

Trois décisions à retenir :

- **D1** : un lot ou une DLC imprimés différents du code-barres = anomalie d'intégrité d'étiquetage — un humain **doit** voir ça, même si la valeur stockée est GS1-exacte.
- **D2** : un modèle qui hallucine mérite un œil humain même si le GS1 a corrigé la valeur — le flag de fabrication n'est jamais blanchi.
- Sans données GS1, la fonction retourne exactement `verdict.outcome` : aucun changement de comportement pour les scans sans code-barres.

10. Contexte ingestion — la couche application

Cas d'usage + ports. Toujours zéro framework (contrat application-purity).

10.1 application/ports.py — les ports

```
class RawStore(Protocol):
    def put(self, content: bytes, *, checksum: str) -> str: ...
    def exists(self, *, checksum: str) -> bool: ...
```

Concept — typing.Protocol (structural typing). Contrairement à une classe abstraite, un Protocol ne s'hérite pas : toute classe qui *a la bonne forme* (mêmes méthodes/signatures) satisfait le port, vérifiée statiquement par le type-checker. C'est le «duck typing» rendu vérifiable. Les adapters (FilesystemRawStore, fakes de test) n'importent même pas ce module.

Le docstring de put **est le contrat** : durable (fsync) *avant* de retourner, contenu-adressé, idempotent (re-put du même contenu = no-op qui rend la même ref). Un adapter qui ne respecterait pas ça casserait la garantie «rien n'est perdu» du système — le contrat vit dans le port, pas dans l'implémentation.

Les autres ports du fichier, avec leurs types de résultat :

```
@dataclass(frozen=True)
class PersistResult:
    ingestion_id: str
    created: bool      # False => replay idempotent d'une ingestion existante

class IngestionWriteRepository(Protocol):
    def persist(self, *, content_sha256, storage_ref, barcode_raw,
                  client_captured_at, principal, route,
                  audit: AuditContext, action: str) -> PersistResult: ...

@dataclass(frozen=True)
class OverriddenField:
    run_id: str; field_name: str; value: str | None
    validation_status: str; source: str
    combined_confidence: float; confidence_band: str
    replayed: bool

class FieldOverrideRepository(Protocol):
    def override_field(self, *, ingestion_id, field_name, value, note,
                       audit, action, idempotency_key=None) -> OverriddenField | None: ...

class ConfirmIngestionRepository(Protocol):
    def confirm(self, *, ingestion_id, audit, action) -> ConfirmedIngestion | None: ...
```

```
class ConfirmNotAllowed(Exception):
    def __init__(self, status: str): ...
```

- Convention de retour uniforme : None = ressource introuvable (le routeur mappe → 404); exception typée = règle métier violée (→ 409); replayed=True = répétition idempotente (aucune écriture).

10.2 application/submit_ingestion.py — le cas d’usage d’upload

Le contrat de durabilité, dans l’ordre exact :

```
class SubmitIngestion:
    def __init__(self, raw_store: RawStore, repository: IngestionWriteRepository):
        self._raw_store = raw_store
        self._repository = repository

    def __call__(self, cmd: SubmitIngestionCommand) -> IngestionAccepted:
        # garde-fous de frontière (pas de logique métier)
        if not cmd.image_bytes:
            raise ValueError("empty payload: no image bytes")
        if cmd.content_type not in _ACCEPTED_TYPES:
            raise ValueError(f"unsupported media type: {cmd.content_type}")

        checksum = hashlib.sha256(cmd.image_bytes).hexdigest() # (1)
        storage_ref = self._raw_store.put(cmd.image_bytes, checksum=checksum) # (2)
        result = self._repository.persist( # (3)
            content_sha256=checksum, storage_ref=storage_ref, ...,
            audit=AuditContext(actor_id=cmd.actor_id,
                               correlation_id=cmd.correlation_id,
                               trace_id=cmd.trace_id),
            action="ingestion.submitted",
        )
        return IngestionAccepted(ingestion_id=result.ingestion_id, # (4)
                                status=IngestionStatus.RAW_STORED.value,
                                http_status=202, replayed=not result.created)
```

L’ordre (1)→(2)→(3)→(4) **est** la garantie :

- (2) avant (3) : les octets sont durables (fsync) **avant** qu’une ligne DB existe. Crash entre les deux ? Un fichier orphelin contenu-adressé traîne — inoffensif, ré-uploadable.
- (3) est une transaction atomique et idempotente (§11.2). Crash pendant ? Rollback complet, le client retente avec la même photo, le même hash retombe sur ses pieds.
- (4) le 202 ne part **qu’après** le commit. Le client qui reçoit 202 a la certitude absolue que rien ne sera perdu.

Concept — `__call__`. Définir `__call__` rend l’objet invocable comme une fonction : `use_case(cmd)`. Idiomme « cas d’usage = un objet, une opération » : les dépendances par le constructeur, l’exécution par l’appel. Facile à injecter, facile à faker.

Concept — Command object. `SubmitIngestionCommand` est une dataclass frozen qui porte **tout** ce dont le cas d’usage a besoin (bytes, content-type, actor, correlation, principal...). Le routeur la construit; le cas d’usage ne touche jamais à `Request`. Frontière transport/métier nette.

10.3 application/extraction_ports.py — les ports providers

```
@dataclass(frozen=True)
class OcrResult:
```



```

raw_json: bytes          # la réponse provider VERBATIM (stockée immuable)
full_text: str
mean_confidence: float
page: int = 1

@dataclass(frozen=True)
class LlmResult:
    raw_json: bytes
    fields: tuple[LlmField, ...]
    extractor_version: str
    model: str
    prompt_version: str

class OcrProvider(Protocol):
    def run(self, image_bytes: bytes) -> OcrResult: ...
    @property
    def name(self) -> str: ...

class LlmExtractor(Protocol):
    def run(self, ocr_text: str, known_field_names: tuple[str, ...] = ()) -> LlmResult: ...
    @property
    def model(self) -> str: ...

```

- `raw_json`: bytes dans les deux résultats : la réponse **brute** du provider est archivée telle quelle dans le store immuable. Pour l'audit et le débogage, on peut toujours rejouer « qu'a répondu le modèle ce jour-là ».
- `known_field_names` : l'indice « ces champs sont déjà résolus par GS1, ne les ré-extrais pas ». **Consultatif seulement** — la correction ne dépend jamais de son respect, la réconciliation fera de toute façon gagner GS1. Distinguer optimisation et correction, c'est ça.
- Partage des responsabilités (docstring) : la **conformité de forme** (JSON valide au bon schéma) est le travail de l'adapter; la **vérité** (pas de fabrication) est le travail du gate. Aucun des deux ne fait le travail de l'autre.

10.4 application/override_field.py — la correction humaine (HITL)

Constantes de gouvernance :

```

FIELD_NAMES: frozenset[str] = frozenset({
    "commercial_designation", "scientific_name", "producer_name", "reseller_brand",
    "batch_number", "origin_country", "FAO_area", "production_method",
    "fishing_gear_or_farming_method", "expiry_date", "packaging_date",
    "storage_temperature", "allergens", "health_mark", "weight", "price", "gtin",
    "product_name", "supplier_name", # legacy v1 – runs historiques immuables
})

GS1_OWNED_FIELDS = frozenset({"batch_number", "expiry_date", "weight", "gtin", "packaging_date"})

```

- `FIELD_NAMES` est le **miroir** de la contrainte CHECK en DB (élargie par la migration 0011) et de la liste de l'adapter LLM. Un champ inconnu est rejeté avant même de toucher la DB. Les noms v1 restent éditables : les runs historiques sont immuables, un revieweur doit pouvoir encore les corriger.
- Les champs GS1 sont protégés : les écraser demande le drapeau explicite `force_gs1` (un désaccord code-barres ↔ opérateur est une anomalie d'intégrité qui mérite sa propre action d'audit `ingestion.gs1_field_overridden`, requêtable séparément des corrections ordinaires).

```

class OverrideField:
    def __call__(self, cmd: OverrideFieldCommand) -> OverriddenField:
        if cmd.field_name not in FIELD_NAMES:
            raise UnknownField(cmd.field_name)          # → 400
        gsl_owned = cmd.field_name in GS1_OWNED_FIELDS
        if gsl_owned and not cmd.force_gsl:
            raise FieldNotEditable(cmd.field_name)      # → 409

        # blanc = champ effacé (null) – cohérent avec "null ⇒ pas de provenance"
        value = cmd.value.strip() if cmd.value and cmd.value.strip() else None

        result = self._repository.override_field(
            ..., action=_ACTION_GS1 if gsl_owned else _ACTION,
            idempotency_key=cmd.idempotency_key)
        if result is None:
            raise IngestionNotFound(cmd.ingestion_id)  # → 404
        return result

```

- Le cas d’usage valide la **règle** (champ connu, éditabilité); l’écriture append-only complexe est déléguée au port. Chaque exception a son mapping HTTP documenté dans sa propre docstring — le routeur n’a plus qu’à traduire.

10.5 application/confirm_ingestion.py — finaliser la revue

```

class ConfirmIngestion:
    def __call__(self, cmd) -> ConfirmedIngestion:
        result = self._repository.confirm(ingestion_id=cmd.ingestion_id,
                                          audit=AuditContext(...),
                                          action="ingestion.confirmed")

        if result is None:
            raise IngestionNotFound(cmd.ingestion_id)
        return result

```

- Sémantique : les overrides enregistrent **ce qui** a été corrigé ; le confirm enregistre **que la revue a eu lieu** (transition terminale `confirmed`, audité avec l’opérateur comme acteur). Les consommateurs aval (exports, reporting HACCP) distinguent ainsi « extrait machine » de « revu humain » sans reconstruction.
- Les règles d’état (seuls `extracted` / `needs_review` / `ocr_skipped_garbage` sont confirmables ; `confirmed` rejoue ; le reste → `ConfirmNotAllowed` → 409) vivent dans le repository, sous verrou — car elles nécessitent l’état courant de la ligne.

11. Contexte ingestion — adapters (1/2) : stockage et providers

11.1 adapters/filesystem_raw_store.py — le store d'objets crash-durable

```
class FilesystemRawStore:
    def __init__(self, base_dir: str):
        self._base = Path(base_dir)
        self._base.mkdir(parents=True, exist_ok=True)

    def _path_for(self, checksum: str) -> Path:
        return self._base / checksum[:2] / checksum # shard par 2 premiers hex
```

- **Contenu-adressé** : le chemin du fichier EST son hash SHA-256. Deux uploads identiques → même chemin → dédup gratuite. Le sharding ab/abcdef... évite un répertoire plat à 100 000 fichiers (lent sur la plupart des FS).

```
def put(self, content: bytes, *, checksum: str) -> str:
    path = self._path_for(checksum)
    ref = f"{checksum[:2]}/{checksum}"
    if path.exists():
        return ref # (1) déjà durable → no-op idempotent
    path.parent.mkdir(parents=True, exist_ok=True)
    tmp = path.with_name(path.name + ".tmp." + os.urandom(6).hex())
    with open(tmp, "wb") as f:
        f.write(content)
        f.flush()
        os.fsync(f.fileno()) # (2) les octets touchent le disque
    os.replace(tmp, path) # (3) publication atomique
    dir_fd = os.open(path.parent, os.O_DIRECTORY)
    try:
        os.fsync(dir_fd) # (4) le rename lui-même est durable
    finally:
        os.close(dir_fd)
    return ref
```

Le rituel « écriture atomique durable », à connaître par cœur :

- (2) `flush()` vide le buffer Python vers l'OS ; `fsync()` force l'OS à écrire **sur le disque**. Sans `fsync`, un crash machine peut perdre des données que `write()` disait écrites.
- (3) `os.replace` est un rename **atomique** (POSIX) : aucun lecteur ne verra jamais un fichier à moitié écrit — soit l'ancien état (absent), soit le nouveau (complet). Le suffixe aléatoire du tmp évite la collision entre deux writers concurrents.
- (4) Le piège que 99 % des devs ignorent : le rename est une **écriture dans le répertoire**. Sans `fsync` du répertoire, un crash peut faire disparaître l'entrée alors que le fichier était `fsyncé`. On ouvre le répertoire (`0_DIRECTORY`) et on le `fsync` aussi.

Tip dev. Ce quatuor write-tmp → fsync → rename → fsync-dir est LE motif pour toute écriture de fichier qui ne doit jamais être perdue ni lue à moitié (configs, snapshots, queues sur disque). En prod, ce port serait implémenté par S3 — même contrat, autre adapter.

11.2 adapters/sql_ingestion_repository.py — l'écriture durable et idempotente

Une transaction fait tout, dans cet ordre :

```
_CLAIM = text(
    "INSERT INTO platform.idempotency_key "
    "(scope_hash, request_fingerprint, state, response_status, response_body_ref, expires_at) "
    "VALUES (:sh, :fp, 'completed', 202, :iid, now() + interval '24 hours') "
    "ON CONFLICT (scope_hash) DO NOTHING "
    "RETURNING response_body_ref")
```

Concept — claim atomique par ON CONFLICT DO NOTHING ... RETURNING. L'insert tente de «réclamer» le scope. Deux issues, **sans course possible** : - il insère → RETURNING rend la valeur → nous sommes le premier; - conflit d'unicité → DO NOTHING → RETURNING rend **zéro ligne** (scalar_one_or_none() → None) → quelqu'un est passé avant.

C'est un test-and-set en une requête. La version « SELECT puis INSERT » aurait une fenêtre de course; celle-ci non, car l'unicité est arbitrée par l'index.

```
@staticmethod
def _scope_hash(principal, route, content_sha256):
    return hashlib.sha256(f"{principal}:{route}:{content_sha256}".encode()).hexdigest()
```

- La clé d'idempotence est le **hash du contenu**, scopé par principal et route : le même appareil qui renvoie la même photo (retry réseau, double-tap) retombe sur le même scope. Remarque : ce n'est *pas* le header Idempotency-Key qui sert de clé ici — la dédup par contenu est plus forte (deux clés différentes pour la même photo dédupliquent quand même).

```
def persist(self, *, content_sha256, storage_ref, ..., audit, action):
    scope_hash = self._scope_hash(principal, route, content_sha256)
    ingestion_id = str(uuid.uuid4())          # id généré AVANT le claim
    with self._engine.begin() as conn:
        set_audit_context(conn, ...)          # (1) sinon les triggers refusent
        claimed = conn.execute(_CLAIM, {"sh": scope_hash, "fp": content_sha256,
                                         "iid": ingestion_id}).scalar_one_or_none()
        if claimed is None:                   # (2) duplicata → relire l'existant
            existing = conn.execute(_FIND_EXISTING, {"sh": scope_hash}).scalar_one()
            return PersistResult(ingestion_id=str(existing), created=False)
        conn.execute(_INSERT_INGESTION, {...}) # (3) l'agrégat (status='raw_stored')
        conn.execute(_INSERT_RAW, {...})      # (4) l'artefact image (append-only)
        conn.execute(_ENQUEUE_OUTBOX, {...})  # (5) l'événement, MÊME transaction
        return PersistResult(ingestion_id=ingestion_id, created=True)
```

- L'id est généré **avant** le claim et stocké dans response_body_ref : la ligne d'idempotence porte déjà la réponse à rejouer. Un duplicata ne relit que cette ligne — zéro écriture.
- (5) est le **transactional outbox** en action : l'événement ingestion.raw_stored (payload : ingestion_id, checksum, image_ref) existe si et seulement si l'ingestion a committé. Ce n'est pas un appel au worker — juste une ligne que le relais ramassera.

11.3 adapters/google_vision_ocr.py — l'adapter OCR

Décisions structurantes :

```

_FEATURE = "DOCUMENT_TEXT_DETECTION"      # modèle texte dense (meilleur rappel étiquettes)
_ALLOWED_FEATURES = frozenset({"DOCUMENT_TEXT_DETECTION", "TEXT_DETECTION"})
_REQUEST_TIMEOUT_S = 30.0
_LANGUAGE_HINTS = ["fr", "en"]            # aide au rappel fr+en, jamais une restriction

```

- TEXT_DETECTION (moins cher/plus rapide) est **exposé en config mais pas par défaut** : le trade-off de rappel doit être mesuré avant adoption. Une valeur invalide échoue **bruyamment** au démarrage (validation dans `__init__`), jamais de fallback silencieux.

```

def __init__(self, *, api_key, endpoint=ENDPOINT, timeout_s=30.0, feature=_FEATURE):
    if not api_key: raise ValueError(...)
    if feature not in _ALLOWED_FEATURES: raise ValueError(...)
    ...
    self._client = None      # client HTTP créé paresseusement au 1er run()

```

- **Client HTTP partagé et paresseux** : la construction ne fait aucune I/O ; le premier `run()` crée un `httpx.Client` réutilisé à vie. Raison mesurée : le handshake TCP+TLS par appel coûtait ~600 ms sur l'egress observé — le keep-alive les économise sur chaque photo suivante.
- `import httpx` est fait **dans les méthodes**, pas en tête de module : importer le paquet `labelscan` ne requiert jamais `httpx` (tests sans réseau, outils CLI...).

```

def run(self, image_bytes: bytes) -> OcrResult:
    payload = {"requests": [{
        "image": {"content": base64.b64encode(image_bytes).decode("ascii")},
        "features": [{"type": self._feature}],
        "imageContext": {"languageHints": _LANGUAGE_HINTS}}]}
    try:
        resp = self._http_client().post(self._endpoint,
                                         params={"key": self._api_key},
                                         json=payload)
    except httpx.HTTPError as exc:
        raise RuntimeError(f"google-vision request failed: {type(exc).__name__}") from None
    if resp.status_code != 200:
        raise RuntimeError(f"google-vision returned HTTP {resp.status_code}")
    raw = resp.content
    full_text, mean_confidence = _parse_annotate_response(json.loads(raw))
    return OcrResult(raw_json=raw, full_text=full_text,
                    mean_confidence=mean_confidence, page=1)

```

- **Sanitisation de sécurité** : la clé API voyage en query param (`?key=...`), donc le texte d'une exception `httpx` pourrait contenir l'URL **avec la clé**. D'où `raise ... from None` (supprime la cause) et un message réduit au nom de la classe d'erreur. Idem pour un statut $\neq 200$: le statut seulement, jamais le corps.
- L'image part **verbatim** (pas de redimensionnement serveur) — c'est le mobile qui downscale à ~1600 px avant l'envoi (\$21.1) ; le serveur ne dégrade jamais ce qu'on lui confie.

```

def _parse_annotate_response(data) -> tuple[str, float]:
    responses = data.get("responses") or []
    first = responses[0] if responses else {}
    error = first.get("error")
    if error: raise RuntimeError(f"google-vision annotate error: {error.get('message', 'unknown')}")
    fta = first.get("fullTextAnnotation") or {}
    full_text = fta.get("text") or ""
    if not full_text:
        annotations = first.get("textAnnotations") or []
        if annotations: full_text = annotations[0].get("description", "") or ""
    pages = fta.get("pages") or []
    confidences = [p.get("confidence") for p in pages]

```

```

        if isinstance(p.get("confidence"), (int, float)):
    mean_confidence = float(sum(confidences)/len(confidences)) if confidences else 0.0
    return full_text, mean_confidence

```

- Parsing **défensif** : chaque niveau utilise `.get(...)` or `[]` (l'API peut omettre n'importe quel bloc). Un bloc `error` par-requête devient une erreur provider (retryable côté consumer).
- Confiance absente → 0.0, jamais une valeur inventée — et le gate qualité OCR (§9.3) sait précisément qu'un 0.0 signifie « non scoré », pas « nul ».

11.4 adapters/claude_llm_provider.py — l'adapter LLM

Le plus riche en décisions d'ingénierie LLM. Configuration :

```

_DEFAULT_MODEL = "claude-haiku-4-5"
_MODEL = os.environ.get("LABELSCAN_LLM_MODEL", _DEFAULT_MODEL)
_REQUEST_TIMEOUT_S = 120.0
_PROMPT_VERSION = "seafood-label-extraction/v2.1.0"

_DEFAULT_ESCALATION_MODEL = "claude-opus-4-8"
_ESCALATION_MODEL = os.environ.get("LABELSCAN_LLM_ESCALATION_MODEL", _DEFAULT_ESCALATION_MODEL)
_ESCALATION_ENABLED = _env_bool("LABELSCAN_LLM_ESCALATION_ENABLED", False)

_PROMPT_CACHE_ENABLED = _env_bool("LABELSCAN_LLM_PROMPT_CACHE_ENABLED", True)
_PROMPT_CACHE_TTL = (os.environ.get("LABELSCAN_LLM_PROMPT_CACHE_TTL") or "1h").strip() or "1h"

```

Architecture coût/latence (les commentaires du fichier la documentent) :

- **Pipeline hybride** : GS1 traite les champs exacts (lot/DLC/poids/GTIN) → le LLM ne traite que le texte libre → un modèle **petit et rapide (Haiku)** suffit. Opus reste le tiers d'**escalade**, OFF par défaut.
- **Prompt caching** : le gros bloc système statique est facturé $\sim 0.1\times$ quand il est en cache. Deux subtilités critiques : (1) Haiku ne cache que les préfixes **≥ 4096 tokens** — le bloc est écrit au-dessus de ce plancher, sinon le cache ne s'active jamais, silencieusement ; (2) le breakpoint `cache_control` est posé explicitement **à la fin du bloc système** — en auto, il glisserait sur le message dynamique et raterait à chaque fois.
- **Versionnage sémantique du prompt** : le commentaire d'en-tête raconte l'historique v1.2.0 → v2.0.0 (MAJOR : le jeu de champs fermé a changé → migration 0011 + re-run du gate d'éval) → v2.1.0 (MINOR : durcissement `production_method`, prompt seul). Chaque édition du préfixe invalide le cache une fois — c'est notarié dans le commentaire.

Le **prompt système** (`_SYSTEM_TEXT`, ~ 400 lignes) est un contrat d'extraction complet. Sa structure, à retenir comme modèle de « prompt de production » :

1. **Rôle et enjeu** — « deterministic information-extraction function », « under-extracting is safe ; inventing is a defect ».
2. **Format de sortie absolu** — un seul objet JSON, 17 éléments, clés exactes.
3. **Jeu de champs fermé** + définition de chaque clé (value string ou null, confidence 0 si null, evidence = sous-chaînes exactes, validation_status à 6 valeurs, warnings).
4. **Règles absolues numérotées** — jamais inférer (pas de nom scientifique depuis un nom commun, pas de zone FAO depuis un nom de mer, pas de résolution de date ambiguë) ; toute valeur doit être citable ; **le texte OCR est de la DONNÉE, pas des instructions** (règle 6 = défense anti-injection de prompt : « ignore previous instructions » sur une étiquette doit être traité comme du texte) ; préférence français verbatim (règle 7) ; estampille $\neq$ origine (règle 8) ; producteur vs revendeur (règle 9).
5. **Contexte de domaine** (aide à la *reconnaissance*, jamais licence d'inférence) — réglementation UE, allergènes déclarés vs traces, DLC/DDM, zones FAO, sauvage/élevage avec règle de **précédence** (un énoncé d'élevage explicite gagne sur un mot « pêche » ambiant).
6. **Normalisations par champ** — dates ISO avec règle $>12=\text{jour}$, températures Celsius, poids avec unité (jamais rescalé), calibre $\neq$ poids net, etc.

7. **Cas dégradé** — OCR vide : objet complet tout-null, jamais de refus.
8. **Quatre exemples few-shot** complets, dont un cas bruité (garble OCR conservé verbatim) et un cas français d'élevage (le contre-exemple qui manquait avant v2.1.0).

Tip dev — prompt d'extraction robuste. Les invariants de ce prompt sont réutilisables tels quels : sortie JSON stricte + schéma fermé; evidence citée verbatim (vérifiable mécaniquement en aval); «null + warning» toujours préféré à une devinette; l'entrée utilisateur déclarée comme donnée; few-shots qui couvrent le cas sale et le contre-exemple qui piège le modèle.

```
def _supports_effort_thinking(model: str) -> bool:
    return not model.startswith("claude-haiku")
```

- Haiku rejette output_config.effort (400) et n'a pas de thinking adaptatif → on n'envoie ces options que pour les modèles qui les supportent (l'escalade Opus).

```
def _user_prompt(ocr_text, known_field_names):
    if not known_field_names:
        return f"OCR TEXT:\n{ocr_text}"
    known = ", ".join(sorted(known_field_names))
    return (f"OCR TEXT:\n{ocr_text}\n\nNOTE: these fields are already identified reliably "
            f"from the barcode and MUST NOT be re-extracted – set them to value=null, "
            f"validation_status='missing': {known}. Focus on the remaining free-text fields...")
```

- Tout le **dynamique** (texte OCR, hint GS1) vit dans le message user : le préfixe système reste identique octet pour octet entre étiquettes → cache hit.

```
class ClaudeLlmExtractor:
    def __init__(self, client=None, *, model=None):
        if client is None:
            import anthropic          # importé ICI seulement
            client = anthropic.Anthropic()
        self._client = client
        self._model = model or _MODEL
```

- Import paresseux + client injectable : les tests passent un faux client, le paquet s'importe sans SDK. model par instance : la composition root construit **deux** instances du même adapter (primaire Haiku, escalade Opus) derrière le même port.

```
def run(self, ocr_text, known_field_names=()) -> LlmResult:
    system_block = {"type": "text", "text": _SYSTEM_TEXT}
    if _PROMPT_CACHE_ENABLED:
        system_block["cache_control"] = {"type": "ephemeral", "ttl": _PROMPT_CACHE_TTL}
    output_config = {"format": {"type": "json_schema", "schema": _OUTPUT_SCHEMA}}
    kwargs = {"model": self._model, "max_tokens": 4096,
              "system": [system_block],
              "messages": [{"role": "user",
                           "content": _user_prompt(ocr_text, known_field_names)}]}
    if _supports_effort_thinking(self._model):
        kwargs["thinking"] = {"type": "adaptive"}
        output_config["effort"] = "high"
        kwargs["output_config"] = output_config

    resp = self._client.with_options(timeout=_REQUEST_TIMEOUT_S).messages.create(**kwargs)
```

- **Structured outputs** (json_schema) : l'API garantit un objet conforme au schéma _OUTPUT_SCHEMA (miroir compact d'extraction.v1 : additionalProperties: false, enum des 17 noms, enum des 6 statuts). La *forme* est garantie; la *vérité* reste au gate.
- with_options(timeout=120) : le défaut SDK est 600 s — un appel bloqué gêlerait le worker; 120 s le borne, l'APITimeoutError résultante est traitée comme transitoire par le consumer.

```

usage = getattr(resp, "usage", None)
_log.info("llm_cache_usage", extra={
    "model": self._model,
    "cache_creation_input_tokens": getattr(usage, "cache_creation_input_tokens", 0) or 0,
    "cache_read_input_tokens": getattr(usage, "cache_read_input_tokens", 0) or 0})
if resp.stop_reason == "refusal":
    raise RuntimeError(f"LLM refused extraction: {getattr(resp, 'stop_details', None)}")

text_out = next((b.text for b in resp.content if b.type == "text"), "")
data = json.loads(text_out)
fields = tuple(LlmField(name=f["name"], value=f["value"],
                        llm_confidence=float(f["confidence"]),
                        evidence=tuple(f.get("evidence") or ()),
                        validation_status=f["validation_status"],
                        warnings=tuple(f.get("warnings") or ()))
               for f in data["fields"])
return LlmResult(raw_json=text_out.encode(), fields=fields,
                 extractor_version=_PROMPT_VERSION, model=self._model,
                 prompt_version=_PROMPT_VERSION)

```

- **Observabilité du cache** : les deux compteurs de tokens par appel prouvent (ou infirment) le hit-rate du cache — champs allow-listés du logger, jamais le contenu du prompt.
- `next((b.text for b in resp.content if b.type == "text"), "")` : premier bloc texte de la réponse, "" par défaut — idiome `next`(générateur, défaut).
- `tuple(... or ())` : normalise null JSON → tuple vide, et fige l’immuabilité des value objects du domaine.

12. Contexte ingestion — adapters (2/2) : le consumer d'extraction, les repos de revue, les routeurs HTTP

12.1 adapters/extraction_consumer.py — l'orchestrateur (923 lignes)

Le handler outbox de ingestion.raw_stored. Son **modèle transactionnel** (docstring d'en-tête) est la clé de toutes les garanties :

- Étapes 1-2 (OCR, LLM) : chacune dans **sa propre transaction committée**, avec la réponse brute stockée en raw_artifact immuable. **L'existence de l'artefact est le garde de dédup** : au retry, l'artefact est déjà là → le provider n'est PAS rappelé (zéro double appel externe, zéro double facturation).
- Étape 3 (gate) : pure, en mémoire.
- Étape 4 (persistance run + champs + statut + événement suivant) : sur **la connexion du worker**, donc atomique avec processed_event + published. Crash avant commit → l'étape 4 est rollbackée, le retry la rejoue — mais retombe sur les artefacts OCR/LLM existants → exactement un run committé au final.

Préliminaires

```
class _RelayMessage(Protocol):
    id: str; payload: dict; correlation_id: str; trace_id: str
```

- Type **structurel** du message : le consumer (contexte ingestion) ne peut pas importer platform.outbox (contrat import-linter §6.2) — il décrit donc la forme attendue au lieu d'importer la classe. Le découplage producteur/relais tient jusque dans les types.

```
SYSTEM_ACTOR = "00000000-0000-0000-0000-000000000001"
```

- L'« acteur système » : les écritures initiées par le worker (pas d'humain) sont auditées sous ce uuid réservé.

```
_NON_RETRYABLE = (TypeError, AttributeError, NameError, ImportError)

def _with_provider_retry(self, fn):
    last = None
    for _ in range(self._max_provider_attempts): # 3 par défaut
        try:
            return fn()
        except _NON_RETRYABLE:
            raise # bug de NOTRE code → remonter, ne pas masquer
    except Exception as e:
```

```

        last = e          # panne provider présumée → retenter
    raise _ProviderExhausted(str(last))

```

Piège évité — retry aveugle. Retenter un `TypeError` est futile (c’est un bug, il échouera identiquement 3 fois) et surtout **masquerait le défaut** derrière un `run extraction_failed` d’apparence transitoire. La liste `_NON_RETRYABLE` fait remonter les bugs (→ DLQ du worker, traceback en `last_error`), et ne retente que les vraies pannes.

Les helpers `_ocr_to_json/_ocr_from_json/_llm_to_json/_llm_from_json` sérialisent les résultats providers en JSON **normalisé** (stable) pour le store — c’est cette forme normalisée qui est hashée et relue au replay.

`__call__` — le flux complet, étape par étape

```

def __call__(self, msg, worker_conn):
    ingestion_id = msg.payload["ingestion_id"]
    corr, trace = msg.correlation_id, msg.trace_id

    # (1) Socle déterministe D'ABORD : le code-barres (exact)
    image_bytes, image_artifact_id, barcode_raw = self._load_image_meta(ingestion_id)
    gsl = parse_gsl(barcode_raw)
    known = tuple(gsl_resolved_field_names(gsl))    # hint prompt (consultatif)

    # (2) OCR avec retry borné ; épuisement → run FAILED + événement consommé
    try:
        ocr, ocr_artifact_id = self._with_provider_retry(
            lambda: self._ensure_ocr(ingestion_id, image_bytes, corr, trace))
    except _ProviderExhausted as e:
        self._persist_failed(worker_conn, ingestion_id, corr, trace, error=str(e))
    return

```

- Sur épuisement, on écrit un `run extraction_failed` **et on retourne normalement** : l’événement est publié (échec métier enregistré, pas d’exception qui traverse) — pas de boucle infinie de retry au niveau outbox.

```

# (3) Gate qualité OCR : image illisible → skip du LLM (coût), GS1 conservé
if self._ocr_quality_gate_enabled and is_ocr_garbage(ocr.full_text,
                                                    ocr.mean_confidence,
                                                    self._ocr_quality):
    reconciled = reconcile((), gsl, ocr_artifact_id=..., image_artifact_id=...)
    self._persist_skipped_garbage(worker_conn, ingestion_id, reconciled, ...)
return

```

- `reconcile((), gsl, ...)` : zéro champ LLM, complément GS1 seul. Même une photo illisible garde ses champs code-barres.

```

# (4) Tier 3 : commit INTERMÉDIAIRE entre OCR et LLM (préviews + statut ocr_done)
try:
    self._persist_interim(ingestion_id, ocr, known, corr, trace)
except Exception as e:
    _log.warning("interim_persist_failed", extra={...})    # préview = jamais fatal

```

- Transaction **séparée, committée immédiatement** (pas `worker_conn`) : le client qui polle voit `ocr_done` + les préviews pendant que le LLM tourne encore. Best-effort assumé : un échec ici ne doit jamais faire échouer l’extraction (except `Exception` large + warning, exception à la règle habituelle, justifiée en commentaire).

```

# (5) LLM (retry borné, dédup par artefact+modèle) + instrumentation
llm = self._with_provider_retry(lambda: self._ensure_llm(..., llm=self._llm,
                                                         model=self._llm.model,

```

```

                                is_primary=True))
_log.info("extraction_timing", extra={"ocr_ms": ..., "llm_ms": ...,
                                     "image_bytes": len(image_bytes), ...})

```

- ocr_ms/llm_ms mesurés avec `time.monotonic()` (jamais `time.time()` pour des durées — l'horloge murale peut sauter). Au replay dédupé, ces durées ≈ 0 : la mesure isole le vrai coût externe.

```

# (6) Le gate anti-fabrication sur les champs LLM INCHANGÉS
verdict = self._gate(llm.fields, ocr)

```

```

# (7) Escalade deux-tiers (Work Item A) — au plus UNE fois, conditions strictes
recoverable = self._recoverable_free_text(verdict, known)
if self._escalation_enabled and self._escalation_llm is not None and recoverable:
    esc = self._with_provider_retry(lambda: self._ensure_llm(..., llm=self._escalation_llm,
                                                            is_primary=False))

    merged = _merge_raw_fields(llm.fields, esc.fields,
                              verdict.fields, self._gate(esc.fields, ocr).fields)
    verdict = self._gate(merged, ocr)          # RE-GATER le fusionné — jamais de relâche
    escalation_model = self._escalation_llm.model

```

Conditions d'escalade (`_recoverable_free_text`) : un champ **requis, texte libre** (pas GS1-owned), null ou sous le seuil, **non couvert par GS1**. Les champs GS1 sont exclus exprès : un conflit code-barres→imprimé doit rester `needs_review`, jamais « réparé » par un second passage LLM. Un échec de l'escalade est logué et **le verdict primaire tient** (best-effort).

`_merge_raw_fields` : champ par champ, garde le meilleur des deux (une valeur *groundée* de confiance supérieure), puis **le jeu fusionné repasse par `evaluate()`** — le gate reste l'unique source de vérité, l'escalade ne peut pas l'assouplir.

```

# (8) Réconciliation GS1 (GS1 gagne) puis outcome ré-dérivé (Option Y)
reconciled = reconcile(verdict.fields, gs1, ...)
outcome = adjusted_outcome(verdict, reconciled)

# (9) Persistance finale sur worker_conn + événement extraction.completed
self._persist(worker_conn, ingestion_id, outcome, reconciled, llm,
              ocr_artifact_id, corr, trace, escalation_model=escalation_model)

```

`_ensure_ocr / _ensure_llm` — le motif « artefact = garde de dédup »

```

def _ensure_ocr(self, ingestion_id, image_bytes, corr, trace):
    with self._engine.begin() as c:
        existing = c.execute(text(
            "SELECT id, checksum_sha256 FROM ingestion.raw_artifact "
            "WHERE ingestion_id=:id AND artifact_kind='ocr_json' LIMIT 1"), ...).first()
        if existing:
            # déjà fait → relire, pas rappeler
            raw = self._raw.read(checksum=existing["checksum_sha256"])
            return _ocr_from_json(raw), str(existing["id"])
        ocr = self._ocr.run(image_bytes)
        # ← appel externe (seulement sur miss)
        normalized = _ocr_to_json(ocr)
        checksum = _sha(normalized)
        ref = self._raw.put(normalized, checksum=checksum)
        set_audit_context(c, actor_id=SYSTEM_ACTOR, action="ingestion.ocr_stored", ...)
        artifact_id = c.execute(text("INSERT INTO ingestion.raw_artifact ... RETURNING id"), ...)
        return _ocr_from_json(normalized), str(artifact_id)

```

- Lecture du garde et insert de l'artefact dans **une** transaction dédiée, committée sur-le-champ : même si l'étape 4 est rollbackée plus tard, l'artefact survit et le retry ne rappellera pas Vision.

- `_ensure_llm` ajoute la dimension **modèle** : le garde est (`ingestion_id`, `'llm_output'`, `model`). Le primaire `match` aussi `model IS NULL` (lignes historiques d'avant la migration 0010) — une vieille ingestion ne sera jamais ré-extraite par Haiku au replay. Primaire et escalade ont ainsi chacun *au plus un* appel externe par ingestion.

`_persist_interim` — le commit intermédiaire (Tier 3)

```
interim = tuple(f for f in extract_interim_fields(ocr.full_text) if f.name not in gsl_known)
with self._engine.begin() as c:
    set_audit_context(c, ..., action="ingestion.ocr_done", ...)
    for f in interim:
        c.execute(text("INSERT INTO ingestion.interim_field ... "
                       "ON CONFLICT (ingestion_id, field_name) DO NOTHING"), ...)
    c.execute(text("UPDATE ingestion.ingestion SET status='ocr_done' "
                  "WHERE id=:id AND status IN ('raw_stored','ocr_running')"), ...)
```

- Filtre `gsl_known` : le code-barres (exact) ne doit jamais être contredit par une preview regex; le client montre déjà ces champs à T+0.
- Idempotent à la relivraison : `ON CONFLICT DO NOTHING` + bump de statut **gardé** aux états pré-LLM (`WHERE status IN (...)`) — un retry après crash ne fait jamais régresser un statut terminal.
- Même sans aucun champ trouvé, le transit `ocr_done` part quand même : c'est le signal auquel la bannière de progression mobile se lie.

`_persist` — l'écriture finale

```
attempt_no = conn.execute(text(
    "SELECT COALESCE(MAX(attempt_no), 0) + 1 FROM ingestion.extraction_run "
    "WHERE ingestion_id=:id"), ...).scalar_one()
run_id = conn.execute(text("INSERT INTO ingestion.extraction_run (...) RETURNING id"), {
    "out": outcome.value, "ev": llm.extractor_version, "pv": llm.prompt_version,
    "ocrp": self._ocr.name, "llm": llm.model, "esc": escalation_model,
    "ocr_ref": ocr_artifact_id, "rsv": self._rule_set.version, ...}).scalar_one()
self._write_fields(conn, run_id, reconciled)
conn.execute(text("UPDATE ingestion.ingestion SET status=:st WHERE id=:id"), ...)
self._emit_completed(conn, ingestion_id, str(run_id), outcome.value, corr, trace)
```

- **Append-only** : chaque tentative = un nouveau run avec `attempt_no` croissant; « le courant » = `max(attempt_no)`, jamais un `UPDATE` d'un run.
- Le run enregistre **toute la provenance de jugement** : versions extracteur/prompt/rule-set, providers, modèles (primaire + escalade), référence de l'artefact OCR. On peut réauditer n'importe quel run des années plus tard.
- `_write_fields` est un « dumb writer » : les invariants (`value≠null ⇒ provenance+source`, `null ⇒ rien`) tiennent **par construction** dans `ReconciledField`; les CHECK DB les révérifient.
- `_emit_completed` insère l'événement `extraction.completed` sur `worker_conn` — même transaction que le run : le chaînage d'événements hérite des mêmes garanties outbox.

Variantes : `_persist_skipped_garbage` (run needs_review, versions 'ocr-skipped', statut `ocr_skipped_garbage`, l'événement porte `needs_review` → `traceability` n'enregistrera pas de lot) et `_persist_failed` (run `extraction_failed`, pas d'`ocr_raw_ref`).

12.2 `adapters/sql_field_override_repository.py` — l'override append-only

L'écriture la plus subtile du backend. Une transaction auditée fait :

- (1) replay par Idempotency-Key ? → retourner le champ du run ORIGINAL
- (2) run le plus récent (parent) → absent ? None (404)

- (3) replay par VALEUR ? → même valeur humaine déjà là ? pas d'écriture
- (4) INSERT nouveau run = COPIE du parent (attempt_no+1, corr/trace de CETTE requête)
- (5) COPIE de tous les champs SAUF celui visé (provenance machine préservée)
- (6) INSERT du champ humain (value + provenance humaine) ou effacé (tout null)
- (7) enregistrer la clé d'idempotence DANS la même transaction

Les requêtes qui portent le motif :

```
_INSERT_RUN_COPY = text(
    "INSERT INTO ingestion.extraction_run (ingestion_id, attempt_no, outcome, ...) "
    "SELECT ingestion_id, :att, outcome, extractor_version, prompt_version, ocr_provider, "
    " llm_model, escalation_model, ocr_raw_ref, rule_set_version, :corr, :trace "
    "FROM ingestion.extraction_run WHERE id = :rid RETURNING id::text")

_COPY_FIELDS_EXCEPT = text(
    "INSERT INTO ingestion.extracted_field (...) "
    "SELECT :new_rid, field_name, value, evidence, provenance, ... "
    "FROM ingestion.extracted_field "
    "WHERE extraction_run_id = :rid AND field_name <> :fn")
```

Concept — INSERT ... SELECT. La copie se fait **entièrement côté SQL** : aucune donnée ne remonte en Python pour redescendre. Le nouveau run garde l'outcome et toutes les versions du parent (l'override enregistre *une valeur*, pas un nouveau verdict machine) mais prend le correlation/trace de la requête de correction.

```
_INSERT_HUMAN_VALUE = text(
    "INSERT INTO ingestion.extracted_field (...) VALUES (:rid, :fn, CAST(:val AS jsonb), "
    " CAST(:ev AS jsonb), CAST(:prov AS jsonb), :src, 'present', '[]'::jsonb, 1.0, 'high', 'human')")
```

- Une valeur humaine cite l'**artefact image** comme source (l'opérateur a validé contre la photo), evidence = la note (ou « Validé par l'opérateur »), provenance = {source:'human', actor_id, note}, confiance 1.0/high. Le champ effacé : tout null, 'missing', 0.0/low — les CHECK DB sont satisfaits dans les deux cas.

La double idempotence, dans l'ordre :

- **Par clé (P3, migration 0013)** — vérifiée **avant** la comparaison de valeur :

```
if idempotency_key:
    seen = conn.execute(_LOOKUP_IDEMPOTENCY, {"actor": audit.actor_id,
                                              "key": idempotency_key}).mappings().first()

    if seen is not None:
        recorded = conn.execute(_CURRENT_FIELD, {"rid": seen["run_id"], ...}).mappings().one()
        return OverriddenField(..., replayed=True)
```

Piège évité — le retry hors-ordre. Scénario : l'opérateur corrige A, le réseau traîne, il corrige B, puis le retry de A arrive. Sans clé, «A» serait appliqué comme une **nouvelle** correction et écraserait B. Avec la clé, le retry de A rejoue *le run que la requête A originale avait produit* — jamais un nouvel append. La clé est liée au run **dans la même transaction** ((7), ON CONFLICT DO NOTHING pour les courses), donc clé et résultat committent ou échouent ensemble.

- **Par valeur** — si le dernier run porte déjà exactement cette valeur humaine, on ne réécrit rien (replayed=True) ; la clé est tout de même enregistrée.

Élégance du résultat : le read-model existant (GET /v1/extraction-runs/{id}, « dernier run = max attempt_no ») **fonctionne sans changement** — il renvoie désormais la valeur validée par l'humain, avec tout l'historique machine intact en dessous.

12.3 adapters/sql_confirm_repository.py

```

with self._engine.begin() as conn:
    status = conn.execute(text(
        "SELECT status FROM ingestion.ingestion WHERE id=:id FOR UPDATE"), ...).scalar_one_or_none()
    if status is None: return None # 404
    if status == "confirmed":
        return ConfirmedIngestion(..., replayed=True) # idempotent
    if status not in ("extracted", "needs_review", "ocr_skipped_garbage"):
        raise ConfirmNotAllowed(status) # 409
    set_audit_context(conn, ...) # sinon le trigger AFTER UPDATE refuse
    conn.execute(text("UPDATE ingestion.ingestion SET status='confirmed' WHERE id=:id"), ...)
    return ConfirmedIngestion(..., replayed=False)

```

- FOR UPDATE : deux confirms simultanés se **sérialisent** — le second voit confirmed et rejoue, au lieu d’auditer deux fois. Le garde d’états empêche d’affirmer une revue qui n’a pas eu lieu (ingestion encore en cours, ou échouée).

12.4 adapters/http/router.py — le routeur d’écriture

Trois endpoints, un motif commun. D’abord le **seam de composition paresseux** :

```

_DEFAULT_USE_CASE: SubmitIngestion | None = None
_USE_CASE_LOCK = threading.Lock()

def get_submit_ingestion() -> SubmitIngestion:
    global _DEFAULT_USE_CASE
    if _DEFAULT_USE_CASE is None:
        with _USE_CASE_LOCK:
            if _DEFAULT_USE_CASE is None: # double-checked locking
                ...construire FilesystemRawStore + SqlIngestionRepository...
    return _DEFAULT_USE_CASE

```

Concept — double-checked locking. FastAPI exécute les endpoints **sync** dans un threadpool : plusieurs premières requêtes concurrentes pourraient chacune construire un cas d’usage (et un pool DB), en orphelinant. Le premier if (sans verrou) rend le chemin chaud gratuit; le second (sous verrou) garantit une seule construction. Les tests remplacent tout via `app.dependency_overrides[get_submit_ingestion]`. Note : le routeur construit ses dépendances depuis **son propre contexte + platform**, jamais en important `app/` — la direction des dépendances (vers l’intérieur) est préservée même ici.

POST /v1/ingestions :

```

@router.post("/v1/ingestions", status_code=202, response_model=IngestionAcceptedResponse)
def submit_ingestion(request, response,
    image: UploadFile = File(...),
    barcode_raw: str | None = Form(None),
    client_captured_at: str | None = Form(None),
    idempotency_key: str | None = Header(None, alias="Idempotency-Key"),
    principal: Principal = Depends(require_scope("ingestion:write")),
    use_case: SubmitIngestion = Depends(get_submit_ingestion)):
    if not idempotency_key:
        raise ApiError("VALIDATION_ERROR", "Idempotency-Key header is required")
    media_type = (image.content_type or "").split(";")[0].strip()
    if media_type not in _ACCEPTED_MEDIA:
        raise ApiError("UNSUPPORTED_MEDIA_TYPE", ...)
    data = image.file.read()
    if not data: raise ApiError("VALIDATION_ERROR", "empty image payload")
    if len(data) > _MAX_BYTES: raise ApiError("PAYLOAD_TOO_LARGE", ...) # 10 Mo

```

- Le header est exigé même si la dédup réelle est par hash de contenu : il force la discipline

client (et servira à d'autres endpoints). `content_type.split(";")[0]` retire les paramètres (image/jpeg; charset=...).

```
command = SubmitIngestionCommand(
    image_bytes=data, content_type=media_type,
    actor_id=principal.actor_id,
    correlation_id=request.state.correlation_id,
    trace_id=request.state.trace_id,
    principal=principal.principal,
    barcode_raw=barcode_raw, client_captured_at=client_captured_at)

try:
    result = use_case(command)
except ValueError as exc:
    raise ApiError("VALIDATION_ERROR", str(exc))          # défense en profondeur
except (sqlalchemy.exc.OperationalError, sqlalchemy.exc.InterfaceError, OSError):
    raise ApiError("DEPENDENCY_UNAVAILABLE",
                  "storage unavailable; retry with the same Idempotency-Key")
if result.replayed:
    response.headers["Idempotency-Replayed"] = "true"
return IngestionAcceptedResponse(...)
```

- **Fail closed** : DB ou disque indisponible → 503 `retriable=true` → la file durable du mobile retentera *avec la même clé*. Jamais un 500 flou pour une panne de dépendance.
- Le routeur ne fait **aucun** métier : garde-fous transport, assemblage de la commande (avec le contexte d'audit), traduction des erreurs. C'est la définition d'un adapter.

PATCH `/v1/ingestions/{id}/fields/{field_name}` (SCOPE extraction:review) et POST `/v1/ingestions/{id}/confirm` suivent le même schéma : construire la commande → appeler le cas d'usage → mapper
UnknownField→400, FieldNotEditable→409 FIELD_NOT_EDITABLE, IngestionNotFound→404, ConfirmNotAllowed→409
INGESTION_NOT_CONFIRMABLE, pannes → 503.

12.5 adapters/http/read_router.py — lectures + long-poll

Deux endpoints en lecture seule (GET `/v1/ingestions/{id}`, GET `/v1/extraction-runs/{id}`), scope ingestion:read. Traits notables :

Le long-poll (Tier 4) — supprimer la latence de découverte du polling :

```
_MAX_WAIT_S = 25.0
_PROBE_INTERVAL_S = 0.3

wait_s = clamp_wait(wait)          # [0, 25] – jamais négatif, jamais illimité
if wait_s > 0 and last_status:
    deadline = time.monotonic() + wait_s
    while True:
        current = _probe_status(engine, ingestion_id) # SELECT status (indexé, conn courte)
        if current is None or current != last_status:
            break                                     # changé (ou introuvable → le 404 habituel suivra)
        remaining = deadline - time.monotonic()
        if remaining <= 0: break                       # hold expiré → renvoyer l'état courant
        time.sleep(min(_PROBE_INTERVAL_S, remaining))
```

- Le client envoie `?wait=25&last_status=raw_stored` : le serveur **retient la requête** jusqu'à ce que le statut diffère, en sondant toutes les 300 ms sur des connexions courtes (jamais un slot de pool retenu pendant le hold). Les deux paramètres sont requis pour armer le hold (un wait sans baseline n'a pas de sens).
- Limites assumées en docstring : un thread sync est retenu pendant le hold — acceptable au nombre d'appareils de cette app; LISTEN/NOTIFY est l'upgrade documenté si ça change.

Le view model embarque tout :

```

class IngestionView(BaseModel):
    ingestion_id: str; status: str; ...
    raw_artifacts: list[RawArtifactView]
    extraction_runs: list[RunSummary]           # is_latest dérivé (max attempt_no)
    latest_fields: list[FieldView] | None       # champs du dernier run, EMBARQUÉS
    interim_fields: list[InterimFieldView] | None # préviews, SEULEMENT avant le 1er run
    audit: list[AuditEntry]

```

- latest_fields embarqué = le client qui polle rend l'écran de revue **sans second aller-retour** au moment précis où l'opérateur attend. Peuplé seulement quand un run existe : les polls intermédiaires restent légers.
- interim_fields n'est requêté que si **aucun** run n'existe : une extraction terminée (ou échouée) ne peut jamais montrer une preview périmée comme définitive.
- is_latest est une **projection de lecture dérivée** (attempt_no == max), pas une colonne : rien à muter quand un nouveau run arrive.

13. Traceability, HACCP, Identity — les autres contextes

13.1 traceability/domain/consistency.py — vérité métier pure

Deuxième couche de validation, distincte du gate d'extraction : la **cohérence business** d'un lot candidat.

```
@dataclass(frozen=True)
class BatchCandidate:
    lot_code: str | None
    scientific_name: str | None
    production_method: str | None
    fao_area: str | None
    supplier_name: str | None
    use_by: date | None
    packaging_date: date | None

def check_consistency(candidate, *, conflicting_supplier_for_lot=None) -> tuple[str, ...]:
    issues = []
    if not candidate.lot_code:
        issues.append("missing_lot_code")
    if not candidate.supplier_name:
        issues.append("missing_supplier")
    if candidate.use_by and candidate.packaging_date \
        and candidate.use_by < candidate.packaging_date:
        issues.append("expiry_before_packaging")
    if candidate.production_method and candidate.production_method not in _PRODUCTION_METHODS:
        issues.append("invalid_production_method")
    if conflicting_supplier_for_lot is not None and candidate.supplier_name is not None \
        and conflicting_supplier_for_lot != candidate.supplier_name:
        issues.append("supplier_mismatch")
    return tuple(issues)
```

- Retour = tuple de **codes** d'incohérence (vide = cohérent) — pas de booléen : l'appelant sait *pourquoi*.
- Astuce de pureté : le check «même lot déjà connu sous un autre fournisseur» nécessiterait la DB. La fonction reçoit donc le **fait déjà résolu** (`conflicting_supplier_for_lot`) en paramètre — l'adapter fait le SELECT, le domaine fait le jugement. C'est la manière standard de garder pur un check qui a besoin d'un registre.

13.2 traceability/adapters/registration_consumer.py

Consomme `extraction.completed`. Première ligne décisive :

```
def __call__(self, msg, conn):
    if msg.payload.get("outcome") != "extracted":
        return # needs_review / failed ne produisent JAMAIS un lot
```

- Seules les extractions **validées** entrent dans le registre de traçabilité. Un retour silencieux suffit : le worker marquera l'événement traité (idempotence standard).

```
vals = {r["field_name"]: r["value"] for r in conn.execute(text(
    "SELECT field_name, value FROM ingestion.extracted_field "
    "WHERE extraction_run_id = :r")), {"r": run_id}).mappings().all()}
candidate = BatchCandidate(
    lot_code=vals.get("batch_number"),
    supplier_name=(vals.get("reseller_brand") or vals.get("producer_name")
                  or vals.get("supplier_name")), # v2 → fallback legacy v1
    use_by=_to_date(vals.get("expiry_date")), ...)
```

- Le consumer relit les champs **par la DB** (par ID de run), jamais en important le contexte ingestion — l'intégration inter-contextes = événements + IDs.
- La chaîne de fallback fournisseur encode l'évolution du prompt v1→v2 (supplier_name scindé en reseller_brand/producer_name) tout en restant compatible avec les runs historiques immuables.
- `_to_date` avale `ValueError` **et** `TypeError` (chaîne malformée, valeur non-chaîne) → `None` : une date pourrie ne crashe pas le consumer (elle produira un lot flaggé si la date était requise).

```
issues = check_consistency(candidate, conflicting_supplier_for_lot=conflicting)
if issues: self._flag(conn, ...) # batch status='flagged' + événement batch.flagged
else:      self._register(conn, ...) # chaîne complète + batch.registered
```

`_register` construit la **chaîne de traçabilité** : produit (find-or-create) → fournisseur (find-or-create) → lot pointant `source_ingestion_id` + `source_extraction_run_id` → événement `batch.registered` (payload : `batch_id`, `use_by`). Le find-or-create utilise le même motif atomique qu'en §11.2 :

```
sid = conn.execute(text(
    "INSERT INTO traceability.supplier (name, correlation_id, trace_id) "
    "VALUES (:n, :c, :t) ON CONFLICT (name) DO NOTHING RETURNING id"), ...).scalar_one_or_none()
if sid is None:
    sid = conn.execute(text("SELECT id FROM traceability.supplier WHERE name=:n"), ...).scalar_one()
```

- Insert-si-absent sans course : l'insert tente, le conflit rend `None`, le `SELECT` relit le gagnant. Pour le produit, le `SELECT` de rattrapage utilise `IS NOT DISTINCT FROM` — l'égalité SQL qui traite `NULL = NULL` comme vrai (un produit sans nom scientifique doit matcher sa propre ligne).
- Un lot **flaggé** est quand même écrit (status 'flagged', sans liens produit/fournisseur) : une donnée incohérente n'est ni acceptée ni jetée — elle est **enregistrée comme incohérente** et l'événement `batch.flagged` (avec les codes) déclenchera une alerte.

13.3 haccp/domain/ — règles et machine à états pures

`control.py` — le moteur de plan de contrôle :

```
@dataclass(frozen=True)
class ControlPlan:
    version: str
    max_temp_c: float | None = None
    min_temp_c: float | None = None
    expiry_warning_days: int | None = None

def check_temperature(temp_c, plan) -> Violation | None:
    # > max → Violation(TEMPERATURE, HIGH, {breach: above_max, ...})
    # < min → Violation(TEMPERATURE, HIGH, {breach: below_min, ...})

def check_expiry(use_by, plan, today) -> Violation | None:
    if use_by < today:
        return Violation(EXPIRY, CRITICAL, {state: "expired", ...})
```

```
if plan.expiry_warning_days is not None and use_by <= today + timedelta(...):
    return Violation(EXPIRY, HIGH, {state: "approaching", ...})
```

- Les seuils sont **de la donnée versionnée** (plan de contrôle, propriété de Compliance) — jamais des constantes en dur : ce serait fabriquer de la vérité réglementaire. `today` est un paramètre (testabilité, encore).

`alert.py` — le cycle de vie d’une alerte :

```
class AlertState(StrEnum):
    OPEN = "open"; ACKNOWLEDGED = "acknowledged"; RESOLVED = "resolved"

def acknowledge(current):
    if current is not AlertState.OPEN:
        raise InvalidAlertTransition(current, "acknowledge")
    return AlertState.ACKNOWLEDGED

def resolve(current):
    if current not in (AlertState.OPEN, AlertState.ACKNOWLEDGED):
        raise InvalidAlertTransition(current, "resolve")
    return AlertState.RESOLVED
```

- **Les transitions valides vivent ici et nulle part ailleurs.** Adapter et repository ne décident rien : ils appliquent. Monotone : `open` → `acknowledged` → `resolved` (resolve accepté directement depuis `open`).

13.4 `haccp/application/alert_service.py` + `adapters/sql_alert_repository.py`

Le motif «le domaine décide, le repo applique sous verrou» :

```
class AlertLifecycleService:
    def acknowledge(self, alert_id, audit):
        return self._repository.run_transition(
            alert_id, action="haccp.alert_acknowledged", decide=acknowledge, audit=audit)
```

- Le service passe la **fonction de décision** (`decide=acknowledge`) au port. Côté SQL :

```
def run_transition(self, alert_id, *, action, decide, audit):
    with self._engine.begin() as conn:
        row = conn.execute(text(
            "SELECT state FROM haccp.alert WHERE id=:id FOR UPDATE"), ...).first()
        if row is None: raise AlertNotFound(alert_id)
        new_state = decide(AlertState(row.state)) # peut lever → rollback total
        set_audit_context(conn, ..., action=action, ...)
        conn.execute(text("UPDATE haccp.alert SET state=:s, "
            "updated_at=clock_timestamp() WHERE id=:id"), ...)

    return new_state
```

- `FOR UPDATE` sérialise deux transitions concurrentes; une `InvalidAlertTransition` levée par le domaine traverse → `rollback` → rien ne change, pas même l’audit. `clock_timestamp()` (pas `now()`) : l’heure réelle, pas l’heure du début de transaction.

13.5 `haccp/adapters/alerting_consumer.py` et `temperature_recorder.py`

`AlertingConsumer` écoute **deux** types (`event_types = ("batch.registered", "batch.flagged")`) :

- `batch.flagged` → alerte inconsistency (high) portant les codes d’incohérence dans `detail`.
- `batch.registered` → charge le plan de contrôle **actif** (`WHERE active ORDER BY created_at DESC LIMIT 1`), applique `check_expiry` sur le `use_by` du payload avec `self._today()` (injecté), lève une

alerte expiry si violation. Pas de plan actif ou pas de seuil → return silencieux (pas de vérité fabriquée).

TemperatureRecorder (pas encore exposé en HTTP) : dans **une** transaction auditée — insert du relevé immuable temperature_log, lecture du plan actif, check_temperature, et si violation, insert de l'alerte. Relevé et alerte committent atomiquement : impossible d'avoir une alerte sans son relevé, ou l'inverse.

13.6 Le contexte identity

domain/user.py — rôle et scopes

```
ADMIN_ROLE = "admin"
ADMIN_SCOPES = frozenset({
    "ingestion:write", "ingestion:read", "extraction:review",
    "haccp:read", "alert:ack", "alert:resolve", "traceability:read"})

def scopes_for_role(role: str) -> frozenset[str]:
    return _ROLE_SCOPES.get(role, frozenset())    # rôle inconnu → RIEN (fail closed)
```

- L'auth est **additive** : elle n'invente pas de nouveaux scopes, elle authentifie qui porte ceux que les require_scope() existants attendaient déjà.

domain/password.py — hachage stdlib

```
_DEFAULT_ITERATIONS = 600_000    # plancher OWASP pour PBKDF2-HMAC-SHA256

def hash_password(password, *, iterations=600_000) -> str:
    salt = secrets.token_bytes(16)
    digest = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, iterations)
    return f"pbkdf2_sha256${iterations}${_b64e(salt)}${_b64e(digest)}"

def verify_password(password, encoded) -> bool:
    try:
        algorithm, iterations_s, salt_s, hash_s = encoded.split("$")
        ...
    except (ValueError, AttributeError):
        return False
    candidate = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, iterations)
    return hmac.compare_digest(candidate, expected)
```

Concept — stockage de mot de passe. (1) secrets.token_bytes pour le sel (jamais random). (2) PBKDF2 à 600 000 itérations : rendre le brute-force coûteux. (3) Le format encodé embarque algo + itérations + sel → vérifiable sans config externe, migrable (on peut monter les itérations plus tard). (4) hmac.compare_digest : comparaison **en temps constant** — == s'arrête au premier octet différent et fuit de l'information par le chrono. (5) verify_password ne lève jamais sur entrée pourrie : False, point.

application/login.py — anti-énumération d'utilisateurs

```
_DUMMY_HASH = hash_password("invalid-credentials-placeholder")

class Login:
    def __call__(self, username, password) -> AuthenticatedUser:
        user = self._users.find_active_by_username(username)
        stored_hash = user.password_hash if user else _DUMMY_HASH
        password_ok = verify_password(password, stored_hash)
        if not user or not password_ok:
```

```

        raise InvalidCredentials()
    return AuthenticatedUser(actor_id=user.id, username=user.username,
                             role=user.role, scopes=scopes_for_role(user.role))

```

Piège évité — l’oracle temporel. Si « utilisateur inconnu » retournait immédiatement et « mauvais mot de passe » après un PBKDF2 de 600 000 itérations (~100 ms), un attaquant pourrait **chronométrer** pour découvrir quels identifiants existent. Ici, le chemin « inconnu » vérifie quand même un hash factice : coût identique. Et une seule exception `InvalidCredentials` sans détail : la réponse HTTP ne distingue jamais les deux cas.

adapters/http/router.py — POST /v1/auth/login

```

@router.post("/v1/auth/login", response_model=LoginResponse)
def login(body: LoginRequest, login_uc: Login = Depends(get_login)):
    try:
        user = login_uc(body.username, body.password)
    except InvalidCredentials:
        raise ApiError("UNAUTHENTICATED", "invalid username or password")
    ttl = jwt_codec.default_ttl_seconds()
    token = jwt_codec.encode({
        "sub": user.username, "actor_id": user.actor_id,
        "principal": user.username, "scopes": sorted(user.scopes),
        "role": user.role}, ttl_seconds=ttl)
    return LoginResponse(access_token=token, token_type="bearer", expires_in=ttl)

```

- Non authentifié par design (c’est l’échange de credentials). Les claims émis sont exactement ceux que `resolve_principal` (§7.7) et le contexte d’audit comprennent — zéro changement dans les routeurs aval. `sorted(user.scopes)` : un frozenset n’a pas d’ordre, le tri rend le token déterministe.

adapters/cli.py — provisionnement

Pas d’endpoint d’inscription **par design**. L’admin est créé par CLI (`python -m labelscan.contexts.identity.adapted`) : credentials lus de l’environnement, `upsert idempotent ON CONFLICT (username) DO UPDATE` — seul le hash est stocké, le plaintext n’est ni persisté ni affiché.

14. La composition root : app/

La seule couche qui a le droit de tout connaître. C'est ici — et seulement ici — que les adaptateurs concrets rencontrent les ports.

14.1 app/http_app.py

```
def create_app() -> FastAPI:
    configure_logging()
    app = FastAPI(title="LabelScan API", version="v1")
    app.add_middleware(CorrelationMiddleware)
    install_error_handlers(app)
    app.include_router(auth_router)           # POST /v1/auth/login
    app.include_router(ingestion_router)      # écritures ingestion
    app.include_router(ingestion_read_router) # lectures ingestion/runs
    app.include_router(batches_read_router)   # GET /v1/batches/{id}
    app.include_router(alerts_read_router)    # GET /v1/alerts
    app.include_router(alerts_lifecycle_router) # ack/resolve
    app.include_router(ops_router)           # health/version
    return app
```

- **Factory pattern** (uvicorn ...:create_app --factory) : chaque appel construit une app propre — indispensable pour les tests (une app par fixture, pas d'état global partagé).

14.2 Les wirings

- ingestion_factory.py : construit SubmitIngestion(FilesystemRawStore(dir), SqlIngestionRepository(engine)) — l'entrée « interne » du cas d'usage.
- ocr_wiring.py : sélection du provider OCR par env. **Aucun provider par défaut** : non configuré → RuntimeError explicite plutôt qu'un appel réseau silencieux ; les tests injectent un fake et ne passent jamais ici. Import de l'adapter **dans la branche** (if provider == "google":) — le module se charge sans httpx.
- extraction_wiring.py : lit les seuils/politiques d'env avec des parseurs stricts (_env_prob valide [0,1], _env_int ≥ 0, cohérence medium_at ≤ high_at — toute valeur invalide **échoue au démarrage**), construit l'ExtractionConsumer complet : engine, raw store, OCR injecté, ClaudeLlmExtractor() primaire, ClaudeLlmExtractor(model=ESCALATION_MODEL) d'escalade (toujours construit, appelé seulement si flag ON), rule-set placeholder _PLACEHOLDER_RULESET (version "placeholder-pending-B2" — de la config assumée, pas de la vérité réglementaire fabriquée), et l'enregistre sur le worker.
- domain_wiring.py : enregistre RegistrationConsumer (sur extraction.completed) et AlertingConsumer (sur ses deux types).

Concept — pourquoi tout ce cérémonial. Le relais (platform) n'importe aucun contexte ; aucun contexte n'importe le relais ; **seul** app/ fait se rencontrer producteurs, relais et consommateurs. Résultat : chaque pièce se teste isolément, et la topologie complète du système se lit dans un seul répertoire.

14.3 app/worker_runtime.py — l’entrypoint worker + heartbeat

```
def build_outbox_worker() -> OutboxWorker:
    configure_logging()
    worker = OutboxWorker(make_engine())
    raw_store = FilesystemRawStore(os.environ["LABELSCAN_RAW_STORE_DIR"]) # (vérifié avant)
    register_extraction_consumer(worker, ocr_provider=build_ocr_provider(), raw_store=raw_store)
    register_domain_consumers(worker)
    return worker
```

Le **heartbeat de vivacité** — un healthcheck sans serveur HTTP :

```
_HEARTBEAT_PATH = os.environ.get("LABELSCAN_WORKER_HEARTBEAT", "/tmp/labelscan_worker.heartbeat")
_HEARTBEAT_MAX_AGE_S = float(os.environ.get("LABELSCAN_WORKER_HEARTBEAT_MAX_AGE", "180"))

def _beat(path=_HEARTBEAT_PATH):
    try:
        with open(path, "w") as fh: fh.write(str(time.time()))
    except OSError:
        pass # best-effort : ne JAMAIS crasher le worker pour ça

def worker_is_healthy(path=..., max_age_s=...) -> bool:
    try:
        return (time.time() - os.path.getmtime(path)) < max_age_s
    except OSError:
        return False # fichier absent → jamais démarré / purgé → unhealthy

def poll_forever(worker, *, interval_seconds=1.0, batch=100):
    _beat() # vivant dès le boot, avant le premier claim
    while True:
        _beat() # un battement par itération
        if worker.run_once(max_messages=batch) == 0:
            time.sleep(interval_seconds)

if __name__ == "__main__":
    if "--health" in sys.argv:
        raise SystemExit(0 if worker_is_healthy() else 1)
    poll_forever(build_outbox_worker())
```

- La boucle : drainer un batch ; s’il était vide, dormir 1 s. Le fichier heartbeat est réécrit à chaque tour ; le healthcheck Docker exécute `--health` qui teste sa fraîcheur.
- Le seuil de 180 s est **volontairement supérieur** au timeout LLM (120 s) : une extraction lente ne doit pas faire clignoter le healthcheck. Dimensionner les seuils de santé au-dessus de la plus longue opération légitime — règle générale.

14.4 app/ops_router.py — health et version

```
@router.get("/v1/health/live")
def live(): return {"status": "alive"} # le process répond, rien de plus

@router.get("/v1/health/ready")
def ready(probe: ReadinessProbe = Depends(get_readiness_probe)):
    checks = probe.check() # database: SELECT 1 ; object_store: isdir + W_OK
    unavailable = sorted(n for n, s in checks.items() if s != "ok")
    if unavailable:
        raise ApiError("DEPENDENCY_UNAVAILABLE", f"not ready: {' '.join(unavailable)}")
    return {"status": "ready", "checks": checks}
```

```

@router.get("/v1/version")
def version():
    return {"version": _build_version(),                # env → pkg metadata → sentinel
           "rule_set_version": _PLACEHOLDER_RULESET.version} # rapporté honnêtement

```

Concept — liveness vs readiness. *Live* = « le process tourne » (aucune dépendance vérifiée — sinon un pépin DB ferait redémarrer des pods sains en boucle). *Ready* = « je peux servir » (DB joignable, store accessible) — l’orchestrateur retire du load-balancer sans tuer. La probe construit un engine **jetable** et le dispose() en finally : un probe ne doit pas laisser fuiter un pool. - _build_version() : env LABELSCAN_VERSION (injectée au déploiement) → sinon version du paquet installé → sinon "0.0.0+unknown" — jamais une valeur fabriquée. Même honnêteté pour le rule-set placeholder.

15. Le modèle de données et les migrations

13 migrations Alembic, essentiellement du SQL brut. Chaque docstring est un mini-document d'architecture — les lire fait partie de la maîtrise du projet.

15.1 0001_schemas_and_roles — schémas et moindre privilège

- Crée le rôle `labelscan_app` en **NOLOGIN** (les tests/l'app font `SET ROLE`; en prod, un rôle `LOGIN` en est membre). Moindre privilège dès la fondation.
- Crée les schémas (`platform`, `audit`, `ingestion`, `traceability`, `haccp`, `identity`, `compliance`) + `GRANT USAGE`.

15.2 0002_audit_immutability_foundations — le socle d'intégrité

Installé **avant tout code écrivain** («so SLO-3 cannot be violated by code that doesn't exist yet»). Quatre pièces :

1. `platform.deny_mutation()` :

```
CREATE OR REPLACE FUNCTION platform.deny_mutation() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    RAISE EXCEPTION
        'append-only violation: % on %.% is forbidden (immutable historical record)',
        TG_OP, TG_TABLE_SCHEMA, TG_TABLE_NAME
    USING ERRCODE = 'restrict_violation';
END $$;
```

- Attaché en trigger `BEFORE UPDATE OR DELETE OR TRUNCATE` sur chaque table historique. **Les triggers s'appliquent à tous les rôles, y compris le propriétaire et le superuser** : l'immutabilité ne dépend pas des `GRANT` (qui sont quand même retirés — défense en profondeur).

2. `platform.audit_on_insert()` — le trigger d'audit incontournable :

```
CREATE OR REPLACE FUNCTION platform.audit_on_insert() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = audit, pg_catalog AS $$
DECLARE
    v_actor text := nullif(current_setting('labelscan.actor_id', true), '');
    v_action text := nullif(current_setting('labelscan.action', true), '');
    v_corr text := nullif(current_setting('labelscan.correlation_id', true), '');
    v_trace text := nullif(current_setting('labelscan.trace_id', true), '');
    v_root oid := COALESCE(pg_partition_root(TG_RELID), TG_RELID);
BEGIN
    IF v_actor IS NULL OR v_action IS NULL OR v_corr IS NULL OR v_trace IS NULL THEN
        RAISE EXCEPTION 'audit context missing: SET LOCAL labelscan.{...} is required ...';
    END IF;
```

```

...
INSERT INTO audit.audit_log (occurred_at, actor_id, action, subject_schema,
                             subject_table, subject_id, correlation_id, trace_id)
VALUES (clock_timestamp(), v_actor::uuid, v_action, v_schema, v_table,
        NEW.id, v_corr, v_trace);
RETURN NULL;  -- AFTER trigger : valeur de retour ignorée
END $$;

```

Ligne par ligne :

- `current_setting('...', true)` : le second argument `true` = « rendre NULL si absent » (au lieu de lever); `nullif(x, '')` traite aussi la chaîne vide comme absente. Les quatre doivent être présents, sinon RAISE → **l'insert métier échoue**.
- SECURITY DEFINER : la fonction s'exécute avec les droits de son *propriétaire*, pas de l'appelant — c'est ce qui permet au rôle applicatif d'écrire dans `audit.audit_log` **sans** avoir de privilège INSERT direct dessus (il ne peut donc pas forger d'audit à la main).
- `pg_partition_root(TG_RELID)` : le trigger se déclenche sur la *partition physique* (`raw_artifact_2026_07`); on enregistre la table *logique* parente — le schéma de partitionnement ne fuit pas dans l'audit.

3. Tables append-only partitionnées — `audit.audit_log`, `ingestion.raw_artifact`, `hacpp.temperature_log` : PARTITION BY RANGE (`occurred_at`), partitions mensuelles + une DEFAULT, PK composite (`id`, `occurred_at`) (la clé de partition doit faire partie de la PK), grants sans UPDATE/DELETE.

Concept — partitionnement par plage. Les tables à croissance infinie (logs, artefacts) sont découpées par mois : les index restent petits, et l'archivage/purge d'un mois = détacher une partition (métadonnée) au lieu d'un DELETE géant. Sur PG 16, les triggers de ligne posés sur le parent se propagent automatiquement aux partitions actuelles **et futures**.

4. Tables de travail mutables — `idempotency_key`, `outbox`, `processed_event` : délibérément **pas** append-only (elles transitionnent des états).

15.3 0003_harden_security_definer — durcissement

- La fonction d'audit est ré-ownée par un rôle dédié `labelscan_auditor` dont les **seuls** privilèges sont USAGE sur `audit` + INSERT sur `audit_log` : même un abus du chemin `definer` ne peut *que* écrire une ligne d'audit.
- `SET search_path = ''` sur les fonctions `definer` : ferme l'attaque classique par capture de `search_path` (un objet malveillant homonyme dans un schéma prioritaire). `pg_catalog` reste implicitement résolu ; tout le reste est qualifié.
- Validation stricte du contexte : `actor_id` doit être un uuid bien formé, les autres bornés en longueur — pas d'acteur poubelle enregistrable.

15.4 Les migrations suivantes, en bref

#	Contenu	Leçon
0004	<code>ingestion.ingestion</code> (mutable, auditée à l'insert); <code>raw_artifact.ingestion_id</code> délibérément sans FK	deux agrégats distincts se référencent par ID; l'immutabilité de l'un ne dépend pas du cycle de vie de l'autre

#	Contenu	Leçon
0005	extraction_run + extracted_field (append-only); CHECK ck_value_requires_provenance / ck_evidence_iff_value; trigger d'audit sur UPDATE de ingestion.status; kind llm_output	les invariants du domaine sont redéclarés en DB — dernier rempart
0006	traceability.product/supplier/batch + haccp.control_plan/alert	batch porte la chaîne complète source_ingestion → source_run
0007	identity.app_user (mutable, PAS audit-triggée : table de credentials, pas un record HACCP)	ne pas sur-appliquer un mécanisme — choisir où l'audit a du sens
0008	CHECK source élargi à 'gs1'	élargir un CHECK = DDL
0009	+ champ gtin outbox : attempts, next_retry_at, last_error, status (DLQ)	sûr sur table append-only donner une <i>histoire</i> et un <i>état terminal</i> à l'échec
0010	raw_artifact.model + extraction_run.escalation_model (nullable, additif)	migration « additive-only » = déployable sans backfill ni changement de comportement flag-off
0011	CHECK field_name élargi en SUPERSET (v2 + legacy v1)	on n'étrécit jamais un CHECK sur de l'append-only : les lignes historiques doivent rester valides
0012	interim_field (append-only, UNIQUE (ingestion_id, field_name), non auditée)	une préview supersédée n'est pas un business record
0013	ingestion.request_idempotency (endpoint, actor, key → run)	la dédup des écritures de revue, côté serveur

Tip dev — migrations sur tables immuables. Le motif qui revient trois fois (0005, 0008, 0011) : pour faire évoluer une table append-only, on **élargit** les contraintes (nouveau kind, nouvelle source, nouveaux noms) sans jamais toucher les lignes. Les données historiques sont un contrat.

16. Les tests backend

~6 000 lignes dans `server/tests/`, exécutés contre un **vrai PostgreSQL 16 migré** (pas de SQLite «à peu près pareil» : les triggers, partitions et CHECK font partie du système sous test).

16.1 `conftest.py` — les fixtures

```
ACTOR_ID = "11111111-1111-1111-1111-111111111111"

os.environ.setdefault("LABELSCAN_JWT_SECRET", "test-jwt-secret-not-for-prod-0123456789abcdef")

def bearer(scopes, *, actor_id=ACTOR_ID, principal="device-01") -> dict[str, str]:
    token = jwt_codec.encode({"sub": principal, "actor_id": actor_id,
                              "principal": principal, "scopes": scope_list, "role": "admin"})
    return {"Authorization": f"Bearer {token}"}
```

- Les tests HTTP s'authentifient avec de **vrais JWT** (le seam par headers est OFF par défaut, comme en prod) : `client.post(..., headers=bearer("ingestion:write"))`. On teste la vraie porte, pas un bypass.

```
@pytest.fixture(scope="session")
def engine():
    url = os.environ.get("DATABASE_URL")
    if not url: pytest.skip("DATABASE_URL not set (needs a running PostgreSQL 16)")
    eng = create_engine(url, future=True)
    yield eng
    eng.dispose()
```

- `scope="session"` : un seul pool pour toute la session de test. `pytest.skip` si pas de DB : la suite se dégrade proprement au lieu d'exploser.

```
@contextmanager
def capture_logger(caplog, name):
    lg = logging.getLogger(name)
    lg.addHandler(caplog.handler); lg.setLevel(logging.INFO)
    try: yield
    finally: lg.removeHandler(caplog.handler); ...
```

- Subtilité réelle : `configure_logging()` met `propagate=False` sur `labelscan` → les records n'atteignent plus le handler racine que `caplog` installe. La fixture attache le handler de `caplog` **directement** sur le logger émetteur — les assertions de logs deviennent indépendantes de l'ordre des tests.

16.2 `_fakes.py` — les doublures de providers

```
class FakeOcr:
    def __init__(self, full_text=OCR_TEXT, confidence=0.95):
```

```

...; self.calls = 0
def run(self, image_bytes):
    self.calls += 1
    return OcrResult(...)

```

- Satisfont les ports `OcrProvider/LlmExtractor` **par structure** (aucun héritage — c'est le Protocol qui joue). Le compteur `self.calls` permet d'affirmer la dédup : «re-livrer l'événement ne rappelle PAS le provider» se teste par `assert fake.calls == 1`.
- `OCR_TEXT` est un texte d'étiquette fixe; `good_fields()` fabrique des champs dont chaque evidence **est réellement une sous-chaine** de ce texte — les tests du gate jouent donc le vrai mécanisme d'ancrage, pas un mock du gate.

16.3 Les tests comme preuves

Les noms de fichiers annoncent des **propriétés**, pas des fonctions : `test_audit_same_transaction` (chaque insert audité produit exactement une ligne d'audit dans la même transaction; sans contexte → rejet), `test_immutability` (UPDATE/DELETE sur l'historique → erreur), `test_ingestion_durability` (le 202 implique la persistance), `test_outbox_dlq` / `test_concurrent_outbox` (poison pill parqué, workers concurrents sûrs), `test_extraction_gate` (fabrication coercée à null), `test_option_y_outcome`, `test_field_override`, `test_confirm_ingestion`, `test_long_poll`, `test_auth_login...`

Exemple (extrait réel) — l'audit dans la même transaction :

```

def test_insert_creates_matching_audit_row_in_same_tx(conn):
    with audited_transaction(conn, actor_id=ACTOR_ID, action="ingest.raw_stored",
                             correlation_id="corr-A", trace_id="trace-A"):
        rid = insert_raw_artifact(conn)
        rows = conn.execute(text(
            "SELECT actor_id::text, action, ... FROM audit.audit_log "
            "WHERE subject_id = :id"), {"id": rid}).all()
        assert len(rows) == 1      # visible AVANT commit → même transaction, prouvé

```

- L'assertion tourne **à l'intérieur** de la transaction : si l'audit était écrit ailleurs (autre connexion, tâche async), la ligne serait invisible ici. Le test prouve l'atomicité, pas seulement l'existence.

Tip dev — tester les propriétés, pas les implémentations. « Un poison pill ne bloque pas ses voisins », « un retry ne rappelle pas le provider », « un rollback efface métier + audit ensemble » : ces phrases sont directement des tests. Elles survivent aux refactors internes ; les tests qui singent l'implémentation, non.

Partie III — L'application mobile (Expo / React Native / TypeScript)

17. Fondations : entrée, config, thème, navigation, auth

17.1 index.ts et App.tsx

```
// index.ts
import { registerRootComponent } from 'expo';
import App from './App';
registerRootComponent(App);
```

- `registerRootComponent = AppRegistry.registerComponent('main', () => App)` + la préparation d'environnement Expo Go / build natif.

```
export default function App() {
  const [fontsLoaded, fontError] = useFonts({ Inter_400Regular, ..., Inter_700Bold });

  // P3 : rejouer les écritures de revue en attente à chaque retour au premier plan
  useEffect(() => registerOutboxDrainOnForeground(), []);

  // workflow v1 : restaurer la file de scans + lier ses polls à l'état de l'app
  useEffect(() => {
    const unsubscribe = registerScanQueueLifecycle();
    void initScanQueue();
    return unsubscribe;
  }, []);

  if (!fontsLoaded && !fontError) return null; // pas de flash de texte non stylé

  return (
    <GestureHandlerRootView style={styles.root}>
      <SafeAreaProvider>
        <AuthProvider>
          <RootNavigator />
        </AuthProvider>
      </SafeAreaProvider>
    </GestureHandlerRootView>
  );
}
```

Ligne par ligne :

- `import 'react-native-get-random-values'` (tout en haut du fichier) : polyfill de `crypto.getRandomValues` **avant** tout import de `uuid` — sans lui, `uuid.v4()` plante sur RN. Il est ré-importé dans chaque module qui utilise `uuid` (l'ordre d'évaluation des modules n'est pas garanti).
- `useEffect(() => registerOutboxDrainOnForeground(), [])` : la fonction retourne son `unsubscribe`, que `useEffect` utilise comme `cleanup` — l'idiome « abonnement qui se désabonne ».
- `void initScanQueue()` : le `void` marque explicitement une promesse *fire-and-forget* (satisfait le linter `no-floating-promises` et documente l'intention).

- `if (!fontsLoaded && !fontError) return null` : on attend les polices **sauf** si leur chargement a échoué (auquel cas on rend quand même — Inter retombera sur la police système). Ne jamais bloquer l'app sur un échec cosmétique.
- L'ordre d'emboîtement : `GestureHandler` (racine obligatoire pour les gestes) → `SafeArea` (mesure les encoches une fois) → `Auth` (état global) → `Navigator`.

17.2 `src/config.ts` — la leçon de sécurité `EXPO_PUBLIC`

```
export const API_BASE_URL: string = (process.env.EXPO_PUBLIC_API_BASE_URL ?? '').trim();
```

Piège — `EXPO_PUBLIC_*` n'est PAS un secret store. Ces variables sont **inlinées dans le bundle JS au build** : n'importe qui peut les extraire d'un APK/IPA. C'est exactement pourquoi l'ancienne clé OCR côté device a été supprimée (audit §7.3) : la clé Google Vision vit désormais **côté serveur uniquement** (`LABELSCAN_GOOGLE_VISION_API_KEY`, sans préfixe `EXPO_PUBLIC`). Dans une app mobile : URL publiques oui, credentials jamais.

- URL vide quand non configurée : le client API lèvera `CONFIG_ERROR` explicite plutôt que de deviner un host.
- `BACKEND_FIRST` : parseur booléen où seuls `"false"/"0"` désactivent — tout le reste (y compris absent) = activé. L'opt-out est d'ailleurs non supporté : le chemin OCR on-device a été retiré.

17.3 Le thème (`src/theme/`)

- `colors.ts` : direction « SaaS sobre » — base neutre + **un seul** accent bleu (`#2563EB`); la couleur est réservée aux **états** (success/warning/error/info). Astuce de refactor : les **noms** de tokens Material-You ont été conservés (`primaryContainer`, `onSurfaceVariant`...) en ne changeant que les **valeurs** — zéro import à toucher dans l'app.
- `spacing.ts` : grille de 4 px (`xs:4 ... 3xl:64`) + tokens de taille nommés (`captureButtonOuter: 72`) — pas de nombres magiques dans les styles.
- `typography.ts` : échelle Inter complète (`headline/title/body/label` × tailles), tracking légèrement négatif sur les gros titres.
- `elevation.ts` : ombres par niveau. Tout composant importe `{ colors, spacing, radius, typography, elevation }` depuis `../theme` (`barrel index.ts`).

17.4 `navigation/RootNavigator.tsx`

```
export type ReviewParams = { pendingScanId: string };

export type RootStackParamList = {
  ArticleList: undefined;
  ArticleDetail: { articleId: string };
  Camera: undefined;
  Review: ReviewParams;
};
```

Concept — param lists typées. Ce type unique donne l'autocomplétion et la vérification de `navigation.navigate('Review', { pendingScanId })` partout. Décision d'architecture dans `ReviewParams` : on ne passe **que l'id local du scan** — jamais la photo, le code-barres ou le résultat par paramètres de navigation. La *scan queue* est la source de vérité ; l'écran lit en live (`useScan(id)`) et reflète l'état courant même s'il a changé pendant qu'il n'était pas monté.

```
const ArticleDetailScreen = React.lazy(() =>
  import('../screens/ArticleDetailScreen').then((m) => ({ default: m.ArticleDetailScreen })));

function ArticleDetailLazy() {
  return <Suspense fallback=<ActivityIndicator .../>><ArticleDetailScreen /></Suspense>;
```



```
}
```

- **React.lazy + Suspense** : l'écran détail sort du chemin de démarrage à froid. Honnêtement du commentaire : sous Metro c'est un **différé d'évaluation de module**, pas un vrai code-splitting du bundle — le pattern reste bon.

```
export function RootNavigator() {
  const { status } = useAuth();
  if (status === 'loading') return <View style={styles.splash}>...spinner...</View>;
  if (status === 'signedOut') return <LoginScreen />;
  return (
    <NavigationContainer>
      <AppNavigator />
    </NavigationContainer>
  );
}
```

- **Porte d'auth à trois états** : loading (restauration du token au démarrage — splash), signedOut (l'app entière est derrière le login), signedIn (le stack). Le stack : ArticleList (home) → ArticleDetail / Camera / Review (presentation: 'modal'), headers masqués (chaque écran dessine le sien).

17.5 context/AuthContext.tsx

```
export type AuthStatus = 'loading' | 'signedOut' | 'signedIn';

export function AuthProvider({ children }) {
  const [status, setStatus] = useState<AuthStatus>('loading');
  const [user, setUser] = useState<string | null>(null);

  useEffect(() => {
    let mounted = true;
    Promise.all([getToken(), getUsername()]).then(([token, username]) => {
      if (!mounted) return;
      setStatus(token ? 'signedIn' : 'signedOut');
      setUser(token ? username : null);
    });
    const unsubscribe = onUnauthenticated(() => {
      if (!mounted) return;
      setStatus('signedOut'); setUser(null);
    });
    return () => { mounted = false; unsubscribe(); };
  }, []);
```

- Restauration de session au montage (token + username depuis le Keychain), avec le **drapeau mounted** : si le composant se démonte avant la résolution de la promesse, on ne setState pas (warning « update on unmounted component » évité). C'est l'idiome standard pré-AbortController.
- onUnauthenticated(...) : l'abonnement à l'événement « session invalide » émis par la couche API (§18.3) — un 401 en cours d'usage ramène **toute l'app** au login, sans que la couche API importe React.

```
const signIn = useCallback(async (username, password) => {
  await apiLogin(username, password); // lève en cas d'échec
  setUser(username); setStatus('signedIn');
}, []);
```

- useCallback fige l'identité de la fonction (les consommateurs du contexte ne re-rendent pas pour rien). signIn **lève** en cas d'échec : c'est l'écran de login qui traduit l'erreur en message.

```
export function useAuth(): AuthValue {
```

```
const ctx = useContext(AuthContext);
if (!ctx) throw new Error('useAuth must be used within an AuthProvider');
return ctx;
}
```

- Le garde runtime classique : consommer le contexte hors provider = bug de structure → crash immédiat et clair plutôt que undefined mystérieux.

17.6 services/authStorage.ts — le token en lieu sûr

```
const TOKEN_KEY = 'labelscan.access_token';
let cachedToken: string | null | undefined; // undefined = pas encore lu ; null = pas de token

export async function getToken(): Promise<string | null> {
  if (cachedToken !== undefined) return cachedToken;
  try {
    cachedToken = (await SecureStore.getItemAsync(TOKEN_KEY)) ?? null;
  } catch {
    cachedToken = null; // keystore indisponible → traiter comme déconnecté
  }
  return cachedToken;
}
```

- **expo-secure-store** = Keychain (iOS) / Keystore (Android) : le JWT ne touche **jamais** AsyncStorage (qui est du plaintext lisible sur un device rooté).
- Le cache mémoire à trois états (undefined/null/valeur) évite une lecture Keychain à chaque requête HTTP. set/clear maintiennent le cache synchrone avec le store.
- Le username partage le même store — non pas parce qu'il est secret, mais parce qu'il partage le **cycle de vie** du token (posé au login, purgé au logout) : un seul mécanisme à raisonner.

```
type Listener = () => void;
const listeners = new Set<Listener>();
export function onUnauthenticated(listener: Listener): () => void {
  listeners.add(listener);
  return () => { listeners.delete(listener); };
}
export function emitUnauthenticated(): void {
  for (const listener of listeners) listener();
}
```

Concept — micro event-emitter. Dix lignes, zéro dépendance, et surtout **zéro import React** : la couche API (bas niveau) peut signaler « session morte » et l'UI (haut niveau) s'y abonner, sans cycle de dépendance. Retourner la fonction de désabonnement depuis l'abonnement est la convention à imiter.

17.7 services/auth.ts et l'écran de login

```
export async function login(username: string, password: string): Promise<void> {
  const res = await apiRequest<LoginResponse>('/v1/auth/login', {
    method: 'POST', body: { username, password },
    skipAuth: true, // ne jamais porter (ni réagir à) un token périmé
  });
  if (!res?.access_token) throw new Error('Login response did not include an access token');
  await setToken(res.access_token);
  await setUsername(username); // pour estampiller les articles ; jamais décodé du JWT
}
```

- `skipAuth: true` a **deux** effets (§18.3) : pas de header `Authorization` sur l'appel, et un 401 « mauvais identifiants » ne déclenche pas la déconnexion globale (sinon : boucle de sign-out pendant le login).

LoginScreen.tsx — la partie instructive est le mapping d'erreurs :

```
function messageForError(err: unknown): string {
  if (err instanceof ApiError) {
    if (err.code === 'UNAUTHENTICATED' || err.status === 401)
      return 'Identifiant ou mot de passe incorrect.';
    if (err.code === 'CONFIG_ERROR')
      return 'L\'application n\'est pas configurée pour joindre le serveur...';
    if (err.code === 'NETWORK_ERROR' || err.code === 'TIMEOUT')
      return 'Serveur injoignable. Vérifiez votre connexion et réessayez.';
  }
  return 'Échec de la connexion. Réessayez.';
}
```

- On branche sur les **codes stables** (le contrat §7.5), jamais sur le texte serveur; trois messages utilisateur distincts : `credentials` / `config` / `réseau`. Le reste de l'écran : `KeyboardAvoidingView` (comportement `padding` sur iOS uniquement), `canSubmit` dérivé, `setSubmitting(false)` seulement dans le chemin d'erreur (en cas de succès, le navigateur remplace l'écran — pas besoin de re-render).

18. Le client API : services/api.ts

18.1 ApiError — l'erreur typée côté client

```
export class ApiError extends Error {
  readonly code: string;           // error_code stable (problem+json) ou code client
  readonly status: number;         // statut HTTP, 0 = réseau/timeout/config
  readonly correlationId?: string;
  readonly retrieable: boolean;
}
```

- Miroir client du catalogue serveur, plus trois codes purement client : CONFIG_ERROR, TIMEOUT, NETWORK_ERROR, FILE_NOT_FOUND (tous status: 0). Toute la politique de retry de l'app (outbox, polling) branche sur code/retrieable.

```
async function parseError(response, fallbackCorrelationId): Promise<ApiError> {
  let code = `HTTP_${response.status}`;
  let message = `request failed with status ${response.status}`;
  let retrieable = response.status >= 500 || response.status === 429;
  try {
    const body = await response.json();
    if (typeof body.error_code === 'string') code = body.error_code;
    if (typeof body.detail === 'string') message = body.detail;
    ...
  } catch { /* corps non-JSON → défauts sûrs basés sur le statut */ }
  return new ApiError({ code, status: response.status, message, correlationId, retrieable });
}
```

- Lecture **défensive** du problem+json : chaque champ est vérifié par typeof avant usage ; un corps non-JSON (proxy, page d'erreur HTML) retombe sur des défauts calculés du statut. On ne fait jamais confiance à la forme d'un corps d'erreur.

18.2 send<T> — le cœur commun

```
async function send<T>(path, init, opts, baseHeaders): Promise<T> {
  const url = resolveUrl(path);           // lève CONFIG_ERROR si URL absente
  const timeoutMs = opts.timeoutMs ?? DEFAULT_TIMEOUT_MS; // 30 s
  const correlationId = opts.correlationId ?? uuidv4();
  const token = opts.skipAuth ? null : await getToken();

  const headers = {
    'X-Correlation-Id': correlationId,
    ...(token ? { Authorization: `Bearer ${token}` } : {}),
    ...(opts.idempotencyKey ? { 'Idempotency-Key': opts.idempotencyKey } : {}),
    ...baseHeaders,
    ...(opts.headers ?? {}),
  };
};
```

- Corrélation générée **par requête** si non fournie — chaque appel est traçable de bout en bout. La composition d’objets par spreads ordonnés définit la précedence des headers (les opts.headers de l’appelant gagnent).

```
const controller = new AbortController();
const timer = setTimeout(() => controller.abort(), timeoutMs);
if (opts.signal) {
  if (opts.signal.aborted) controller.abort();
  else opts.signal.addEventListener('abort', () => controller.abort(), { once: true });
}
```

Concept — composition d’AbortSignal. Deux causes d’annulation (le timeout interne, et le signal de l’appelant — p.ex. un écran démonté) sont fusionnées dans **un** controller passé à fetch. Le timeout par AbortController couvre le fetch **et** la lecture du corps (contrairement à un simple timeout de connexion). { once: true } : le listener se retire seul.

```
try {
  const response = await fetch(url, { ...init, headers, signal: controller.signal });
  if (!response.ok) {
    const apiError = await parseError(response, correlationId);
    if (response.status === 401 && !opts.skipAuth) {
      await clearToken();
      emitUnauthenticated(); // → AuthContext → retour au login
    }
    throw apiError;
  }
  if (response.status === 204) return undefined as T;
  const body = await response.text();
  return (body ? JSON.parse(body) : undefined) as T;
} catch (err) {
  if (err instanceof ApiError) throw err;
  const aborted = controller.signal.aborted;
  throw new ApiError({
    code: aborted ? 'TIMEOUT' : 'NETWORK_ERROR',
    status: 0, ..., retrieable: true });
} finally {
  clearTimeout(timer);
}
}
```

- **La réaction 401 centralisée** : un token rejeté sur n’importe quel appel authentifié purge la session et notifie l’UI — un seul endroit, pas 15 écrans qui vérifient.
- Le catch requalifie tout échec non-ApiError : si controller.signal.aborted, c’était le timeout (ou l’appelant) → TIMEOUT; sinon panne réseau → NETWORK_ERROR. Les deux retrieable: true.
- finally { clearTimeout(timer) } : jamais de timer fantôme.

18.3 apiRequest et apiUpload

```
export function apiRequest<T>(path, options = {}): Promise<T> {
  const { method = 'GET', body, ...opts } = options;
  const init: RequestInit = { method };
  if (body !== undefined) {
    baseHeaders['Content-Type'] = 'application/json';
    init.body = JSON.stringify(body);
  }
  return send<T>(path, init, opts, baseHeaders);
}
```

apiUpload (multipart) contient deux leçons durement apprises :

```
// Vérifier l'existence du fichier AU point d'étranglement de l'upload
const info = await FileSystem.getInfoAsync(parts.file.uri);
if (!info.exists) {
  throw new ApiError({ code: 'FILE_NOT_FOUND', status: 0, retrieable: false, ... });
}
```

Piège réel (observé sur device). Le fetch RN lit l'URI du fichier **au moment d'envoyer**; si l'OS a déjà purgé le cache (répertoire Camera/ d'expo-camera, cache ImageManipulator), l'erreur native NSCocoaErrorDomain 260 remonte comme une NETWORK_ERROR générique **retryable** → 5 retries effrayants pour un fichier qui ne reviendra jamais. Le check préalable transforme ça en FILE_NOT_FOUND **non-retryable** → l'op dead-letter proprement et la carte affiche « Envoi impossible ».

```
form.append(parts.fileField ?? 'image', parts.file as unknown as Blob);
```

- RN accepte { uri, name, type } comme part de fichier; le cast as unknown as Blob réconcilie avec le typage DOM. Le Content-Type du formulaire est **laissé vide** : fetch pose lui-même le multipart/form-data; boundary=... (le fixer à la main casse le boundary).
- idempotencyKey: options.idempotencyKey ?? uuidv4() : un upload est une écriture — clé toujours présente.

```
export function getIngestionStatus(ingestionId, options = {}) {
  ...
  if (waitSeconds && waitSeconds > 0 && lastStatus) {
    path += `?wait=${encodeURIComponent(String(waitSeconds))}&last_status=${encodeURIComponent_
    ↪ (lastStatus)}`;
  }
  ...
}
```

Piège RN. La construction de query se fait à la main car le polyfill URL de React Native **n'implémente pas** URLSearchParams.toString() (il lève à l'exécution — et Node l'implémente, donc Jest ne l'attraperait jamais). encodeURIComponent est universel. Méfie-toi des APIs « standard » sous RN : le runtime n'est ni Node ni un navigateur.

Le reste du fichier : wrappers typés fins (createIngestion, confirmIngestion, getExtractionRun, overrideField — qui n'envoie force_gs1: true que si demandé) + ré-export des types API pour un import unique.

18.4 types/api.ts et types/Article.ts

- types/api.ts **reflète le contrat serveur** (sous-ensembles fidèles des vues : IngestionStatusResponse, ExtractionField, ExtractionRunSummary...) — champ pour champ, avec les mêmes unions littérales (IngestionStatus = les 12 états, ValidationStatus, ConfidenceBand). Les champs que le client ne consomme pas (raw_artifacts, audit) sont volontairement omis.
- types/Article.ts est le **modèle local** : projection compacte des champs (ArticleField : nom, valeur, confiance, bande, statut, edited?) + métadonnées (ingestion_id, photo_uri, saved_by...) + raw_extraction_run complet gardé pour debug (stocké **hors du chemin chaud**, §20.6). Le drapeau edited marque une correction humaine locale — le run machine original reste intact dans raw_extraction_run (doctrine d'immutabilité, jusque dans le téléphone).

19. Le pipeline mobile : outbox locale, soumission, polling, scan queue

Quatre couches, du bas vers le haut : **outbox** (transport durable) → **ingestionSubmit** (exécution d'un op) → **ingestionPolling/ingestionResult** (suivi serveur) → **scanQueue** (état de workflow UI). Chacune a une responsabilité stricte et ne réimplémente pas celle du dessous.

19.1 services/outbox.ts — la file durable côté client

Le miroir conceptuel de l'outbox serveur, sur AsyncStorage :

```
export type OutboxStatus = 'pending' | 'in_flight' | 'succeeded' | 'dead_letter';
export type OutboxOperationType =
  | 'create_ingestion' | 'poll_ingestion_status' | 'override_field' | 'confirm_ingestion';

interface OutboxOperationBase {
  id: string; // id stable de l'op
  idempotencyKey: string; // stable – envoyé en Idempotency-Key
  correlationId: string; // stable – envoyé en X-Correlation-Id
  status: OutboxStatus;
  attempt_count: number;
  next_attempt_at: string; // ISO – l'op est "due" quand <= now
  last_error_code: string | null;
  last_error_message: string | null;
  result: OutboxResult | null; // posé au succès (porte l'ingestion_id)
  created_at: string; updated_at: string;
}
```

- **La garantie centrale** : id / clé d'idempotence / corrélation sont générés **une seule fois à l'enqueue** et jamais régénérés. Chaque retry envoie donc la même clé → le serveur dédup → un retry ne peut jamais créer un doublon. (Générer une clé par *tentative* est le bug classique qui rend l'idempotence inutile.)

```
export type OutboxOperation =
  | CreateIngestionOperation | PollIngestionStatusOperation
  | OverrideFieldOperation | ConfirmIngestionOperation;
```

Concept — union discriminée. Chaque interface fixe son type littéral (type: 'create_ingestion'). Un if (op.type === 'create_ingestion') **rétrécit** automatiquement le type de op.payload — le compilateur garantit qu'on ne lit pas le payload d'un autre op.

Backoff et politique de retry (pur, exporté pour les tests) :

```
export function backoffDelayMs(attemptCount, baseMs = 1_000, maxMs = 300_000): number {
  const exponent = Math.max(0, attemptCount - 1);
```

```

const capped = Math.min(baseMs * 2 ** exponent, maxMs);
const jitter = Math.random() * capped * 0.5; // jusqu'à +50 %
return Math.min(Math.round(capped + jitter), maxMs);
}

const NON_RETRYABLE_CODES = new Set([
  'CONFIG_ERROR', 'VALIDATION_ERROR', 'UNAUTHENTICATED', 'FORBIDDEN', 'NOT_FOUND',
  'UNSUPPORTED_MEDIA_TYPE', 'PAYLOAD_TOO_LARGE', 'IDEMPOTENCY_KEY_CONFLICT',
  'ALERT_INVALID_TRANSITION', 'FILE_NOT_FOUND']);

export function isRetryableError(code, status): boolean {
  if (NON_RETRYABLE_CODES.has(code)) return false;
  if (status === 429) return true; // rate-limit → backoff puis retry
  if (status === 0) return true; // réseau/timeout
  if (status >= 500) return true; // panne serveur
  if (status >= 400) return false; // autres erreurs client
  return true;
}

```

- La liste des non-retryables se lit comme une politique produit : une erreur que le retry ne peut pas réparer (config, validation, fichier disparu) va **directement** en dead_letter — pas 5 tentatives pour rien.

Le mutex d'écriture :

```

let writeQueue: Promise<unknown> = Promise.resolve();

function mutate<T>(fn: (ops: OutboxOperation[]) => { ops: OutboxOperation[]; result: T }): Promise<T>
↪ {
  const run = writeQueue.then(async () => {
    const current = await loadAll();
    const { ops, result } = fn(current);
    await AsyncStorage.setItem(OUTBOX_KEY, JSON.stringify(ops));
    return result;
  });
  writeQueue = run.then(() => undefined, () => undefined);
  return run;
}

```

Concept — mutex par chaînage de promesses. AsyncStorage n'a pas de compare-and-set : deux read-modify-write concurrents s'écraieraient. Chaque mutation est **chaînée** derrière la précédente via writeQueue : dans un runtime JS mono-thread, cela sérialise totalement les écritures. Le rechaînage run.then(ok, ok) avale l'erreur *dans la chaîne* (pour ne pas bloquer les mutations suivantes) tout en la laissant se propager à l'appelant via return run. Motif à connaître par cœur pour tout stockage clé-valeur sans transaction.

Transitions d'état (toutes via mutate) : markInFlight (claim : ne prend que pending → protège d'une double exécution), markSucceeded (enregistre le result), markFailed :

```

const attempt_count = op.attempt_count + 1;
const retrieable = error.retrieable ?? isRetryableError(error.code, error.status);
const toDeadLetter = !retrieable || attempt_count >= MAX_ATTEMPTS; // 5
updated = { ...op, attempt_count,
  status: toDeadLetter ? 'dead_letter' : 'pending',
  next_attempt_at: toDeadLetter ? isoAt(now) : isoAt(now + backoffDelayMs(attempt_count)),
  last_error_code: error.code,
  last_error_message: error.message?.slice(0, 1000) ?? null, ... };

```

- error.retrieable ?? isRetryableError(...) : la vérité du serveur (champ retrieable du problem+json) prime ; l'heuristique locale ne sert que si le serveur n'a rien dit.

Hygiène : purgeTerminalOps (succeeded > 1 jour, dead_letter > 14 jours — jamais pending/in_flight)

empêche la file de croître sans borne; `requeueDeadLetter` rend un op à la file avec budget frais (action manuelle d'opérateur).

19.2 services/ingestionSubmit.ts — exécuter un op de capture

Découpé en deux **exprès** :

```
export function enqueueCapture(input: CaptureInput): Promise<CreateIngestionOperation> {
  return enqueueCreateIngestion({
    file: { uri: input.fileUri, name: 'label.jpg', type: 'image/jpeg' },
    barcode_raw: input.barcodeRaw,
    client_captured_at: input.capturedAt });
}

export async function executeCreateIngestionOp(opId: string): Promise<SubmitOutcome> {
  const claimed = await markInFlight(opId);
  if (!claimed || claimed.type !== 'create_ingestion')
    return { kind: 'pending', code: 'NOT_CLAIMABLE' };
  try {
    const res = await createIngestion(claimed.payload.file, {...},
      { idempotencyKey: claimed.idempotencyKey, correlationId: claimed.correlationId });
    await markSucceeded(claimed.id, { result: { ingestion_id: res.ingestion_id, ... } });
    return { kind: 'succeeded', ingestionId: res.ingestion_id, replayed: res.replayed };
  } catch (err) {
    const opError = toOperationError(err);
    const updated = await markFailed(claimed.id, opError);
    if (updated?.status === 'dead_letter')
      return { kind: 'dead_letter', code: opError.code, message: ... };
    return { kind: 'pending', code: opError.code };
  }
}
```

- Pourquoi deux temps ? La scan queue **enregistre l'id de l'op avant** la (lente) tentative HTTP : si l'app est tuée en plein upload, le scan persiste avec son `submitOpId`, et la réconciliation au redémarrage relira l'op pour connaître le sort de l'upload. Le résultat à trois issues (`succeeded/pending/dead_letter`) est un type somme que l'appelant *doit* traiter — pas d'exception.

19.3 services/ingestionPolling.ts — le poll borné et annulable

La classification (miroir exact des 12 états serveur) :

```
const PROCESSING = new Set(['raw_stored', 'ocr_running', 'ocr_done', 'extraction_running',
  'processing', 'extracting', 'pending']); // défensif futur
const REVIEW_READY = new Set(['extracted', 'needs_review', 'ocr_skipped_garbage', 'confirmed']);
const FAILED = new Set(['extraction_failed', 'ocr_failed', 'rejected', 'halted_missing_context']);

export function classifyIngestionStatus(status: string): StatusClass {
  if (REVIEW_READY.has(status)) return 'review_ready';
  if (FAILED.has(status)) return 'failed';
  if (PROCESSING.has(status)) return 'processing';
  return 'unknown';
}
```

- `confirmed` est `review_ready` : un re-scan d'une étiquette déjà validée (dédup par hash → même ingestion) doit afficher ses champs, pas une « erreur statut inconnu ». Chaque état du serveur **doit** classifier — `unknown` remonte comme erreur explicite.

La boucle :

```

export async function pollIngestionUntilReady(ingestionId, options = {}): Promise<PollResult> {
  const { signal, maxDurationMs = 30_000, maxAttempts = 24, baseDelayMs = 1_000,
    maxDelayMs = 5_000, requestTimeoutMs = 10_000, longPollSeconds = 25, onInterim } = options;
  let lastStatus: string | null = null;
  let longPollTrusted = longPollSeconds > 0;
  let interimNotifiedKey: string | null = null;

  while (!signal?.aborted && attempt < maxAttempts && Date.now() - start < maxDurationMs) {
    attempt += 1;
    const remainingS = Math.floor((maxDurationMs - (Date.now() - start)) / 1000);
    const waitS = longPollTrusted && lastStatus !== null
      ? Math.max(0, Math.min(longPollSeconds, remainingS)) : 0;
    const requestStartedAt = Date.now();
    try {
      const ingestion = await getIngestionStatus(ingestionId, {
        signal,
        timeoutMs: waitS * 1000 + requestTimeoutMs, // le timeout doit SURVIVRE au hold
        waitSeconds: waitS > 0 ? waitS : undefined,
        lastStatus: waitS > 0 && lastStatus !== null ? lastStatus : undefined,
      });
      const cls = classifyIngestionStatus(ingestion.status);
      if (cls === 'review_ready') return { kind: 'review_ready', ingestion };
      if (cls === 'failed') return { kind: 'failed', status: ingestion.status };
      if (onInterim && ingestion.status === 'ocr_done') {
        const key = `ocr_done:${ingestion.interim_fields?.length ?? 0}`;
        if (key !== interimNotifiedKey) { interimNotifiedKey = key; onInterim(ingestion); }
      }
      if (cls === 'unknown') return { kind: 'error', code: 'UNKNOWN_STATUS', ... };

      const unchanged = ingestion.status === lastStatus;
      lastStatus = ingestion.status;
      if (waitS > 0 && unchanged && Date.now() - requestStartedAt < MIN_LONG_POLL_MS) {
        longPollTrusted = false; // anti-spin : le serveur n'a pas vraiment retenu
      }
      if (longPollTrusted) continue; // le serveur fait l'attente → re-demander direct
    } catch (err) {
      if (signal?.aborted) return { kind: 'aborted' };
      const retrieable = err instanceof ApiError ? err.retriable : false;
      if (!retriable) return { kind: 'error', ... };
    }
    await cancellableDelay(backoffDelayMs(attempt, baseDelayMs, maxDelayMs), signal);
  }
  if (signal?.aborted) return { kind: 'aborted' };
  return { kind: 'timeout' };
}

```

Les six idées à retenir :

1. **Premier GET immédiat** (pas de wait sans baseline), puis chaque requête suivante demande au **serveur** de retenir jusqu'au changement — latence de découverte ≈ 0 au lieu de la cadence de poll.
2. **Le timeout HTTP dépasse le hold** ($\text{waitS} \times 1000 + 10\text{s}$) — sinon chaque long-poll arrivant à expiration serait un faux TIMEOUT.
3. **Anti-spin** : un long-poll revenu *inchangé* en < 750 ms prouve que le serveur (ancienne version?) ignore les paramètres → on cesse de lui faire confiance et on repasse à la cadence classique. Jamais de boucle chaude contre le backend.
4. **Cadence classique en deux régimes** : 12 tentatives plates ~ 1 s (une extraction atterrit presque toujours < 12 s — un backoff exponentiel ici n'ajouterait que du temps mort d'écran squelette), puis backoff exponentiel plafonné 5 s, avec petit jitter pour désaligner les clients.

5. **Triple borne** : signal d'annulation, `maxAttempts`, `maxDurationMs` — ce poll ne peut structuellement pas durer pour toujours.
6. **onInterim déduplicué** par clé `status:count` — le callback (préviews wave 2) tire au plus une fois par changement réel, pas à chaque poll.

`cancellableDelay` mérite le détour :

```
function cancellableDelay(ms, signal): Promise<void> {
  return new Promise((resolve) => {
    if (signal?.aborted) return resolve();
    const timer = setTimeout(resolve, ms);
    signal?.addEventListener('abort', () => { clearTimeout(timer); resolve(); }, { once: true });
  });
}
```

- Un `setTimeout` que l'abort **réveille immédiatement** (il *résout*, ne rejette pas — la boucle vérifie `signal.aborted` elle-même). Sans ça, annuler un poll attendrait la fin du délai en cours.

19.4 services/ingestionResult.ts — du poll au résultat de revue

```
export async function waitForIngestionResult(ingestionId, options = {}): Promise<IngestionResult> {
  const result = await pollIngestionUntilReady(ingestionId, {
    signal, maxDurationMs, baseDelayMs, longPollSeconds,
    onInterim: onInterim ? (ingestion) => {
      const values: Record<string, string> = {};
      for (const f of ingestion.interim_fields ?? []) values[f.field_name] = f.value;
      onInterim(values);
    } : undefined,
  });
  if (result.kind !== 'review_ready') return result;

  const latest = runs.find((r) => r.is_latest) ?? runs[runs.length - 1];
  const embedded = result.ingestion.latest_fields;
  if (latest && embedded) {
    run = { run_id: latest.run_id, ..., fields: embedded }; // chemin rapide : 0 round-trip
  } else if (latest) {
    try { run = await getExtractionRun(latest.run_id, { signal }); } // fallback vieux serveur
    catch { run = null; } // review-ready mais corps illisible → état "champs indisponibles"
  }
  return { kind: 'ready', ingestion: result.ingestion, run };
}
```

- **Chemin rapide** : le serveur embarque `latest_fields` (§12.5) → le run est reconstruit localement, zéro requête supplémentaire au moment exact où l'opérateur attend. Le fallback réseau ne sert qu'aux vieux serveurs ; son échec produit `run: null` (l'écran affiche un état dégradé propre) plutôt qu'une boucle.

19.5 services/scanQueue.ts — la machine à états du workflow (le chef d'orchestre)

Responsabilité : posséder le cycle de vie de chaque scan en cours, à *travers les écrans*. La caméra tire et oublie ; cette file conduit `submit` → `poll` → `ready` même si aucun écran n'est monté ; l'accueil la rend en section « En cours ».

Frontières (à ne jamais brouiller, dixit le docstring) :

- l'**outbox** reste le transport durable (retry/backoff/DLQ/clés stables) — la file référence ses ops par id, ne réimplémente pas sa politique ;

- les **entrées de file** = état d'UI (quel scan, quelle étape, quelle photo), persistées sous **une** clé AsyncStorage (borné : quelques dizaines d'entrées);
- les **résultats** (payloads ingestion/run) = mémoire seulement — après redémarrage, un GET les re-fetch (le serveur embarque latest_fields : il est la vérité).

```
export type PendingScanStatus =
  | 'submitting' | 'extracting' | 'ready' | 'submit_error' | 'extract_error';

export interface PendingScan {
  id: string; // uuid local — aussi le nom du fichier photo pending/
  createdAt: string; // T0 de la latence photo→ready
  photoUri: string; // copie durable pending/<id>.jpg
  barcodeRaw?: string;
  capturedAt: string;
  submitOpId: string; // lien vers l'op outbox (transport)
  ingestionId: string | null; // null tant que le submit n'a pas abouti
  status: PendingScanStatus;
  ocrDone?: boolean; // wave 2 atteinte (détail du stepper)
  errorCode?: string;
  edits?: Record<string, string>; // brouillon de revue persisté (session v2)
}
```

Le store externe (module singleton)

```
let scans: PendingScan[] = [];
let results: Record<string, ScanResult> = {};
let interim: Record<string, Record<string, string>> = {};
let snapshot: ScanQueueSnapshot = { scans: [], results: {}, interim: {} };
const listeners = new Set<() => void>();

function notify(): void {
  snapshot = { scans: [...scans], results: { ...results }, interim: { ...interim } };
  for (const cb of listeners) cb();
}

export function subscribe(cb: () => void): () => void {
  listeners.add(cb);
  return () => listeners.delete(cb);
}

export function getSnapshot(): ScanQueueSnapshot { return snapshot; }
```

Concept — store useSyncExternalStore. Le contrat React : subscribe(cb) (rappeler cb à chaque changement) + getSnapshot() (retourner une référence **stable** tant que rien n'a changé). D'où le rebuild du snapshot **une fois par notify** — si getSnapshot construisait un objet neuf à chaque appel, React re-rendrait en boucle infinie. Les hooks (§19.7) ne sont que des adaptateurs fins ; l'état vit hors de React, donc les polls tournent écrans démontés.

La persistance sérialise ses écritures comme l'outbox (writeQueue), et seule la liste scans est persistée (résultats en mémoire par design).

Le scheduler de polls

```
const MAX_CONCURRENT_POLLS = 3; // chaque long-poll retient ~25 s de connexion serveur
const FIRST_PASS_MS = 45_000; // 1er passage : long-poll
const DEGRADED_PASS_MS = 60_000; // ensuite : cadence lente classique
const DEGRADED_BASE_DELAY_MS = 5_000;
const TOTAL_POLL_BUDGET_MS = 5 * 60_000; // budget total par scan
```

```

const controllers = new Map<string, AbortController>(); // 1 par scan en poll
const pollStartedAt = new Map<string, number>(); // ancre du budget total
const waitList: string[] = []; // FIFO au-delà du cap
let pollsPaused = false; // app en arrière-plan

function schedulePoll(id: string): void {
  if (pollsPaused) return;
  if (controllers.has(id) || waitList.includes(id)) return; // déjà en cours/attente
  if (controllers.size >= MAX_CONCURRENT_POLLS) { waitList.push(id); return; }
  startPoll(id);
}

function startPoll(id: string): void {
  const scan = findScan(id);
  if (!scan?.ingestionId) return;
  const controller = new AbortController();
  controllers.set(id, controller);
  if (!pollStartedAt.has(id)) pollStartedAt.set(id, Date.now());
  void runPoll(id, scan.ingestionId, controller).finally(() => {
    controllers.delete(id);
    startNextWaiting(); // libérer le slot → tirer le suivant de la FIFO
  });
}

```

- **Cap de concurrence** : au-delà de 3 long-polls simultanés, les scans attendent en FIFO — avec la découverte quasi instantanée du Tier 4, la file avance vite, et le pool serveur n'est pas pilonné par N cadences parallèles.

```

async function runPoll(id, ingestionId, controller): Promise<void> {
  const startedAt = pollStartedAt.get(id) ?? Date.now();
  const elapsed = Date.now() - startedAt;
  const remaining = TOTAL_POLL_BUDGET_MS - elapsed;
  if (remaining <= 0) {
    pollStartedAt.delete(id);
    updateScan(id, { status: 'extract_error', errorCode: 'TIMEOUT' });
    return;
  }
  const degraded = elapsed > FIRST_PASS_MS;
  const result = await waitForIngestionResult(ingestionId, {
    signal: controller.signal,
    maxDurationMs: Math.min(degraded ? DEGRADED_PASS_MS : FIRST_PASS_MS, remaining),
    ...(degraded ? { longPollSeconds: 0, baseDelayMs: DEGRADED_BASE_DELAY_MS } : {}),
    onInterim: (values) => { interim[id] = values; updateScan(id, { ocrDone: true }); },
  });
  if (!findScan(id)) return; // supprimé/validé pendant le poll
  switch (result.kind) {
    case 'ready':
      results[id] = { ingestion: result.ingestion, run: result.run };
      delete interim[id]; pollStartedAt.delete(id);
      updateScan(id, { status: 'ready', ocrDone: true, errorCode: undefined });
      break;
    case 'failed': ... 'extract_error' avec le statut ...
    case 'error': ... 'extract_error' avec le code ...
    case 'timeout': schedulePollAfterCurrent(id); break; // re-armer (budget persistant)
    case 'aborted': break; // background/suppression
  }
}

```

- **Dégradation en trois temps** : 45 s de long-poll (réactivité maximale), puis passes classiques lentes (5 s) jusqu'au budget total de 5 min, puis `extract_error` `TIMEOUT`

(l'opérateur peut « Réessayer » — budget frais). L'ancre `pollStartedAt` survit aux passes; elle n'est effacée qu'aux issues terminales.

```
function schedulePollAfterCurrent(id: string): void {
  void Promise.resolve().then(() => {
    if (findScan(id)?.status === 'extracting') schedulePoll(id);
  });
}
```

- Détail de concurrence fin : au moment du timeout, le `finally` de `startPoll` n'a pas encore libéré le slot — re-planifier immédiatement verrait un compteur faux. `Promise.resolve().then(...)` diffère d'une microtâche, après le `finally`.

Soumission et réconciliation

```
export async function enqueueScan(input): Promise<PendingScan> {
  const id = input.id ?? uuidv4();
  const photoUri = (await persistPendingPhoto(id, input.tempUri)) ?? input.tempUri; // copie durable
  ↪ D'ABORD
  const op = await enqueueCapture({ fileUri: photoUri, ... }); // op durable
  const scan: PendingScan = { id, ..., submitOpId: op.id, ingestionId: null, status: 'submitting' };
  scans = [...scans, scan]; persist(); notify();
  void executeCreateIngestionOp(op.id).then(
    (outcome) => handleSubmitOutcome(id, outcome),
    () => undefined);
  return scan; // résout dès que l'entrée existe — la caméra n'attend PAS le réseau
}

function handleSubmitOutcome(id, outcome): void {
  if (outcome.kind === 'succeeded') {
    const duplicate = outcome.replayed
      ? scans.find((s) => s.id !== id && s.ingestionId === outcome.ingestionId)
      : undefined;
    if (duplicate) { // deux photos identiques → même ingestion serveur
      const removed = removeScan(id);
      if (removed) void deletePendingPhoto(removed.photoUri);
      return; // sinon : double poll + double revue du même contenu
    }
    updateScan(id, { ingestionId: outcome.ingestionId, status: 'extracting', errorCode: undefined });
    schedulePoll(id);
  } else if (outcome.kind === 'dead_letter') {
    updateScan(id, { status: 'submit_error', errorCode: outcome.code });
  }
  // 'pending' : l'op reste sur l'outbox avec son backoff ; le drain la rejouera
  // et reconcileScanQueue() avancera la carte quand elle aura abouti.
}
```

- La **dédup replay** ferme la boucle de l'idempotence serveur jusque dans l'UI : si le serveur dit `replayed=true` et qu'une autre carte porte déjà cette ingestion, la carte dupliquée se supprime.

```
export async function reconcileScanQueue(): Promise<void> {
  for (const scan of [...scans]) {
    if (scan.status === 'submitting') {
      const op = await getOperation(scan.submitOpId);
      if (!op) updateScan(scan.id, { status: 'submit_error', errorCode: 'OP_MISSING' });
      else if (op.status === 'succeeded' && op.result)
        handleSubmitOutcome(scan.id, { kind: 'succeeded', ingestionId: op.result.ingestion_id,
          replayed: op.result.replayed });
      else if (op.status === 'dead_letter')
    }
  }
}
```

```

        updateScan(scan.id, { status: 'submit_error', errorCode: op.last_error_code ?? 'UNKNOWN' });
        // pending / in_flight : le drain s'en occupe - l'étape 1 reste "en cours"
    } else if (scan.status === 'extracting' || (scan.status === 'ready' && !results[scan.id])) {
        schedulePoll(scan.id);
    }
}
}
}

```

- **La réconciliation est l'auto-réparation** : appelée au démarrage, au retour au premier plan et après chaque drain. Elle relit la vérité du transport (l'op) et re-arme les polls — y compris pour un scan ready dont le résultat en mémoire a disparu (redémarrage).

```

export async function initScanQueue(): Promise<void> {
    if (hydrated) return;
    hydrated = true;
    ...charger QUEUE_KEY, parser défensivement...
    notify();
    void sweepPendingPhotos(scans.map((s) => s.photoUri)); // orphelins d'un crash
    await reconcileScanQueue();
}

export function registerScanQueueLifecycle(): () => void {
    const onChange = (state: AppStateStatus) => {
        if (state === 'active') { pollsPaused = false; void reconcileScanQueue(); }
        else if (state === 'background') {
            pollsPaused = true;
            for (const [id] of controllers) abortPoll(id);
            waitList.length = 0;
        }
    };
    const sub = AppState.addEventListener('change', onChange);
    return () => sub.remove();
}

```

- En arrière-plan, **tous les polls sont annulés** (un hold de 25 s dans une app backgroundée est du travail perdu); au retour, réconciliation → re-armement. `waitList.length = 0` vide le tableau **en place** (les références restent valides).
- `_resetScanQueueForTests()` expose la remise à zéro complète du singleton — indispensable pour Jest (l'état module survit entre tests).

19.6 services/outboxDrain.ts — le vidage au premier plan

```

let draining = false;
let rerunRequested = false;

export async function drainOutbox(now = Date.now()): Promise<DrainResult> {
    if (draining) { rerunRequested = true; return result; }
    draining = true;
    try {
        do {
            rerunRequested = false;
            const due = await listPendingDue(Math.max(now, Date.now())); // horloge fraîche par passe
            for (const op of due) {
                if (op.type !== 'override_field' && op.type !== 'confirm_ingestion'
                    && op.type !== 'create_ingestion') { result.skipped += 1; continue; }
                const claimed = await markInFlight(op.id);
                if (!claimed) continue; // raflé par un autre exécuteur
                try {

```

```

    const exec = await executeOperation(claimed); // dispatch par op.type
    await markSucceeded(claimed.id, exec.handled && exec.result ? { result: exec.result } : {});
    result.succeeded += 1;
  } catch (err) {
    await markFailed(claimed.id, toOperationError(err));
    result.failed += 1;
  }
}
} while (rerunRequested);
await purgeTerminalOps({ now });
} finally {
  draining = false; rerunRequested = false;
}
if (result.succeeded > 0) void reconcileScanQueue(); // APRÈS le latch – jamais de re-drain
return result;
}

```

- **Latch + trailing re-pass** : un déclenchement pendant un drain n'est pas perdu — il lève `rerunRequested`, et la boucle `do/while` refait une passe (l'op enqueued mi-drain, p.ex. le confirm d'un save arrivé pendant le drain de premier plan, part immédiatement). L'horloge est reprise à chaque passe : un `now` figé ne verrait jamais « due » un op créé en cours de drain.
- La sûreté vient du serveur : chaque replay porte la clé d'idempotence stable → migration 0013 → l'op qui AVAIT atteint le backend rejoue son résultat original, jamais une double écriture.
- `registerOutboxDrainOnForeground()` : drain sur chaque passage à active **et** un drain immédiat à l'enregistrement (l'app vient de démarrer = le moment le plus plausible du retour de connectivité).

19.7 hooks/useScanQueue.ts

```

export function useScanQueue(): ScanQueueSnapshot {
  return useSyncExternalStore(subscribe, getSnapshot);
}

export function useScan(id: string): ScanView {
  const snap = useSyncExternalStore(subscribe, getSnapshot);
  return {
    scan: snap.scans.find((s) => s.id === id) ?? null,
    result: snap.results[id] ?? null,
    interimValues: snap.interim[id] ?? NO_INTERIM, // constante stable, pas {} neuf
  };
}

```

- Adaptateurs purs, zéro cycle de vie : les polls appartiennent au service. `NO_INTERIM` est une constante module : retourner un `{}` littéral neuf à chaque rendu casserait les mémoisations en aval (nouvelle référence à chaque fois).

20. Les services métier purs du mobile

Une douzaine de modules **purs et sans framework** (testés sans React) — le pendant mobile du « domaine » backend.

20.1 services/gsl.ts — le miroir client du parseur GS1

Port TypeScript fidèle de server/domain/gsl.py (mêmes AI, mêmes règles, mêmes warnings). Son rôle : **l’affichage à T+0** — lot et DLC remplis à l’écran de revue dès l’instant de la photo, pendant que le serveur travaille. Le backend reste la vérité au save (il refait le même décodage + réconciliation); ceci est une preview.

Différences d’implémentation dictées par le langage :

```
function lastDay(year: number, month: number): number {
  return new Date(year, month, 0).getDate(); // jour 0 du mois suivant = dernier jour
}
```

- L’astuce JS `new Date(y, m, 0)` remplace le calcul ordinal Python. `decodeDate` ajoute un garde `dd > lastDay` que la construction `date()` Python faisait implicitement.

```
export function gslFieldValues(gsl: GslDecoded): Record<string, string | undefined> {
  const expiry = gsl.expiryDate ?? gsl.bestBefore;
  return {
    batch_number: gsl.lot ?? undefined,
    expiry_date: expiry ? displayDate(expiry) : undefined,
    packaging_date: gsl.packagingDate ? displayDate(gsl.packagingDate) : undefined,
    weight: gsl.netWeightKg !== null ? formatGslWeightKg(gsl.netWeightKg) : undefined,
    gtin: gsl.gtin ?? undefined,
  };
}
```

- La projection « valeurs par nom de champ d’extraction », **formatée pour l’affichage** (dates en JJ/MM/AAAA) — exactement ce que les lignes de l’écran de revue consomment. Même priorité DLC > DDM que la réconciliation serveur.

20.2 services/inputMasks.ts — masques de saisie et validation neutre

Le module le plus « métier » de l’UI, avec un principe de conformité en tête de fichier :

Jamais de valeur calculée. Il n’y a délibérément **pas** de « DLC = emballage + N jours » : la durée de vie n’est pas une constante (frais = jours, congelé = mois) — une DLC calculée serait une **fabrication d’une valeur de sécurité sanitaire** et déclencherait de fausses alertes HACCP. Les masques formatent les frappes de l’opérateur, ils n’inventent jamais un caractère.

```
export function maskDate(input: string): string {
  const d = input.replace(/\D/g, '').slice(0, 8); // chiffres seuls, max JJMMAAAA
  if (d.length <= 2) return d;
  if (d.length <= 4) return `${d.slice(0, 2)}/${d.slice(2)}`;
  return `${d.slice(0, 2)}/${d.slice(2, 4)}/${d.slice(4)}`;
}
```

- Auto-insertion des / en re-masquant **toute** la chaîne à chaque frappe : le backspace re-masque naturellement la chaîne raccourcie (pas de gestion de curseur piègeuse).

```
export function displayDate(value: string): string { // ISO → JJ/MM/AAAA
  const iso = /^(d{4})-(d{2})-(d{2})$/.exec(value.trim());
  if (!iso) return value; // déjà FR, partiel, ou verbatim → INCHANGÉ
  const [, y, m, d] = iso;
  return `${d}/${m}/${y}`;
}
export function toIsoDate(value: string): string { // JJ/MM/AAAA → ISO (inverse exact)
  const m = /^(d{2})\/(d{2})\/(d{4})$/.exec(value.trim());
  if (!m) return value.trim();
  const [, dd, mm, yyyy] = m;
  return `${yyyy}-${mm}-${dd}`;
}
```

Concept — stockage canonique vs affichage. ISO YYYY-MM-DD est le format **canonique** (celui du backend; son check chronologique expiry < packaging repose sur l'ordre lexical ISO, que JJ/MM/AAAA casse dès qu'on change d'année). JJ/MM/AAAA est **présentation pure**. Les deux fonctions sont inverses exactes; toute synchronisation vers le backend reconvertit à la frontière. Ne jamais laisser un format d'affichage s'infiltrer dans le stockage.

Les parseurs d'affixes (poids/température/prix) découpent une chaîne stockée en morceaux éditables et la reconstruisent :

```
export function parseWeight(value): { amount: string; unit: 'kg' | 'g' } {
  const num = /-?\d+(?:[.]\d+)?/.exec(v);
  const amount = num ? num[0].replace(',', '.') : '';
  const unit = /\bg\b/i.test(v) && !/kg/i.test(v) ? 'g' : 'kg';
  return { amount, unit }; // PAS de conversion g→kg : "320 g" ne devient jamais "320 kg"
}
```

- parseTemp gère l'ambiguïté du - (signe négatif **et** séparateur de plage) : une vraie plage (0-4, -2-4) remplit min et max; <=4/>=-18/valeur nue remplit une borne. parsePrice défaut EUR (le standard créée) sans jamais inventer un montant.

Les **hints de validité** — neutres et non bloquants (doctrine « Clean UI » : jamais de rouge, jamais de blocage, jamais d'auto-correction) :

```
export function validateDate(value: string): string | null {
  ...
  if (month < 1 || month > 12) return 'Mois invalide';
  if (year < 2000 || year > 2100) return 'Année invalide';
  if (day < 1 || day > new Date(year, month, 0).getDate()) return 'Jour invalide';
  return null; // vide, valide, ou EN COURS DE FRAPPE → pas de hint
}
```

- Une saisie partielle n'est **pas** « invalide » (return null au lieu de crier pendant la frappe). validateTempRange : min > max, ou hors [-100, 60] → « Température inhabituelle ». validateHealthMark : une estampille sans chiffre est suspecte — signalée, jamais bloquée. maskHealthMark force juste la casse (l'estampille officielle est en majuscules).

20.3 fieldOrder.ts, fieldLabels.ts, fieldCompleteness.ts

```
export const FIELD_ORDER = [
  'commercial_designation', 'scientific_name', 'producer_name', 'reseller_brand',
  'production_method', 'fishing_gear_or_farming_method', 'FAO_area', 'origin_country',
  'health_mark', 'batch_number', 'expiry_date', 'packaging_date',
  'storage_temperature', 'weight', 'allergens', 'price', 'gtin',
] as const;
```

- L'ordre de lecture HACCP (identité → méthode/origine → sanitaire → lot/dates → conservation), **partagé** par l'écran de revue et la fiche produit : l'app lit les champs dans un seul ordre de bout en bout. `as const` fige le tuple en littéraux.

`fieldLabels.ts` : les libellés français professionnels (`commercial_designation` → « Dénomination commerciale »), les statuts de validation, le cycle de vie (`INGESTION_STATUS_FR` couvre **les 12 états** — aucun code technique n'atteint jamais l'opérateur), le vocabulaire contrôlé (`wild_caught` → « Pêche sauvage »). `displayFieldValue` francise la méthode de production et formate les dates — présentation seulement, la valeur stockée reste canonique. Nuance conformité : `FAO_area` s'affiche **verbatim** (précision pleine « 27.8.b.1 ») — le joli nom de zone est une annotation décorative, jamais ce qui est persisté/exporté.

`fieldCompleteness.ts` — la jauge « n/17 » :

```
export function filledCountFromRun(fields, edits?): number {
  const byName = new Map(...);
  let count = 0;
  for (const name of FIELD_ORDER) {
    if (edits && name in edits) { // le brouillon PRIME, dans les 2 sens
      if (isFilledValue(edits[name])) count += 1;
      continue;
    }
    const f = byName.get(name);
    if (!f) continue;
    if (f.validation_status === 'missing') continue; // "missing" ne compte jamais
    if (isFilledValue(f.value)) count += 1;
  }
  return count;
}
```

- Trois variantes (run final / préview interim / map de valeurs effectives) alimentent la carte d'accueil **et** le verrou de save 17/17 de l'écran de revue avec la même logique. Le brouillon d'édition (`edits`) écrase dans les deux sens : une valeur tapée remplit, un champ vidé dé-remplit — la jauge suit la session de l'opérateur en direct.

20.4 fieldHistory.ts et allergenSuggestions.ts — l'aide à la saisie, sans fabrication

`fieldHistory` — autocomplétion depuis **l'historique validé de l'opérateur** (une criée re-saisit les mêmes espèces/producteurs/zones chaque jour) :

```
export const HISTORY_FIELDS = ['commercial_designation', 'scientific_name', 'producer_name',
  'reseller_brand', 'production_method', 'fishing_gear_or_farming_method',
  'FAO_area', 'origin_country', 'health_mark'];
```

- Exclusions raisonnées : lot/dates/gtin (uniques par lot), poids/température/prix (numériques par lot), allergènes (suggestion réglementaire dédiée).

```
export function buildFieldHistory(articles): FieldHistory {
  // par champ : dédup par forme normalisée (casse/accents pliés),
  // la casse d'affichage LA PLUS RÉCENTE gagne,
```

```
// tri par fréquence puis récence
}
export function suggestForField(history, fieldName, draft, max = 3): string[] {
  // brouillon vide → le top ; sinon matchs par PRÉFIXE d'abord, puis substring ;
  // jamais la valeur déjà entièrement tapée
}
```

- Position de conformité : les suggestions ne proposent que des valeurs **qu'un humain a déjà validées** sur un arrivage précédent, et en accepter une EST une édition humaine (même chemin que la frappe) — le gate no-fabrication n'est pas touché.

allergenSuggestions — support de décision (PAS de l'extraction) :

```
if (MIXED.test(text)) return null; // "fruits de mer" → jamais deviner
if (matchesAny(text, CRUSTACEANS)) return 'Crustacés'; // plus spécifique d'abord
if (matchesAny(text, MOLLUSCS)) return 'Mollusques';
if (matchesAny(text, FISH)) return 'Poisson';
return null;
```

- Mappe l'espèce vers l'une des trois familles d'allergènes de l'Annexe II UE, par listes de mots-clés FR/EN/latin **bornées par des frontières non alphabétiques** ((^|[^a-z])kw([a-z]|\$)) — « bar » ne matche pas « barque ». Une suggestion est **affichée, jamais auto-appliquée** ; l'accepter en fait une valeur humaine. Le scope est volontairement limité aux allergènes : port→FAO est interdit (mapping non déterministe d'un champ réglementé).
- La normalisation d'accents mérite l'œil :

```
const decomposed = s.toLowerCase().normalize('NFD');
// puis filtrage des code points U+0300–U+036F (diacritiques combinants)
```

Concept — normalisation Unicode NFD. `normalize('NFD')` décompose « é » en « e » + accent combinant ; on jette ensuite les combinants (U+0300–036F). Résultat : comparaison insensible aux accents sans table de correspondance. Le filtre par code point (au lieu d'un `/[-]/g` littéral) garde le fichier source exempt de caractères invisibles.

20.5 faoDisplay.ts, articleSearch.ts, articleGrouping.ts, calendar.ts, dates.ts

`faoDisplay` : re-présente une zone FAO stockée (« 27.5 » → « 27 Atlantique Nord-Est - Sous zone : (V) ») via la **nomenclature officielle déterministe** (numéro ↔ nom officiel, sous-zone → romain). Ce n'est ni un lookup nom-de-lieu→code ni une suggestion : zone inconnue → verbatim, jamais inventé.

`articleSearch` — l'omni-recherche :

```
function haystack(article): string { // tous les champs + le FR du vocabulaire contrôlé +
  ↪ code-barres
  ...
}
function hayMatchesQuery(hay, q): boolean { // multi-termes en AND
  return q.split(/\s+/).every((term) => hay.includes(term));
}
```

- Et le **chemin indexé** : `searchArticles` re-normalise tout à chaque appel — $O(n \times \text{longueur})$ par frappe, janky à 5 000 lots. `buildSearchIndex` précalcule le haystack normalisé **une fois par changement de liste** ; chaque frappe ne fait plus que des includes. Le hook `useArticleSearch` mémorise l'index (`useMemo` sur la liste) et filtre sur la requête.

Tip dev — sépare « préparer » de « chercher ». Quand une recherche re-dérive des données immuables à chaque frappe, extrais la dérivation dans un index construit au changement de données. C'est le même geste que les projections de lecture côté serveur.

articleGrouping : tri A→Z par nom commun via Intl.Collator('fr', { sensitivity: 'base', numeric: true }) (insensible casse/accents, « 2 » < « 10 »); sans-nom en dernier; égalité → plus récent d'abord.

calendar : les maths pures du calendrier d'arrivages (heat-map façon GitHub). dayKey bucketise par **jour local** (jamais UTC — le jour du poissonnier, pas celui de Greenwich); monthGrid produit une grille fixe 6×7 démarrant lundi ((getDay() + 6) % 7) — hauteur de panneau constante en naviguant; intensityLevel mappe le compte du jour sur 0–4 relativement au jour le plus chargé.

dates.ts : formatDate (« 20 juin 2026 ») / formatDateShort (« 20/06/2026 ») — **une** source de vérité de formatage, et une valeur absente/malformée retombe sur aujourd'hui plutôt que d'afficher « Invalid Date » à l'opérateur.

latencyLog.ts : instrumentation dev-only (logLatency('capture', {...})) — de simples lignes console horodatées pour mesurer la latence photo→ready; no-op en production.

export.ts : sérialise les articles en JSON (fidèle) ou CSV (les champs structurés restent du JSON dans une colonne — jamais aplatis en blob), écrit dans documentDirectory, puis feuille de partage native (expo-sharing), avec garde isAvailableAsync.

20.6 Le stockage des articles : port + adapters (audit §7.2)

articleStore.ts — encore un **port**, côté mobile :

```
export interface ArticleStore {
  getAll(): Promise<Article[]>; // LIGHT : raw_extraction_run omis
  getById(id: string): Promise<Article | null>;
  put(article: Article): Promise<void>;
  remove(id: string): Promise<void>;
}
```

Le problème qu'il résout (docstring) : l'implémentation précédente gardait **tous** les articles (chacun avec son run brut complet) dans **un** blob JSON AsyncStorage — réécriture O(n) à chaque save, et plafond dur ~6 Mo (CursorWindow Android) → **perte de données silencieuse** à quelques milliers de lots.

articleStoreAsyncStorage.ts — l'adapter production, une clé par article :

```
const articleKey = (id) => `@labelscan:article:${id}`; // record LIGHT (chemin chaud)
const rawKey     = (id) => `@labelscan:articleRaw:${id}`; // run brut (debug, hors chemin chaud)

async put(article) {
  await this.ensureMigrated();
  await AsyncStorage.multiSet([
    [articleKey(article.id), JSON.stringify(withoutRawRun(article))],
    [rawKey(article.id),     JSON.stringify(article.raw_extraction_run ?? null)],
  ]);
}
```

- Deux clés indépendantes : le record léger sur le chemin chaud, le run lourd à part (aucun écran ne le lit — write-only debug). Save = O(1), plus de plafond global (la limite CursorWindow est par ligne), getById = O(1), getAll ne lit **que** les clés légères (le préfixe article: avec deux-points final ne matche jamais articleRaw: — détail noté en commentaire).
- **Migration paresseuse et idempotente** : au premier accès, le blob legacy est copié en clés par-article, un drapeau one-shot est posé, et le blob est **conservé en backup** (rien n'est détruit). parseArticle ignore tout ce qui n'est pas un record backend_extraction (compat avant/arrière).

storage.ts — la **façade** que les écrans consomment (getAllArticles, saveBackendArticle, updateBackendArticle, deleteArticle) : API publique inchangée pendant que le moteur dessous a été remplacé (setArticleStore(new SqliteArticleStore()) est le point d'injection futur). Elle garde en propre le **cycle de vie des photos** :

- photos/ (documents) : photos des articles sauvés — copiées depuis le cache par `persistPhotoInto` (copie **puis** vérification `getInfoAsync` que la destination existe; extension `.jpg` fixe car dériver l'extension d'un uri de cache `ImageManipulator` est fiable... n'est PAS fiable);
- pending/ : photos des scans en file (possédées par `scanQueue`) — durables car l'uri de cache d'expo-camera peut être purgé par l'OS avant la revue; `deletePendingPhoto` refuse de supprimer hors de `pending/` (un scan retombé sur son uri de cache d'origine n'est pas à nous); `sweepPendingPhotos` supprime au démarrage tout fichier qu'aucun scan ne référence (crash entre copie et écriture de file).

21. Les écrans

21.1 CameraScreen.tsx — le viseur à capture enchaînée

Philosophie : viser → shutter → la photo part dans la scan queue → **on continue de shooter**.
Ni modal, ni attente ; la revue vit sur l'écran d'accueil.

La géométrie du cadre et le crop

```
const TOP_BAR_H = 90; const BOTTOM_TRAY_H = 120; const CAPTION_RESERVE = 44;
const FRAME_LEFT = SCREEN_WIDTH * 0.02;
const FRAME_WIDTH = SCREEN_WIDTH - FRAME_LEFT * 2;      // ~96 % de la largeur
const FRAME_TOP = TOP_BAR_H + 12;
const FRAME_HEIGHT = SCREEN_HEIGHT - FRAME_TOP - BOTTOM_TRAY_H - CAPTION_RESERVE;
const CROP_SAFETY_MARGIN = 0.08;    // +8 % par axe autour du cadre
```

- Le cadre à l'écran est **à la fois** le guide de placement et la région de crop. Il couvre $\approx$ toute la zone utile (l'ancien petit cadre paysage rognait le contenu qui débordait — régression corrigée); la marge de sécurité évite de couper une étiquette collée aux crochets.

computeFrameCrop — inverser la transformation d'affichage :

```
function computeFrameCrop(photoWidth, photoHeight) {
  if (!photoWidth || !photoHeight) return null;
  // Garde d'orientation : photo et écran doivent partager la même orientation
  if (SCREEN_WIDTH > SCREEN_HEIGHT !== photoWidth > photoHeight) return null;

  // Fit "cover" : un seul facteur d'échelle (le plus grand des deux ratios)
  const scale = Math.max(SCREEN_WIDTH / photoWidth, SCREEN_HEIGHT / photoHeight);
  const offsetX = (photoWidth * scale - SCREEN_WIDTH) / 2;    // débord centré
  const offsetY = (photoHeight * scale - SCREEN_HEIGHT) / 2;

  // Cadre élargi de la marge, PUIS inversion écran → pixels photo
  let originX = (frameLeft + offsetX) / scale;
  ...
  // Clamp dans la photo — jamais de pixels hors bornes
  ...
  if (width < 1 || height < 1) return null;
  return { originX: Math.floor(originX), ... };
}
```

Concept — inverser un « cover fit ». La preview remplit l'écran en mode cover : l'image est agrandie jusqu'à couvrir, centrée, le débord est rogné. Pour savoir quels pixels de la photo sont sous le cadre : échelle = max des ratios; débord = (dimension×échelle - écran)/2 ; puis (coordonnée écran + débord) / échelle. Le **garde d'orientation** est le filet : si le buffer revient transposé (mismatch portrait/paysage, sens de rotation inconnaisable), on retourne null et l'appelant envoie l'image entière — dégradé sans perte plutôt que crop faux.

takePhoto — le chemin critique du shutter

```
const takePhoto = useCallback(async () => {
  if (!cameraRef.current || taking) return; // `taking` est le SEUL verrou
  setTaking(true); setFrameState('capturing');
  Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light);
  flashRef.current?.trigger();

  let photo;
  try {
    photo = await cameraRef.current.takePictureAsync({
      quality: 1.0, // fidélité max : les petits caractères doivent OCRer
      skipProcessing: false, // appliquer la rotation capteur → image droite
    });
  } catch (err) { ...frameState 'error' 2,5 s puis 'ready'... return; }

  // La caméra est LIBRE dès que la frame est prise : shutter ré-armé ICI
  if (mountedRef.current) { setTaking(false); setFrameState('ready'); }
  if (!photo || !mountedRef.current) return;

  const capturedAt = new Date().toISOString();
  const barcodeRaw = lastBarcodeRef.current?.data;
  lastBarcodeRef.current = null;

  void (async () => { // pipeline lourd EN ARRIÈRE-PLAN
    try {
      // (1) copie durable D'ABORD — le cache expo-camera peut disparaître mi-crop
      const scanId = uuidv4();
      const durableRawUri = await persistPendingPhoto(`${scanId}-raw`, capturedPhoto.uri);
      if (!durableRawUri) throw new Error('Could not persist the captured photo');

      // (2) crop (si mappable) + downscale (long côté ≤ 1600 px) + compress 0.8
      const crop = computeFrameCrop(capturedPhoto.width, capturedPhoto.height);
      const resize = srcW >= srcH ? { width: Math.min(srcW, 1600) }
        : { height: Math.min(srcH, 1600) };
      const out = await ImageManipulator.manipulateAsync(
        durableRawUri,
        crop ? [{ crop }, { resize }, { rotate: -90 }] : [{ resize }, { rotate: -90 }],
        { compress: 0.8, format: ImageManipulator.SaveFormat.JPEG });

      // (3) enqueue — la scan queue possède la suite
      const scan = await enqueueScan({ id: scanId, tempUri: croppedUri ?? durableRawUri,
        barcodeRaw, capturedAt });
      if (scan.photoUri !== durableRawUri) void deletePendingPhoto(durableRawUri);
    } catch (err) {
      // filet : enqueue l'original du cache tel quel — un cliché ne se perd JAMAIS en silence
      try { await enqueueScan({ tempUri: capturedPhoto.uri, barcodeRaw, capturedAt }); }
      catch (fallbackErr) { console.error('... shot lost:', fallbackErr); }
    }
  })();
}, [taking]);
```

Les décisions, une à une :

- **Le shutter est ré-armé dès la frame saisie** — copie/crop/enqueue tournent hors du chemin critique. C'est ce qui rend la capture enchaînée fluide : N scans en vol est le but, rien n'attend le réseau.
- `skipProcessing: false` : sur Android, `true` renvoyait un buffer **non tourné** → `computeFrameCrop` croppait la mauvaise région. La rotation capteur doit être appliquée pour que `width/height` décrivent l'image droite.

- **Copie durable avant le crop** : bug observé sur device — sous capture rapide enchaînée, l'OS réclamait le fichier du cache **pendant** le crop (NSCocoaErrorDomain 260 à l'upload). On crope depuis notre propre copie.
- La rotate: -90 est appliquée **en dernier** (le calcul de crop raisonne dans l'espace de pixels droit) et **cuite dans le fichier** : étiquettes shootées portrait, lues paysage — le JPEG stocké/uploadé est déjà droit, aucun composant d'affichage n'a à tourner quoi que ce soit.
- Découplage crop/resize volontaire : un échec de crop (orientation) ne doit **pas** envoyer une image pleine résolution à Vision — le downscale s'applique toujours.
- Le fallback final enqueue **l'original du cache** : dégradé (non croppé/non tourné) mais récupérable; la carte d'erreur à l'accueil est le signal ultime.

Code-barres et cycle de vie

```
const handleBarcodeScanned = useCallback((result) => {
  if (taking) return;
  lastBarcodeRef.current = result;    // stocké pour la PROCHAINE capture – pas d'auto-shoot
  setFrameState('barcodeFound');
}, [taking]);
```

- Le scanner natif (ean13/ean8/upc/code128/qr...) tourne en continu; le dernier code lu est associé au prochain shutter puis consommé (lastBarcodeRef.current = null).
- useIsFocused() conditionne le montage du <CameraView> : la ressource caméra (unique) est libérée dès qu'on quitte le viseur; un <View> noir la remplace.
- Torche plutôt que flash : l'ancien flash 3 états ne se déclenchait qu'À la capture — le bouton paraissait cassé. enableTorch éclaire immédiatement (feedback visible) et reste allumé pendant la capture.
- mountedRef : le pipeline async ne touche jamais l'état d'un écran démonté.

21.2 ReviewScreen.tsx — la revue/édition (957 lignes)

L'écran le plus riche. Ouvert depuis l'accueil pour **un** scan (pendingScanId); tout est lu **en live** de la scan queue :

```
const { scan, result, interimValues } = useScan(pendingScanId);
const ready = scan?.status === 'ready';
const run = result?.run ?? null;
```

La liste stable — zéro layout shift

L'idée maîtresse : **une seule liste, en FIELD_ORDER, du premier instant au save.**

```
{FIELD_ORDER.map((name) => {
  const field = fields.find((f) => f.field_name === name);
  if (field) return <EditableFieldRow key={name} field={field}
    draft={effectiveValues[name]} ... />;
  return <PendingFieldRow key={name} fieldName={name} gs1Value={pendingValues[name]} />;
})}
```

- Avant le run : PendingFieldRow — le **libellé réel** + la valeur GS1 (T+0) ou la preview interim, sinon un squelette pulsant. Même géométrie que la ligne éditée → quand le run atterrit, les valeurs **se substituent sans que rien ne bouge** (audit §2.1/2.2 : zéro reflow). Pas de cascade animée, pas de compteur — sobriété radicale.

```
const pendingValues = useMemo(() => {
  const out: Record<string, string> = {};
  for (const [name, value] of Object.entries(interimValues))
    out[name] = isDateField(name) ? displayDate(value) : value;
```

```
return { ...out, ...gs1Values }; // GS1 par-dessus : l'exact ne cède jamais à la regex
}, [interimValues, gs1Values]);
```

L'état d'édition — brouillon par champ

```
const [edits, setEdits] = useState<Record<string, string>>(() => scan?.edits ?? {});
const editsRef = useRef(edits);
editsRef.current = edits;
useEffect(() => () => { saveScanEdits(pendingScanId, editsRef.current); }, [pendingScanId]);
```

- Lazy init depuis le brouillon persisté du scan («session» workflow v2 : un arrivage à moitié rempli survit à la navigation). La sauvegarde du brouillon se fait **une fois, au démontage** — pas à chaque frappe — via le motif ref qui capture la dernière valeur pour le cleanup.

```
const handleFieldChange = useCallback((name: string, text: string) => {
  setEdits((prev) => ({ ...prev, [name]: text }));
}, []);
```

Concept — un callback stable pour N lignes. EditableFieldRow est enveloppé de React.memo, et le parent passe **une seule** fonction (name, text) stable (useCallback sans dépendances, forme fonctionnelle de setEdits). Résultat : une frappe ne re-rend **que la ligne éditée**, pas les 16 autres (audit §7.1). L'alternative naïve — une closure par ligne recréée à chaque rendu — casserait tous les memo.

```
const effectiveValues = useMemo(() => {
  const out: Record<string, string> = {};
  for (const name of FIELD_ORDER) {
    const field = fields.find((f) => f.field_name === name);
    const extracted = field
      ? (isDateField(name) ? displayDate(field.value ?? '') : field.value ?? '') : '';
    out[name] = edits[name] ?? extracted;
  }
  return out;
}, [fields, edits]);
const filledCount = useMemo(() => filledCountFromValues(effectiveValues), [effectiveValues]);
```

- La «valeur effective» = brouillon s'il existe, sinon valeur extraite formatée. **La même map** nourrit chaque ligne ET le verrou 17/17 : ce qui est compté est exactement ce qui sera enregistré — aucune divergence possible.

Les lignes éditables

EditableFieldRow choisit l'éditeur par champ : WeightInput (pavé décimal + bascule d'unité kg<->g), TempRangeInput (min – max °C), PriceInput (montant + devise), sinon TextInput avec masque de date (number-pad, JJ/MM/AAAA), casse forcée d'estampille, hints neutres (validateDate...), tag bleu «À compléter» **réactif au brouillon** (disparaît à la frappe), chips d'historique (affichées seulement quand la ligne a le focus — suggestForField retourne [] pour les champs sans historique, aucun câblage par champ), et chip de suggestion allergène (seulement si la ligne est vide; l'appliquer = une édition humaine).

Doctrine visible : **pas de score de confiance à l'écran, pas de badge «à vérifier»** — la validation manuelle est l'unique source de vérité (Clean UI radicale, audit §6.2).

Les inputs à affixes ont un détail de conception : leur état local est **seedé une fois** depuis le brouillon (useState(seed.amount)) plutôt que contrôlé par lui — taper «1,» puis «5» ne doit pas être mangé par un round-trip de reformatage. Ils émettent la forme canonique ("5 kg", "0-4 °C", "8.95 EUR") via onChange.

handleSave — la sortie

```
const savedFields: ArticleField[] = run.fields.map((f) => {
  const draft = edits[f.field_name];
  const hasEdit = draft !== undefined;
  const rawNext = hasEdit ? (draft.trim() === '' ? null : draft.trim()) : f.value;
  const nextValue = rawNext !== null && isDateField(f.field_name) ? toIsoDate(rawNext) : rawNext;
  const changed = hasEdit && nextValue !== f.value; // comparaison en CANONIQUE
  return { field_name: f.field_name, value: nextValue, ...,
    validation_status: changed ? (nextValue == null ? 'missing' : 'present')
      : f.validation_status,
    edited: changed || undefined };
});
```

- Les dates repartent en **ISO** (toIsoDate, inverse exact du displayDate qui a seedé la ligne). Et la détection «édité» compare **canonique à canonique** : re-taper la même date en JJ/MM/AAAA n'est plus un faux «édité» (l'ancien code comparait un brouillon FR à une valeur ISO — bug d'audit réel).

```
await saveBackendArticle({ ..., fields: savedFields, saved_by: user, raw_extraction_run: run });

const corrections = savedFields.filter((f) => f.edited)
  .map((f) => ({ field_name: f.field_name, value: f.value }));

void (async () => {
  if (corrections.length > 0) await submitFieldOverrides({ ingestionId, fields: corrections });
  await enqueueConfirmIngestion({ ingestion_id: ingestionId });
  await drainOutbox();
})();

closingRef.current = true;
await completeScan(scan.id);
Haptics.notificationAsync(Haptics.NotificationFeedbackType.Success);
navigation.goBack();
```

- **Séquencement délibéré** : corrections d'abord, PUIS le confirm (le statut confirmed affirme «revue faite» — il ne doit jamais doubler les corrections qu'il valide), puis un drain pour rejouer les traînards des saves précédents. L'ensemble est fire-and-forget : le save local est déjà acquis, un hoquet de sync ne bloque jamais la boucle de capture.
- closingRef : handleSave va retirer le scan de la queue → l'effet-garde «scan disparu → goBack» se déclencherait **pendant** le propre goBack du save (double navigation). Le ref posé avant completeScan neutralise la course. En cas d'erreur du save : closingRef.current = false ré-arme le garde.
- Le bouton d'action raconte l'état : «Analyse en cours...» (pas ready) → «Compléter (n/17)» (désactivé) → «Enregistrer l'arrivée» → «Enregistrement...».
- submitFieldOverrides (§20 : enqueue + exécution immédiate par champ, force_gsl posé automatiquement pour les champs code-barres) pousse chaque correction vers le store **autoritaire** serveur — la fermeture de l'écart de cohérence HACCP (audit §4.2 : avant, les valeurs validées ne vivaient que dans le téléphone).

21.3 ArticleListScreen.tsx — l'accueil

- **Liste scopée au jour** (aujourd'hui par défaut); le panneau calendrier (heat-map) est le sélecteur de date; la recherche omni reste **globale** (taper une requête outrepassa le scope, l'effacer le restaure).
- Section «En cours» en ListHeaderComponent : les PendingScanCard (3 étapes : Photo envoyée / Extraction / À valider — mapping pur scanStepFromStatus), jauge n/17 champs vivante (run, interim et brouillon), carte cliquable dès l'extraction (la revue s'ouvre mi-analyse et se remplit en direct), erreurs avec Réessayer/Supprimer (retryScan/discardScan).
- **Perf de liste** : chaque ArticleCard a une hauteur FIXE (CARD_HEIGHT) → getItemLayout place

les lignes sans mesure — scroll $O(1)$ à des milliers de lots. La hauteur du header « En cours », elle, est **mesurée** (`onLayout`) et injectée comme offset constant — pas de constante codée en dur qui dériverait au premier changement de style (le commentaire le dit explicitement).

- `useFocusEffect` recharge les articles au retour sur l'écran ; export JSON/CSV et sign-out dans le cluster d'actions du header.

21.4 ArticleDetailScreen.tsx — la fiche produit

- Même `FIELD_ORDER` que la revue (cohérence de lecture bout en bout), photo fixe en tête avec la fiche qui scrolle par-dessus, champs **éditables en place** (crayon → édition) — y compris les champs GS1 (l'opérateur reste maître ; `force_gs1` automatique côté sync).
- Un save d'édition **re-enregistre** l'article (`updateBackendArticle` : mêmes champs remplacés, `saved_at` frais, `saved_by` = l'éditeur) et pousse l'override vers le backend, best-effort. Clean UI : pas de badge « Édité », pas de score — le qui/quand vit dans le méta d'en-tête.
- Chargement à la demande par `getArticleById` (et l'écran lui-même est lazy — §17.4).

21.5 Les composants notables

- `ScanStepper / PendingScanCard` : le stepper 3 étapes sobre — un état par étape (`done/active/pending/error`), un libellé au présent pour l'étape active, un `PulseDot` comme unique signe de vie (jamais de spinner).
- `ExtractionProgress` : la bannière de progression étagée pendant l'extraction, pilotée par `extractionStage(elapsedMs, ready, ocrDone)` — pur : `ready` gagne toujours ; `ocrDone` (le vrai transit serveur, Tier 3) épingle « Analyse » ; sinon estimation temporelle (« Lecture du texte » < 2 s < « Analyse »). L'attente bouge toujours.
- `SkeletonFieldList / SkeletonValue` : placeholders pulsants aux dimensions exactes des vraies lignes.
- `FrameOverlay` (les crochets du cadre, avec états `ready/capturing/barcodeFound/error`), `FlashOverlay` (l'éclair blanc via ref impérative), `CaptureButton`, `CaptureFab`, `PhotoViewerModal` (photo plein écran), `CalendarPanel` (rend les maths de `calendar.ts`), `ArticleCard`, `EmptyState`.

22. Les tests mobile (Jest)

src/__tests__/ — 20 suites, presque toutes sur les **services purs** (le choix d'architecture paie : la logique testable vit hors des composants).

- `__mocks__` : `async-storage.ts` (Map en mémoire), `expo-file-system.ts`, `secure-store.ts`, `random-values.ts` — les modules natifs sont substitués, les tests tournent en Node.
- `scanQueue.test.ts` (381 lignes) : le cycle complet `enqueue` → `outcome` → `poll` → `ready`, la `dédup replay`, la `réconciliation`, les erreurs — avec `_resetScanQueueForTests()` entre les cas.
- `ingestionPolling.test.ts` : classification des 12 états, `long-poll`, `anti-spin`, `abort`, `budgets`.
- `inputMasks.test.ts` (269 lignes) : chaque masque/parseur/hint, aller-retour ISO<->FR.
- `outbox.test.ts` / `outboxDrain.test.ts` : `backoff`, `retryabilité`, `claim`, `latch de drain`, `purge`.
- `gsl.test.ts`, `fieldCompleteness`, `fieldHistory`, `allergenSuggestions`, `faoDisplay`, `articleSearch`, `articleGrouping` (via `articleStore`), `calendar`, `dates`, `scanSteps`, `extractionStage`, `ingestionResult`, `fieldOverrideSubmit`, `apiUpload`, `api`, `latencyLog` : un fichier de test par service, au plus près des règles.

Commandes : `npm test` (ou `npm run test:watch`), `npx tsc --noEmit` pour le typage (la « CI » minimale du mobile).

Partie IV — Synthèse

23. Les 20 tips dev à emporter

Architecture

1. **Rends l'architecture exécutable.** Couches, pureté du domaine, indépendance des contextes : tout est un contrat `import-linter` qui casse la CI. Une règle non testée meurt.
2. **Ports partout où une techno peut changer** : store d'objets, DB, OCR, LLM, stockage d'articles mobile. Le port porte le *contrat* (docstring) ; l'adapter porte la techno. Les fakes de test satisfont le port par structure.
3. **Une seule composition root.** Seul `app/` (backend) assemble adapters + relais + consommateurs. La topologie du système se lit dans un répertoire.
4. **Corriger = ajouter.** Sur les données historiques (runs, audit, artefacts), jamais d'UPDATE : un nouveau run, une projection « le dernier gagne ». L'histoire complète reste auditable.

Fiabilité

5. **Transactional outbox** pour tout « écrire + publier » : l'événement est une ligne dans la même transaction. Relais `FOR UPDATE SKIP LOCKED`, dédup (`consumer, event_id`), DLQ + backoff exponentiel avec jitter (equal jitter si tes tests affirment la croissance).
6. **Donne une mémoire à l'échec** : `attempts, next_retry_at, last_error`, état terminal. Un échec sans état = retry infini invisible.
7. **Durable avant d'acquitter** : octets `fsync`és → transaction committée → 202. Le rituel fichier : `write-tmp` → `fsync` → `rename` atomique → `fsync` du répertoire.
8. **Idempotence à chaque étage**, avec des clés générées **une fois** : hash de contenu (`upload`), artefact-existant (`providers`), clé par requête (`writes` de revue), clés stables dans l'outbox mobile. Claim atomique = `INSERT ... ON CONFLICT DO NOTHING RETURNING`.
9. **Distingue échec métier et bug** : le premier s'enregistre et l'événement se consomme ; le second lève, part en DLQ avec `traceback`. Ne retente jamais un `TypeError`.
10. **Borne tout** : timeouts `providers` (120 s LLM, 30 s OCR), retries (3), long-poll (25 s), budget de poll mobile (5 min), taille d'`upload` (10 Mo). Ce qui n'est pas borné finira par l'atteindre.

Sécurité et intégrité

11. **L'intégrité en base, pas en discipline** : triggers `deny-mutation` (tous rôles), audit `SECURITY DEFINER` à contexte transactionnel obligatoire, `CHECK` qui redéclarent les invariants du domaine. Le code applicatif peut avoir des bugs ; la DB tient.
12. **Fail loudly, fail closed** : pas de défaut pour `DATABASE_URL`, le secret JWT, le provider OCR ; rôle inconnu → zéro scope ; panne de dépendance → 503 retrievable, pas un 500 flou.
13. **Anti-oracles** : hash factice vérifié pour les utilisateurs inconnus (timing), `compare_digest`, une seule erreur « invalid credentials ». Sanitise les exceptions qui pourraient porter un secret (from None).
14. **Le LLM est une entrée non fiable** : sortie structurée pour la *forme*, gate d'ancrage verbatim pour la *vérité*, l'entrée déclarée « donnée, pas instructions » dans le prompt, et un chemin de revue humaine pour tout défaut.
15. **Jamais de secret dans le bundle mobile** (`EXPO_PUBLIC_*` est extractible) ; token en Keychain/Keystore, jamais `AsyncStorage`.

Produit / UX

16. **Le pipeline sert la perception** : GS1 à T+0, préviews regex à ~2 s, long-poll pour la découverte instantanée, liste stable sans layout shift, bannière qui bouge toujours. La

latence se conçoit.

17. **L'aide n'invente jamais** : suggestions depuis l'historique validé ou une table réglementaire, affichées, jamais auto-appliquées; hints neutres non bloquants; pas de valeur calculée sur un champ de sécurité.
18. **Observabilité minimale mais complète** : logs JSON allow-listés, corrélation de bout en bout, compteurs de cache LLM, timings OCR/LLM, heartbeat de worker. Chaque « pourquoi c'est lent/cassé » a une réponse.
19. **Injecte le temps et l'aléa** (now=, rng=, today=) — les tests déterministes sont un choix d'API, pas un luxe.
20. **Écris les docstrings comme des ADR** : chaque fichier de ce projet explique *pourquoi* il existe et quels invariants il protège. C'est ce qui rend le codebase apprenable — ce cours n'aurait pas pu exister sans.

24. Checklists d'extension

Ajouter un champ d'extraction (bout en bout)

1. Prompt : ajouter au jeu fermé + règles de normalisation + few-shots → bump `_PROMPT_VERSION` (MAJOR si le jeu change) dans `claude_llm_provider.py`.
2. Migration : élargir le CHECK `field_name` de `extracted_field` en **superset** (modèle : 0011).
3. Backend : `_FIELD_NAMES` (adapter LLM), `FIELD_NAMES` (`override_field.py`), éventuellement `RuleSet.required_fields`.
4. Mobile : `FIELD_ORDER` (`fieldOrder.ts`), libellé FR (`fieldLabels.ts`), masque/hint éventuel (`inputMasks.ts`), historique éventuel (`HISTORY_FIELDS`).
5. Tests : gate (champ requis/ancrage), override, complétude mobile. Re-passer l'éval du prompt.

Ajouter un endpoint

1. Cas d'usage + port dans `application/` (exceptions typées documentant leurs statuts HTTP).
2. Adapter SQL dans `adapters/` (transaction unique, `set_audit_context` avant toute écriture audité, idempotence pensée).
3. Routeur : garde-fous transport, `require_scope`, commande, mapping d'erreurs vers le catalogue (append-only!).
4. `include_router` dans `http_app.py`; `scope` dans `ADMIN_SCOPES` si nouveau; tests HTTP via `bearer(...)`.

Ajouter un consommateur d'événement

1. Classe avec `consumer_name`, `event_type(s)`, `__call__(msg, conn)` — **écrire uniquement via conn** (la transaction du worker).
2. Idempotence gratuite via `processed_event`; ne lever que pour les vrais bugs; enregistrer les échecs métier et retourner.
3. Enregistrer dans `app/domain_wiring.py` (ou un wiring dédié). Événements sortants : INSERT outbox sur `conn`.

Ajouter une table

- Historique? → append-only : triggers `deny_mutation` + `audit_on_insert`, partitionnement si volumineuse, grants sans UPDATE/DELETE. Mutable? → audit sur INSERT (et UPDATE si transitions d'état). Toujours : `correlation_id/trace_id`, CHECK pour les invariants, docstring de migration expliquant le pourquoi.

25. Glossaire

Terme	Définition
AI (GS1)	Application Identifier — préfixe numérique typant un segment du code-barres (01=GTIN, 10=lot, 17=DLC)
Append-only	Table sans UPDATE/DELETE (triggers); l'histoire est immuable, corriger = ajouter
At-least-once	Garantie de livraison : ≥ 1 fois, jamais 0; à coupler avec des consommateurs idempotents
Backoff (exponentiel, jitter)	Délai de retry croissant (2^n), plafonné, avec part aléatoire anti-troupeau
Bounded context	Module métier étanche (ingestion, haccp...) — pas d'imports croisés, événements + IDs
Composition root	L'unique endroit où les implémentations concrètes sont assemblées (app/)
DLC / DDM	Date Limite de Consommation (sécurité) / Durabilité Minimale (qualité) — jamais interchangeables sans warning
DLQ	Dead-Letter Queue — parking des événements qui ont épuisé leurs retries
Gate (anti-fabrication)	<code>evaluate()</code> : chaque valeur LLM doit être ancrée verbatim dans l'OCR, sinon null + revue
HACCP	Hazard Analysis Critical Control Points — méthode de maîtrise des dangers alimentaires (ici : DLC, température)
HITL	Human-In-The-Loop — la revue/correction humaine comme étape du pipeline
Idempotence	Refaire l'opération ne change pas le résultat; la base du retry sûr
Long-poll	GET que le serveur retient jusqu'à un changement d'état (ou expiration) — découverte ~ 0 ms
Outbox (transactional)	L'événement écrit comme ligne DB dans la transaction du producteur, relayé ensuite
Poison pill	Message qui fait toujours échouer son handler — sans DLQ, il bloque la file pour toujours
Port / Adapter	Interface définie par le métier / implémentation technique interchangeable
Principal	L'identité authentifiée (actor_id, principal, scopes) résolue à la porte HTTP
Problem+json (RFC 9457)	Format standard de réponse d'erreur HTTP; ici enrichi d' <code>error_code</code> stable et retrievable
Prompt caching	Facturation réduite ($\sim 0.1 \times$) du préfixe statique identique entre appels LLM

Terme	Définition
SECURITY DEFINER	Fonction PostgreSQL exécutée avec les droits de son propriétaire — à durcir (owner minimal, search_path vide)
SKIP LOCKED	SELECT FOR UPDATE qui saute les lignes verrouillées — la primitive des files de jobs en SQL
Structured outputs	Mode API LLM garantissant une sortie conforme à un JSON Schema
useSyncExternalStore	Hook React pour s'abonner à un store externe (subscribe + getSnapshot stable)